

QASP: A Modular, differentiable and distributed software package for quantum machine learning

André Sequeira
[TODO: add affiliation, institutional address, e-mail, and ORCID]

QASP v0.1.0 — September 2026

Abstract

We present QASP (Quantum Agent Software Package), an open-source Python framework for modular, differentiable, and distributed quantum machine learning. Its foundation is a set of configuration-driven building blocks for feature maps, parameterized ansatzes, measurements, devices, gradient methods, classical decoding, and publication-ready circuit drawings. These primitives support five high-level modules: reinforcement learning with configurable quantum policies, action-value functions, and training loops; quantum kernels with fidelity, projected-feature, alignment, and Fisher constructions; parameterized quantum classifiers with probability or expectation readouts; generative learning with quantum-circuit Born machines; and combinatorial optimization with QUBO representations. Each module retains explicit contracts for outputs, gradients, sampling, and decoding, enabling controlled comparisons across models and execution strategies. High-performance experimentation is supported through optimized state-vector, tensor-network, stabilizer, Pauli-propagation, circuit-cutting, and Kokkos/MPI execution paths, together with analytical workload and distributed-job planning. QASP also provides finite-sample cost and output-concentration diagnostics for trainability studies and exploratory utilities for sampling policy outputs and inspecting heuristic generator closures. Together, these layers provide a reproducible research platform spanning model construction, task-level learning, scalable simulation, and diagnosis.

Repository: https://github.com/andre-sequeira10/QASP **License:** MIT **Language:** Python ≥ 3.10

Contents

1 Introduction **3**

2 Statement of need and position in the software ecosystem **4**
2.1 Target users and research questions . . . 4
2.2 Design principles . . . 4

3 QASP building blocks **5**
3.1 Composable quantum models . . . 5
3.2 Gradient-based optimization . . . 9
3.3 High-performance and approximate simulation . . . 11
3.4 Distributed workload planning and execution . . . 14

4 Reinforcement learning **14**
4.1 Policy-based reinforcement learning . . . 16
4.1.1 Policy and action-readout primitives . . . 16
4.1.2 Policy contract at the trainer boundary . . . 19
4.1.3 Windy Target Navigation . . . 20
4.1.4 Matched trainability-aware policy experiments . . . 21
4.1.5 Inspectable policy construction and circuit drawings . . . 21
4.1.6 REINFORCE . . . 25
4.1.7 Executable CartPole comparison . . . 25
4.1.8 Contextual bandits and AWR . . . 27
4.1.9 PPO and TorchRL . . . 27
4.2 Value-based reinforcement learning . . . 28
4.2.1 Quantum Q-functions . . . 28
4.2.2 Deep Q-learning: replay, targets, and exploration . . . 30
4.2.3 CartPole value-readout study . . . 34

5 Quantum kernels **35**
5.1 Shared construction and factory contract . . . 36
5.2 One projector-configurable fidelity kernel . . . 37
5.3 Projected expectation-feature kernel . . . 38
5.4 Trainable kernels and centered target alignment . . . 38
5.5 Fisher-information kernel . . . 40
5.6 Executable IQP example and validation . . . 40

6 Parameterized quantum classifiers **41**
6.1 Shared construction and factory contract . . . 42
6.2 Born-probability classification . . . 42
6.3 Expectation-score classification . . . 43
6.4 Objectives, training, and evaluation . . . 44

7 Generative learning **46**
7.1 Quantum-circuit Born-machine abstraction . . . 46
7.2 Probability and expectation objectives . . . 47
7.3 Training, execution, and sampling contracts . . . 48
7.4 Executed three-qubit example . . . 49
7.5 MPI and high-performance use . . . 49

8 Combinatorial optimization **49**
8.1 QUBO and Ising problem contract . . . 51
8.2 The QAOA family . . . 51
8.3 Pauli correlation encoding . . . 53
8.4 Classical control, results, and executable validation . . . 53

9 Trainability and structural diagnostics **55**

10 Validation and reproducibility **59**
10.1 Recommended experiment record . . . 61
10.2 Benchmark protocol for the release . . . 61

11 Limitations and responsible interpretation **62**

12 Availability, extensibility, and research impact **64**

13 Conclusion **64**

A Configuration and API reference **68**

B Ancilla parity policy example **69**

C Exact majority–modulo policy example **69**

D Exact multi-action recursive parity **70**

E Windy navigation with an exact majority policy **71**

F Release checklist **71**

1 Introduction

Variational quantum algorithms place a parameterized quantum circuit (PQC) inside a classical optimization loop. The same pattern underlies quantum neural networks, quantum policies, and circuit Born machines: classical data are encoded into a quantum state; a trainable unitary transforms the state; measurements define a scalar, vector, or distribution-valued model output; and a classical optimizer updates the parameters. Although the diagram is simple, a reproducible implementation must coordinate circuit topology, data re-uploading, observable design, measurement shots, differentiation, simulator capabilities, batching, and the conventions by which bit strings become labels or actions. Small differences in any of these choices can change both the represented hypothesis class and its trainability.

The problem is especially acute in quantum reinforcement learning (QRL). A policy must produce a valid action distribution at every environment step, and a single policy-gradient update may require many circuit evaluations. The quantum readout may be a full Born distribution, a family of Pauli expectations, or a sampled bit string; each choice has different execution and gradient costs. Experimental code frequently entangles these decisions with a particular environment or training loop, making controlled comparisons difficult. At the same time, gradient concentration and barren plateaus can make apparently minor choices—observable locality, connectivity, parameter initialization, and feature scaling—scientifically decisive [1, 2].

Qasp addresses this integration layer. It builds on PennyLane's device-independent circuit and differentiation model [3] and PyTorch's automatic-differentiation and optimization ecosystem [4]. Rather than replacing either framework, Qasp contributes domain-specific abstractions for quantum agents and adjacent learning workflows. In particular, release 0.1.0 provides:

- composable angle and neural feature maps, IQP and general Pauli-rotation ansatz templates, a measurement-aware `CircuitBuilder`, and configuration-resolved Matplotlib or publication-ready Quantikz circuit drawings;
- a vectorized decoded-Born policy core shared by probability aggregation and sample decoding, with contiguous, recursive-parity, hidden-parity, and majority–modulo action maps;
- exact multi-action parity reconstruction from logical Pauli moments, exact sparse majority action projectors, expectation-value softmax policies, and a learnable measurement-selection model;
- gradient-based training through backpropagation, adjoint differentiation, parameter-shift, finite differences, SPSA, and differentiable Pauli propagation, together with tested small-system Fisher-preconditioned Born-machine updates;
- REINFORCE, advantage-weighted contextual-bandit training, PPO utilities, replay-based Double DQN, Gymnasium interoperability, TorchRL adapters, and seeded control benchmarks;
- fast Pauli propagation using packed symplectic keys, plus accelerated state-vector and Kokkos execution inherited through PennyLane devices;
- one projector-configurable fidelity class for global and localized overlap similarities, projected expectation-feature kernels, centered target alignment, and a Fisher-information kernel;
- binary and multiclass parameterized quantum classifiers with Born or expectation readouts, configurable class decoders, differentiable losses, training utilities, and classification metrics;
- QUBO and Ising problem construction, standard and multi-angle QAOA, relaxation-based warm starts, recursive variable elimination, fixed linear-ramp schedules, Pauli correlation encoding, finite-shot solution sampling, and unified optimization results;
- explicit-loss quantum-circuit Born-machine training, policy-output variance sampling, finite-sample concentration diagnostics, and exploratory generator-list and Lie-closure-size utilities;
- workload inspection, analytical Deucalion memory/placement planning, Slurm-script rendering, and MPI-aware execution callbacks that keep resource estimates distinct from measured runtime.

This is a software paper, not an empirical claim of quantum advantage. Its contribution is a reusable experimental substrate that makes modelling and execution choices explicit and allows like-for-like comparisons. The presentation follows the pattern established by successful quantum-software papers: a statement of need, a concrete architecture, executable usage examples, backend boundaries, and an honest account of validation and limitations [5–7].

Table 1: Positioning of Qasp relative to the principal software layers it uses or complements. A check mark indicates a primary, user-facing responsibility rather than every technically possible use.

Project	Device-agnostic circuits	Autodiff tensors	RL infrastructure	QRL policy decoders	Pauli-propagation path
PennyLane	✓	via interfaces	limited	–	specialized
PyTorch	–	✓	foundational	–	–
Gymnasium	–	–	environments	–	–
TorchRL	–	✓	✓	–	–
TensorCircuit	✓	✓	limited	–	–
Qasp	via PennyLane	via PyTorch	QRL-focused	✓	✓

2 Statement of need and position in the software ecosystem

2.1 Target users and research questions

Qasp is aimed at researchers who need to ask questions such as:

- How does a Born policy compare with an expectation-value policy under the same ansatz and optimizer?
- How do local, parity, hidden-qubit, or majority-based action decoders affect variance and trainability?
- Which conclusions survive a change from exact state-vector simulation to a tensor-network, stabilizer, distributed Kokkos, or approximate Pauli-propagation backend?
- Can input-scaling weights, circuit angles, observable heads, and value baselines be trained in one PyTorch computation graph?
- How do finite-sample output concentration and exploratory generator closure sizes change with connectivity, depth, and readout choice?
- Can the same circuit primitives be reused for a policy, a quantum kernel, a supervised classifier, a Born machine, and a combinatorial optimizer without obscuring their task-specific measurements?
- How do optimized, warm-started, recursive, fixed-schedule, and correlation-encoded QUBO solvers compare under one result and drawing contract?

Existing packages solve important parts of this workflow. PennyLane provides a general differentiable quantum-programming layer; PyTorch supplies tensor automatic differentiation and optimizers; Gymnasium standardizes environment interaction [8]; and TorchRL supplies composable data structures, collectors, actors, and losses [9]. High-performance frameworks such as TensorCircuit emphasize differentiable simulation and accelerator execution [7]. Qasp deliberately composes with these projects. Its narrower contribution is a shared experimental layer for configurable PQC construction, research-oriented policy and classifier readouts, quantum kernels, QUBO optimization algorithms, task-level training utilities, structural diagnostics, and differentiable Pauli propagation behind consistent high-level interfaces.

2.2 Design principles

Five principles organize the package.

1. **Separate representation from execution.** Feature maps, ansatz templates, measurements, devices, and gradient estimators are configured independently.
2. **Make decoder semantics explicit and invariant.** A basis measurement does not become an action or class distribution until a documented decoder maps basis states to task labels; probability aggregation, expectation reconstruction, sparse projectors, and direct samples must realize the declared map.
3. **Preserve end-to-end gradients.** Trainable input scales, quantum angles, and optional classical heads remain visible to PyTorch; the differentiable Pauli engine retains the graph for surviving terms.
4. **Fail on incompatible contracts.** Specialized readouts validate shots, device, differentiation, batching, MPI, and circuit-cutting requirements rather than silently switching semantics.
5. **Treat approximation as a controlled experiment.** Pauli-weight, coefficient, and term-count thresholds are explicit parameters, not hidden defaults in the scientific result.

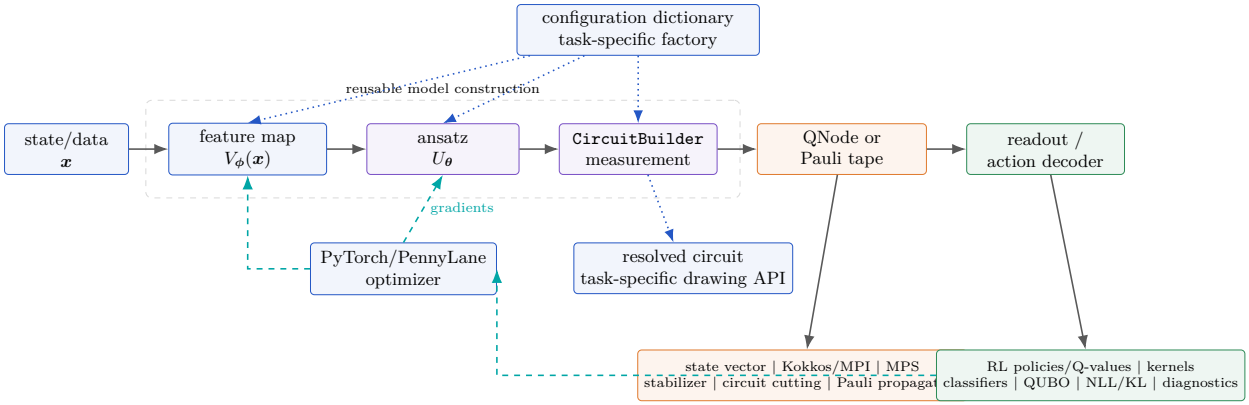


Figure 1: High-level QASP architecture. Solid arrows denote the forward path, dashed cyan arrows differentiable feedback, and dotted blue arrows configuration or inspection. Task-specific factories resolve the same components exposed by the low-level builder, while `draw_model`, `draw_qfunction`, `draw_kernel`, `draw_classifier`, and `draw_optimization` visualize their executable circuits. One construction can feed different task heads and backends.

3 QASP building blocks

The package is organized around four compositional boundaries: feature maps encode classical data, ansatz templates define trainable circuit structure, `CircuitBuilder` binds measurements and execution, and high-level task modules turn circuit outputs into learning objects. The orange feature boxes throughout the paper identify these public abstractions at the point where each is introduced; the subsequent reinforcement-learning, quantum-kernel, parameterized-classifier, generative-learning, and combinatorial-optimization sections specialize the same building blocks to their task-level contracts.

Figure 1 shows the main data flow. A user can enter through a low-level circuit-builder interface or through the configuration-driven model, kernel, classifier, or optimization constructors. All routes converge on a circuit, a measurement contract, and an execution strategy. Task heads then interpret the quantum result as action probabilities, classifier scores, class probabilities, a generative distribution, a kernel feature, or a QUBO solution.

QASP feature: `create_model` and `draw_model`

`create_model(policy_class, config, n_qubits)` is the high-level model factory. It resolves string-valued feature maps, ansatz templates, policy classes, measurements, devices, and differentiation methods into one PyTorch module. `draw_model` accepts that resolved module and renders the circuit that will actually execute. Its `output` option selects an interactive Matplotlib drawing or editable, vector-quality Quantikz source, making the configuration dictionary both a model specification and an inspectable experiment record.

3.1 Composable quantum models

Let n be the number of model qubits, $\boldsymbol{x} \in \mathbb{R}^d$ an input, $\boldsymbol{\phi}$ trainable encoding parameters, and $\boldsymbol{\theta}$ variational circuit parameters. The common QASP model state is

$$|\psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})\rangle = U_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})|0\rangle^{\otimes n}, \qquad U_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) = \prod_{\ell=1}^{L} W_{\boldsymbol{\theta}_\ell} V_{\boldsymbol{\phi}_\ell}(\boldsymbol{x}), \tag{1}$$

where the order and presence of the first or repeated V blocks are configurable. This factorization is deliberately schematic: a template may place encoders before selected layers, omit final entanglers, or use explicit Pauli words rather than a uniform gate family.

3.1.1 Feature maps

`AngleEmbedding` supports two input modes. If `rotations` is a string such as `"YZ"`, each character selects a PennyLane angle-embedding rotation over the model wires. If it is a list such as `"ZZ 0 1"`, entry x_j controls

Table 2: Module map for release 0.1.0. "Experimental" denotes a research-facing API that may evolve; it does not imply that the module is unused.

Module	Responsibility	Status
`qasp.circuits.constructor`	Circuit assembly; probability, state, sample, expectation, and variance measurements; device and gradient selection; adaptive measurements; TorchLayer creation.	Core
`qasp.circuits.templates`	`IQPAnsatz` and `AngleAnsatz`; topology, rotations, entanglers, re-uploading, and parameter-shape calculation.	Core
`qasp.circuits.tikz`	Quantikz export from builders, QNodes, quantum functions, or tapes; symbolic labels, semantic gate colors, wire labels, standalone documents, and repeated-layer compression.	Core
`qasp.circuits.uqc`	Retained historical UQC function with an older broadcast interface and unresolved imports; not a supported template path.	Legacy
`qasp.feature_maps.templates`	`AngleEmbedding` and `NeuralEmbedding`, with optional input scaling and entangling patterns.	Core
`qasp.devices`	Shared PennyLane device creation with explicit backend and MPI option propagation; no silent serial fallback.	Core
`qasp.rl.model`	Configuration resolution, `create_model`, and circuit visualization through `draw_model`.	Core
`qasp.rl.policies`	Born, enhanced Born/variance, softmax, and adaptive-measurement models; one vectorized decoder for probability and sample paths; logical-moment reconstruction; exact sparse projectors.	Core
`qasp.rl.qfunctions`	Differentiable `PQDQN` action-value models; shared and per-action features, parity and majority support presets, configurable affine bias and preset size, QNode or QASP Pauli execution, and configuration/drawing factories.	Research
`qasp.rl.pp_sim`	Packed-key, non-differentiable fast truncated Pauli propagation.	Experimental
`qasp.rl.differentiable_pp_sim`	PyTorch-connected Pauli propagation with differentiable retained coefficients.	Experimental
`qasp.rl.torchrl`	Episode-batched REINFORCE, replay-based Double DQN, contextual-bandit AWR, PPO utilities, value baselines, Gymnasium environment, and TorchRL adapter.	Research
`qasp.rl.trl`	Historical TorchRL PPO prototype retained for compatibility; the maintained trainer implementations live in `qasp.rl.torchrl`.	Legacy
`qasp.rl.envs`	Seed-reproducible `WindyTargetNavigationEnv` with three or four thrust actions, persistent wind, inertial dynamics, and waypoint rewards.	Research
`qasp.rl.metrics`	Policy-output variance sampling and optional Ray parallelism, plus legacy exploratory routines that return heuristic generator lists or Lie-closure sizes rather than certified circuit algebras.	Experimental
`qasp.diagnostics.barren_plateau`	Model-agnostic finite-sample cost concentration for classifiers, trainable kernels, and RL policies.	Research
`qasp.qkernels.quantum_kernels`	One projector-configurable `FidelityKernel` for global and localized overlap similarities, projected expectation-feature kernels, trainable ansatz support, and differentiable Gram matrices.	Core
`qasp.qkernels.factory`	Configuration-driven `create_kernel` and `draw_kernel` entry points.	Core
`qasp.qkernels.kta`	Centered kernel–target alignment and gradient-based kernel fitting.	Research
`qasp.qkernels.kernels`	Small-system Fisher-score Gram matrices for bit-string data; exhaustive basis enumeration, zero-probability sensitivity, and an unused legacy normalization option.	Experimental
`qasp.classifiers.models`	Binary and multiclass `PQCClassifier` models with probability or expectation readouts.	Core
`qasp.classifiers.decoders`	Contiguous, recursive-parity, and majority–modulo class-probability aggregation and sample decoding.	Core
`qasp.classifiers.losses, training`	Differentiable supervised objectives and minibatch PyTorch training.	Core
`qasp.classifiers.metrics`	Confusion-matrix, accuracy, F1, calibration, ROC-AUC, and PR-AUC evaluation.	Core
`qasp.genlearn.models`	`PQCBM` construction and training with Torch or Autograd, probability NLL or expectation-mixture MMD, optional fixed conditioning features, direct device sampling, and a deprecated `BornMachine` compatibility wrapper.	Research
`qasp.genlearn.losses`	Negative log likelihood, Kullback–Leibler helpers, Gaussian MMD^2, binary-index conversion, exact or sampled Pauli-mask mixtures, and finite-shot corrections.	Research
`qasp.genlearn.factory, drawing`	Configuration-driven `create_qcbm` and text, Matplotlib, or publication-ready Quantikz circuit export.	Core
`qasp.optimization.problem`	QUBO construction, exact energy evaluation, QUBO–Ising conversion, Hamiltonians, and brute-force references.	Core
`qasp.optimization.qaoa`	Standard, multi-angle, warm-start, and fixed linear-ramp QAOA with analytic or finite-shot execution.	Research
`qasp.optimization.recursive`	Correlation-driven recursive QAOA reduction, exact cutoff solution, and assignment reconstruction.	Research
`qasp.optimization.pce`	Pauli correlation encoding with dictionary-defined QASP ansatz circuits and soft-spin decoding.	Research
`qasp.optimization.optimizers, drawing, results`	SciPy optimizer adapters, common result containers, and text, Matplotlib, or Quantikz circuit drawings.	Core
`qasp.hpc.workload, planner`	Side-effect-free workload inspection and conservative Deucalion memory/-placement estimates from a dated hardware profile.	Research
`qasp.hpc.slurm, run`	Slurm-script rendering and MPI-aware execution callbacks; scripts are generated but never submitted or used to transfer/install code.	Research

the corresponding Pauli rotation

$$R_{P_j}(x_j) = \exp\left(-\frac{i}{2} x_j P_j\right), \qquad P_j \in \{I, X, Y, Z\}^{\otimes n}. \tag{2}$$

With input scaling, x_j is replaced by $\phi_j x_j$. Optional CNOT or CZ patterns can be specified as a chain, ring, all-to-all graph, explicit edge list, or callable topology.

QASP feature: `AngleEmbedding`

A direct data encoder for angle and Pauli rotations. It maps input coordinates to configured generators, optionally learns multiplicative input scales, and supports reusable entangling patterns. Use it when input coordinates already have a meaningful one-to-one relationship with circuit rotations.

`NeuralEmbedding` assigns a trainable bias-free linear projection to each rotation site. For wire or Pauli word j it applies

$$R_{P_j}(\langle \boldsymbol{\phi}_j, \boldsymbol{x} \rangle)\,. \tag{3}$$

Thus a small observation vector can drive many circuit locations without a separate classical network. Both $\boldsymbol{\phi}$ and $\boldsymbol{\theta}$ are registered in the resulting `qml.qnn.TorchLayer` and can be assigned different optimizer groups. The current reduction is single-sample; batched workloads should evaluate samples sequentially rather than assume broadcast support.

QASP feature: `NeuralEmbedding`

A trainable linear quantum embedding. Every configured rotation receives a learned projection of the complete classical input, allowing a low-dimensional state to drive many circuit sites while preserving gradients to the projection weights. It is the default embedding in the CartPole and windy-navigation examples.

3.1.2 Ansatz families

`AngleAnsatz` is the general template. Each layer contains an arbitrary sequence of single- or multi-qubit Pauli rotations, followed optionally by fixed CNOT or CZ entanglers. Rotation strings such as `"ZY"` broadcast one-qubit rotations over all wires; lists such as `["Y 0", "ZZ 0 1"]` define an explicit generator set. The template exposes data re-uploading [10], custom topology, optional initial Hadamards, and control over initial and final feature-map or entangler blocks.

QASP feature: `AngleAnsatz`

The general layered ansatz abstraction. Users select local or multi-qubit Pauli generators, layer count, feature-map placement, re-uploading, entangler type, and graph topology. Its explicit parameter-shape contract is consumed by `CircuitBuilder` when constructing a differentiable Torch layer.

`IQPAnsatz` constructs commuting phase-and-entangler blocks inspired by instantaneous quantum polynomial circuits. For one layer with local terms and edge set E, its implemented form is

$$H^{\otimes n} \left(\prod_{j=1}^{n} R_{Z_j}(\theta_j) \right) \left(\prod_{(j,k)\in E} R_{Z_j Z_k}(\theta_{jk}) \right) H^{\otimes n}, \tag{4}$$

with repeated blocks, optional local rotations, and optional feature-map insertion. The explicit parameter-shape methods on both ansatz classes prevent a common class of silent indexing errors when constructing Torch layers.

QASP feature: `IQPAnsatz`

A graph-configurable IQP template built from commuting local and edge phase generators. It supports repeated layers, optional local terms, feature-map insertion, and string or explicit-edge topologies. The same abstraction feeds trainability analysis, exact simulators, Pauli propagation, and the matched RL experiments.

`UQC` records an earlier functional experiment with affine data angles. It retains unresolved imports and an obsolete broadcast convention, so it is documented for provenance rather than presented as an executable release template. New models should implement the class-based ansatz contract.

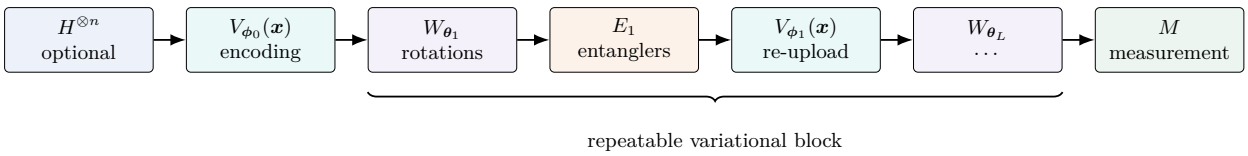


Figure 2: A configurable QASP circuit. The initial Hadamards, feature-map placement, re-uploading, generator sequence, topology, and final entangler are independently selectable.

QASP feature: Legacy `UQC`

A historical functional circuit kept to make older experiments interpretable. Its current source is not a supported construction path; `AngleAnsatz` and `IQPAnsatz` are the maintained extension points.

3.1.3 CircuitBuilder and measurement contracts

`CircuitBuilder` separates circuit definition from execution. `build()` returns a plain quantum function; after the ansatz and measurement are fixed, `compile()` creates the device-bound QNode, and `create_torchlayer()` allocates its trainable weights. Merely calling `set_feature_map()` stores metadata: the encoding must also be attached to the ansatz that queues the gates. The measurement contract supports full or marginal probabilities, state vectors, samples, expectation values, and the explicit name `"variance"`. Observable strings such as `"ZZ 0 1"` are parsed into Pauli words; constructed PennyLane operators are accepted directly, enabling sparse action projectors.

The adaptive builder variant accepts both data and a run-time measurement specification. It is used by `LearnablePQCBorn` to select a measurement subset from the observation. Keeping this logic in the builder ensures that policies and non-RL models use the same circuit conventions.

QASP feature: `CircuitBuilder`

The low-level assembly and execution boundary. It combines an ansatz with a measurement contract, returns a plain quantum function from `build()`, binds a backend and differentiation method in `compile()`, and then can allocate a `qml.qnn.TorchLayer`. Probability, state, sample, expectation, variance, adaptive-measurement, and sparse-observable outputs share this interface.

QASP feature: `circuit_to_tikz`, `save_tikz`, and paper-ready drawings

Any `CircuitBuilder`, PennyLane QNode, quantum function, or recorded tape can be exported as editable Quantikz source. The same path is exposed by `CircuitBuilder.to_tikz`, `draw_model(output="tikz")`, and `draw_kernel(output="tikz")`. Users may request symbolic parameters, custom wire labels and semantic gate colors, a complete standalone document, or compressed repeated layers in which the first layer remains explicit and later layers become register-wide $U(\boldsymbol{\theta}_\ell)$ blocks. The output is vector LaTeX, so circuit typography and line weights follow the paper rather than a raster screenshot. Measurement meters represent quantum readouts; any classical affine head, decoder, or reconstruction coefficients remain external and must be stated in the caption or accompanying equations.

The legacy initializer name `discrete_0_pi_over_2` should not be read as a mathematical contract: the present builder samples the two values 0 and π. Throughout this paper we therefore report the actual initialization values or use an unambiguous initializer name.

```python
import pennylane as qml
import torch
from qasp import AngleEmbedding, AngleAnsatz, CircuitBuilder

feature_map = AngleEmbedding(
    n_qubits=4, features=4, rotations="YZ", input_scaling=True
)
ansatz = AngleAnsatz(
    n_qubits=4,
    n_layers=3,
    rotations="ZY",
    feature_map=feature_map,
    reuploading=True,
    include_imprimitive=True,
```

```
    imprimitive="CZ",
    imprimitive_pattern="ring",
)

builder = CircuitBuilder(n_qubits=4)
builder.set_ansatz(ansatz)
builder.set_measurement(
    "expectation",
    wires=range(4),
    observables=["Z 0", "Z 1", "ZZ 0 1", "ZZ 2 3"],
)
builder.build()
builder.compile(device="lightning.qubit", diff_method="adjoint")
quantum_layer = builder.create_torchlayer(initialization="small_normal")

x = torch.tensor([0.1, -0.2, 0.3, 0.5])
loss = quantum_layer(x).square().mean()
loss.backward()  # gradients for circuit and input-scaling parameters
```

Listing 1: Low-level construction of a differentiable quantum layer. The snippet uses public release-0.1.0 interfaces.

3.2 Gradient-based optimization

Gradient flow is a cross-cutting concern in QASP, not a trainer-specific feature. The package supports three distinct but interoperable layers: PennyLane differentiates device-bound QNodes; `qml.qnn.TorchLayer` registers the resulting parameters with PyTorch; and task-level code differentiates RL, kernel, classifier, or generative objectives. The optimization module uses the same PennyLane derivative layer for QAOA Hamiltonian expectations and hands values or Jacobians to a reusable SciPy optimizer adapter.

3.2.1 Device-mediated derivatives

The `diff_method` argument is forwarded to PennyLane. Depending on device and measurement support, users can select backpropagation, adjoint differentiation, parameter-shift, finite differences, SPSA, or PennyLane's `"best"` dispatch. For a gate generated by a two-eigenvalue Pauli operator, the familiar parameter-shift identity is

$$\frac{\partial f}{\partial \theta_j} = \frac{1}{2}\left[f\left(\theta_j + \frac{\pi}{2}\right) - f\left(\theta_j - \frac{\pi}{2}\right)\right], \tag{5}$$

which permits hardware-compatible analytic gradients [11]. Adjoint differentiation instead propagates derivative information through an analytic simulator and is particularly effective when many parameters feed a small number of expectation values. SPSA uses simultaneous random directions and can reduce circuit-query dependence on parameter count at the cost of stochastic derivative noise [12].

These methods are not interchangeable for every measurement. QASP therefore keeps the requested measurement and derivative method visible and adds validation for specialized combinations such as sparse-projector adjoints. Table 3 summarizes the intended usage; the installed PennyLane device remains the final authority on a particular combination.

QASP feature: Differentiable QNodes and Torch layers

`CircuitBuilder.compile` binds the selected PennyLane device and derivative method, while `create_torchlayer` registers circuit and embedding parameters with PyTorch. Backpropagation, adjoint, parameter-shift, finite-difference, and SPSA estimators therefore meet the same task-level optimizer interface, subject to device and measurement support.

For QAOA, `diff_method="auto"` selects adjoint differentiation under analytic execution and parameter shift when the training device has finite shots. A derivative-aware optimizer such as BFGS receives `QAOA.gradient` when `use_gradients=True`; derivative-free methods such as COBYLA instead minimize the same analytic or noisy Hamiltonian expectation directly. The shot count used to report the final bit-string distribution is configured independently from the training shot count.

Table 3: Gradient and optimization paths exposed by QASP. "Analytic" refers to shots set to `None`; finite-shot compatibility depends on the selected PennyLane device and estimator.

Path	Typical outputs	Strength	Principal boundary
Backpropagation	probabilities or expectations	Native autodiff through simulator	Simulator-only; memory can scale as a state vector
Adjoint	expectations	Many-parameter gradients without two shifts per parameter	Analytic expectation-oriented devices
Parameter-shift	probabilities or expectations	Analytic, hardware-compatible rule	Circuit evaluations generally grow with differentiated gates
Finite difference	general scalar/vector outputs	Broad fallback and gradient checking	Step-size bias and cancellation
SPSA	stochastic objectives	Query-efficient directions in high dimension	Higher estimator variance
Natural gradient	Born-machine loss	Geometry-aware preconditioning with quantum/classical Fisher matrix	Fisher construction and linear solve
Differentiable PP	Pauli expectations	Backpropagation through retained operator paths	Approximation and supported-gate restrictions

3.2.2 Readout-aware gradient equivalence

Let $\boldsymbol{r}_{\boldsymbol{\Theta}}(s)$ denote the raw circuit output and g the differentiable readout map, so that $\boldsymbol{\pi}_{\boldsymbol{\Theta}}(s) = g(\boldsymbol{r}_{\boldsymbol{\Theta}}(s))$. For a scalar learning objective $\mathcal{L}$, the hybrid chain rule is

$$\nabla_{\boldsymbol{\Theta}} \mathcal{L} = J_{\boldsymbol{r}}(\boldsymbol{\Theta})^{\mathsf{T}} J_g(\boldsymbol{r})^{\mathsf{T}} \nabla_{\boldsymbol{\pi}} \mathcal{L}. \tag{6}$$

The readout Jacobian is explicit in every QASP path. For the full-probability route, $g(\boldsymbol{p}) = D\boldsymbol{p}$ and $J_g = D$. For the moment route, Equation (24) gives

$$\frac{\partial p(z)}{\partial \theta_j} = 2^{-m} \sum_t (-1)^{t \cdot z} \frac{\partial \mu_t}{\partial \theta_j}, \tag{7}$$

so the FWHT commutes with differentiation. For a sparse action projector, linearity gives the exact identity

$$\frac{\partial}{\partial \theta_j} \langle \Pi_a \rangle = \sum_{z:\, d(z)=a} \frac{\partial p(z)}{\partial \theta_j}. \tag{8}$$

Accordingly, full probabilities, logical moments, and sparse projectors must produce the same gradients whenever they implement the same d and use exact analytic execution. QASP treats this equality as a regression invariant, not as a backend assumption. The only extra operation in the sparse output path is roundoff protection at the probability-simplex boundary.

3.2.3 Hybrid parameter groups

A constructed Torch layer may contain circuit parameters $\boldsymbol{\theta}$ and input-scaling parameters $\boldsymbol{\phi}$. Softmax policies can add a classical readout $(W, \boldsymbol{b})$ and a scale β; RL trainers may add a value network. QASP exposes these as ordinary PyTorch trainables, so a user can select separate learning rates,

$$\boldsymbol{\Theta} \leftarrow \boldsymbol{\Theta} - \eta \widehat{\nabla_{\boldsymbol{\Theta}} \mathcal{L}}, \qquad \boldsymbol{\Theta} = \{\boldsymbol{\theta}, \boldsymbol{\phi}, W, \boldsymbol{b}, \beta\}. \tag{9}$$

The training utilities explicitly collect nonstandard leaf tensors from a PennyLane Torch layer when needed. The TorchRL-oriented trainer additionally verifies that a nonzero finite gradient reaches at least one trainable, and the RL optimizers optionally clip the global gradient norm.

3.2.4 Small-system natural-gradient branch

The `PQCBM` natural-gradient option is motivated by the regularized update

$$(F(\boldsymbol{\theta}) + \lambda I)\, \boldsymbol{v} = \nabla_{\boldsymbol{\theta}} \mathcal{L}, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \boldsymbol{v}. \tag{10}$$

Table 4: Execution strategies available through the release-0.1.0 builder. Availability of optional plugins depends on the local PennyLane installation.

Selector	Intended regime	QASP integration
`default.qubit`	Portable exact reference and back-propagation	Explicit portable selection for analytic simulation
`lightning.qubit`	Compiled state-vector execution	Shots, observable batching, adjoint-compatible workflows
`lightning.kokkos`	Performance-portable CPU/GPU execution	Optional MPI, shots, and batched observables
`default.tensor`	Circuits admitting compact tensor-network structure	Matrix-product-state method selectable by the builder
`default.clifford`	Clifford-dominated circuits	Optional Clifford+T decomposition and SPSA directions
arbitrary device name	Hardware or third-party simulator	Forwarded to PennyLane with QASP wire/shot configuration
circuit cutting	Circuits partitionable into smaller fragments	Fragment generation and reconstruction on a smaller Kokkos device
`lightning.pp`	Pauli-observable workloads with manageable operator growth	QASP logical selector: traces the configured circuit and evaluates expectations with differentiable Torch-based Pauli propagation; it is not an upstream Lightning plugin

This follows the information-geometric motivation of quantum natural gradient [13]. The tested implementation requires `interface="autograd"`, analytic execution, no trainable input scales, and positive regularization λ. The quantum option uses four times the full adjoint metric tensor, while the classical option constructs the probability Fisher matrix. Setting `commuting=True` reuses the initial Fisher matrix as a caller-selected approximation; QASP does not infer the required commutativity. Because the branch forms a dense parameter-by-parameter matrix, it is a small-system method rather than the scalable expectation-mixture route described in Section 7.2.

3.3 High-performance and approximate simulation

No single simulator dominates across qubit count, depth, entanglement, observable locality, shot model, and derivative method. QASP therefore exposes a portfolio through `CircuitBuilder.compile`; Table 4 describes the execution capacity rather than asserting universal speedups.

QASP feature: Distributed and backend-portable execution

`CircuitBuilder.compile` selects portable state-vector, compiled Lightning, Kokkos/MPI, tensor-network, stabilizer, circuit-cutting, or custom PennyLane devices behind one circuit contract. Ray-backed diagnostic sweeps provide task-level distribution, while Kokkos/MPI and circuit cutting provide distributed or fragmented quantum-simulation paths where supported.

State-vector simulation stores 2^n amplitudes and remains the exact reference for generic noiseless circuits. Tensor-network methods can replace this dependence with one controlled by bond dimension when entanglement structure is favourable. Stabilizer simulation is efficient for Clifford circuits but restricts the gate set. Circuit cutting exchanges quantum width for the number and cost of fragment evaluations. Pauli propagation instead targets expectation values by evolving observables in the Heisenberg picture. These approaches make different structural assumptions, so QASP exposes them at a common boundary rather than selecting one globally.

3.3.1 Pauli propagation

Consider a Pauli observable O and circuit $U = G_D \cdots G_1$. Rather than evolving $|0\rangle$, Pauli propagation forms

$$O_0 = U^\dagger O U = G_1^\dagger \cdots G_D^\dagger O G_D \cdots G_1, \qquad \langle O \rangle = \langle 0^{\otimes n} | O_0 | 0^{\otimes n} \rangle. \tag{11}$$

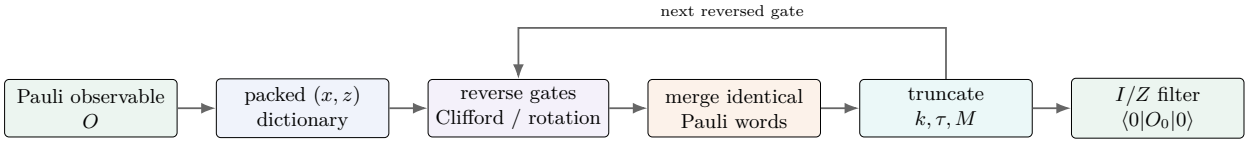


Figure 3: Pauli-propagation pipeline. Clifford gates do not branch; anticommuting parameterized Pauli rotations can create two terms. Periodic merging and truncation control the active operator representation.

Write $O_t = \sum_j c_j^{(t)} P_j$. Clifford gates map each Pauli word to one Pauli word, while a Pauli rotation $R_P(\theta) = e^{-i\theta P/2}$ either leaves a commuting word Q unchanged or branches an anticommuting word:

$$R_P(\theta)^\dagger Q R_P(\theta) = \cos(\theta) Q + i \sin(\theta) PQ, \qquad \{P, Q\} = 0. \tag{12}$$

The final zero-state expectation retains only words containing I and Z, because any X or Y factor has zero expectation in $|0\rangle^{\otimes n}$. This operator-backpropagation viewpoint and its truncations are developed more generally in the Pauli-propagation literature [14, 15].

QASP encodes P_j with two n-bit masks (x_j, z_j) packed into one Python integer. The local code $x + 2z$ represents I, X, Z, Y. CNOT, CZ, and Hadamard conjugations become bit or small-table updates; identical output words are merged immediately in a dictionary. One- and two-qubit X, Y, and Z rotations implement Equation (12). Tapes are first compiled into small reversed operation tuples, removing type checks and string parsing from the hot loop.

Three truncation controls bound operator growth:

$$\mathcal{T}_{k,\tau,M}(O) = \mathrm{TopM}_M \left(\sum_{j:\, \mathrm{wt}(P_j) \le k,\, |c_j| \ge \tau} c_j P_j \right), \tag{13}$$

where k is the maximum Pauli weight, τ a coefficient threshold, and M a maximum number of retained terms. Coefficient and top-M pruning can be applied every configurable number of gates. Terms are merged before pruning so interference between identical Pauli words is respected.

3.3.2 Fast and differentiable engines

`FastTruncatedPauliProp` stores ordinary Python complex coefficients and converts tape parameters to floating-point numbers. It is intended for fast evaluation or analysis where gradients are not required. `DifferentiablePauliProp` stores scalar Torch tensors instead. Its $\cos\theta$ and $\sin\theta$ branches stay attached to the autograd graph, so gradients propagate through every retained path.

QASP feature: `FastTruncatedPauliProp` and `DifferentiablePauliProp`

Two packed-integer Heisenberg simulators expose the same truncated Pauli path with different numerical contracts. The fast engine targets evaluation and diagnostics; the differentiable engine retains Torch coefficients so gradients flow through the currently selected Pauli paths. Both make weight, coefficient, term-count, and pruning-cadence budgets explicit.

Hard selection is necessarily non-smooth. The differentiable engine computes τ and top-M masks from detached coefficient magnitudes, then preserves the original tensors for selected terms. Gradients therefore describe the currently retained path set; they do not differentiate the discrete event at which a term enters or leaves that set. Weight-k truncation is structural. This distinction is important when interpreting an approximate training trajectory.

The controls remain independent: `pp_prune=False` disables coefficient pruning but does not disable `pp_truncation`. Thus there is no straight-through estimator for a pruning mask, and differentiability of the retained computation is not an accuracy or learning guarantee.

With M_t active Pauli terms, a local Clifford update is $O(M_t)$ and a supported rotation creates at most $2M_t$ pre-merge terms. The advertised benefit is consequently conditional: execution is attractive when M_t remains bounded by locality, cancellations, or chosen truncation, and unattractive when operator growth fills the available term budget. Exactness on the supported gate set is recovered only with no active weight, coefficient, or count truncation and enough memory to retain all generated terms. Users should report k, τ, M, pruning cadence, and an exact-reference convergence check.

```
1 from qasp.rl import PQDQN
```

```
model = PQDQN(
    n_qubits=10, n_actions=2,
    ansatz="AngleAnsatz",
    ansatz_config={
        "n_layers": 4, "rotations": "YZ",
        "reuploading": True,
        "include_imprimitive": True,
        "imprimitive": "CNOT",
        "imprimitive_pattern": "ring",
        "include_final_imprimitive": True,
    },
    feature_map="AngleEmbedding",
    feature_map_config={
        "features": 10, "rotations": "Y",
        "input_scaling": True,
    },
    observables=[f"Z {i}" for i in range(10)],
    fc_layer=True, fc_bias=True,
    input_activation="tanh",
    batch_mode="sequential",
    device="lightning.pp", diff_method="backprop",
    pp_truncation=2,
    pp_prune=False, pp_topM=None, pp_tau=0.0,
).double()
```

Listing 2: The ten-qubit `PQDQN` configuration used for the $k = 2$ Pauli-propagation benchmark. In QASP, `"lightning.pp"` is a logical backend selector, not a separately installed PennyLane-Lightning device.

The selector is available to observable–softmax policies, recursive-parity logical moments, binary and multiclass expectation-readout classifiers, and `PQDQN` Pauli readouts. Gradients remain connected to circuit rotations, trainable input scaling or neural embeddings, and any classical head. The current engine consumes scalar-angle tapes, so arbitrary leading batch dimensions are evaluated sample by sample and stacked while preserving the PyTorch graph; `batch_mode="broadcast"` does not create native Pauli-engine broadcasting. Born-probability outputs and sparse diagonal projector measurements remain outside this backend because it accepts Pauli expectations, not probability vectors or general sparse matrices. This is an interface boundary, not an approximation of their semantics.

3.3.3 Ten-qubit PQDQN simulator microbenchmark

We benchmarked the model in Listing 2 and Figure 4a: ten qubits, four reuploaded R_Y/R_Z ansatz layers, trainable R_Y input scaling, a CNOT ring in every layer including the final layer, and ten shared expectations $z_i = \langle Z_i \rangle$ followed by the biased two-output head $\boldsymbol{Q} = W\boldsymbol{z} + \boldsymbol{b}$. The model has 142 trainable parameters. Five matched parameter initializations and normal input batches use seeds 7–11; QASP's `normal_0_1` initializer means $\mathcal{N}(0, 1)$, not standard deviation 0.1. Each saved parameter state and input tensor is identical between the exact `lightning.qubit`/adjoint reference and the two `lightning.pp`/backprop approximations.

The Pauli runs set $k \in \{2, 3\}$ with `pp_prune=False`, `pp_topM=None`, and `pp_tau=0`; thus only Pauli-weight truncation is active. Both backends evaluate samples sequentially in CPU float64. For each seed, batch size, backend, and workload, two warmups precede seven timed repeats, with backend order shuffled inside every repeat. Torch intra- and inter-op counts and OMP/OpenBLAS/MKL counts are fixed to one. Measurements were collected in one local session on an Apple M1 with 16 GiB memory and eight hardware cores, without core pinning or thermal control.

Table 5: Ten-qubit PQDQN timings in milliseconds per complete batch. Entries are mean ± sample SD of five within-seed medians, each based on seven timed measurements after two warmups; the SD is descriptive, not a confidence interval.

Batch	Backend	Inference	Forward + backward
1	Lightning adjoint	26.90 ± 7.44	28.59 ± 5.45
1	PP $k = 2$	13.89 ± 4.27	16.69 ± 1.87
1	PP $k = 3$	30.87 ± 8.15	48.24 ± 10.12
8	Lightning adjoint	190.47 ± 27.71	198.02 ± 9.00
8	PP $k = 2$	83.18 ± 7.53	110.29 ± 8.16
8	PP $k = 3$	215.66 ± 28.74	326.91 ± 30.35

The forward–backward workload consists of `zero_grad`, one complete PQDQN call, a selected-action mean Huber loss, and `backward`. It uses fixed alternating actions and targets linearly spaced from 0.25 to 1.75; optimizer updates, replay sampling, target-network evaluation, and environment interaction are excluded. It is therefore the differentiable kernel of a Q-learning update, not a full DQN step or a learning curve.

For batch eight, $k = 2$ is approximately $1.80\times$ faster than Lightning for this gradient workload, while $k = 3$ is slower. Timing alone is misleading: the batch-eight feature RMSE is 0.03164 for both cutoffs and Q-value RMSE is about 0.0176, yet feature relative ℓ_2 errors are 0.9984 and 0.9973. The quantum-gradient relative ℓ_2 errors are 1.0002 and 0.9991, with cosine similarities -0.015 and 0.037 for $k = 2$ and $k = 3$, respectively. Quantum gradients include all circuit rotations and input scales but exclude the classical head. The small absolute Q-value errors therefore do not imply accurate quantum propagation: the shared bias is unchanged and the affine head can attenuate discarded-feature errors.

The exact Lightning result and all parameter gradients agree with `default.qubit`/backprop for seed 7 and batch eight, with maximum gradient discrepancy 2.52×10^{-10}. Directional finite differences separately confirm the derivative of each truncated Pauli function. These checks verify differentiation of the respective exact or truncated functions; they do not make truncation exact. The experiment provides no evidence of equivalent or faster RL learning, accuracy-preserving speedup, asymptotic scalability, or universal backend superiority. Five parameter seeds are also not five independent machine sessions.

3.4 Distributed workload planning and execution

The `qasp.hpc` package separates resource planning from execution. `inspect_workload` reads supported configurations, builders, or models without constructing a large simulator or invoking user callbacks; explicit `MemoryHints` fill dimensions that cannot be inferred safely. `plan_deucalion` then estimates state-vector storage, differentiation storage, and replicated outputs and chooses a placement for one experiment or independent repetitions. These values are conservative analytical estimates based on the documented 12 October 2026 machine profile. They are neither measured peak memory nor runtime guarantees, and host-memory, accelerator- memory, and Slurm-memory units must not be conflated.

QASP feature: `inspect_workload`, `plan_deucalion`, and `render_slurm`

The HPC layer turns a QASP workload description into an inspectable resource plan and a reproducible Slurm script. Rendering never submits a job, copies source code, or installs dependencies; `DeucalionJob.write` also refuses to overwrite an existing file. At runtime an `ExecutionContext` gives callbacks their experiment index, MPI rank/world size, shared seed, and root- rank status. Every rank must enter a distributed simulation callback, while ordinary artifact writing should be restricted to the root rank.

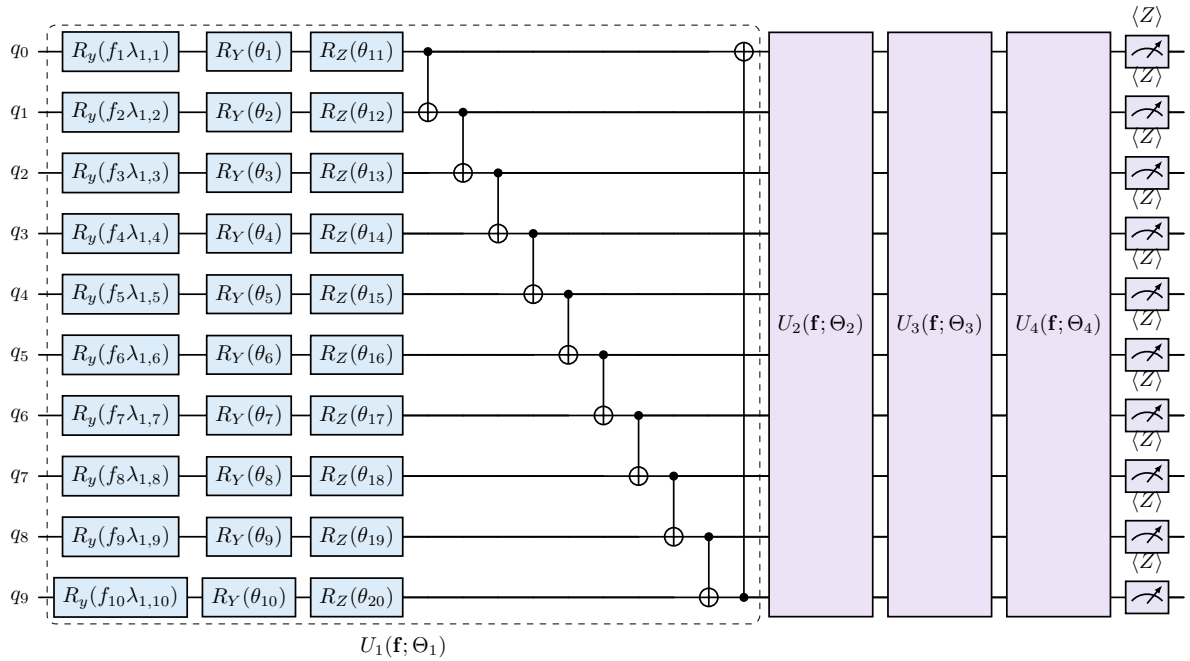
Independent repetitions can be expressed as array or packed jobs; a single distributed state vector instead occupies one MPI world. QASP propagates the requested backend and MPI options without silently falling back to serial execution, so the installed PennyLane plugin and system MPI stack remain part of the experiment contract. The generated script is consequently a plan that must still be submitted and monitored by the user or site scheduler.

4 Reinforcement learning

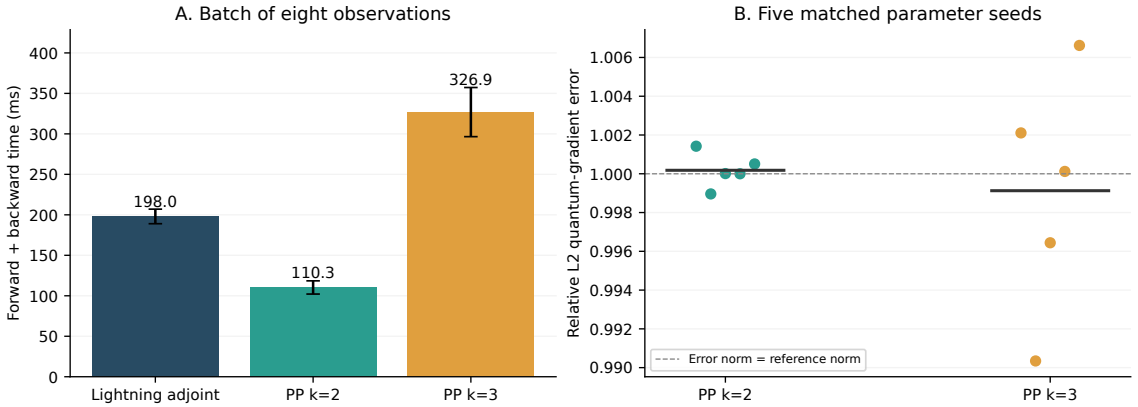
QASP supports two complementary reinforcement-learning contracts. In policy-based RL, the quantum model returns logits or normalized action probabilities and is optimized directly through REINFORCE, AWR, or PPO. In value-based RL, it returns unconstrained action values and is optimized from Bellman targets through deep Q-learning. In both cases the quantum component remains orthogonal to environment interaction, experience collection, and classical optimizer state. Figure 5 shows the policy-based path.

QASP feature: High-level RL modules

`create_model` specializes the common feature-map, ansatz, builder, and backend stack as either `PQCSoftmax` or a decoded-Born model. The factory selector `"PQCBorn"` currently constructs the enhanced `PQCBornVariance`; direct `PQCBorn` is a distinct legacy sample-and-log-probability interface. `create_qfunction` builds the value-based `PQDQN` counterpart. Each maintained object is an ordinary PyTorch module, but trainers must honor its logits, probabilities, or unconstrained-Q-value contract.



(a) Compact Quantikz export of the common circuit. The first complete layer is expanded and the remaining reuploaded layers are compressed. The ten meters denote separate local-Z expectations; $W\boldsymbol{z}+\boldsymbol{b}$ is classical and lies outside the circuit.



(b) Batch-eight forward–backward timing and per-seed quantum-gradient relative ℓ_2 error. Timing bars show mean ± sample SD across five seed medians; gradient dots are the five matched model seeds and horizontal marks are their means. Error bars are not confidence intervals.

Figure 4: Ten-qubit PQDQN simulator microbenchmark. Low-weight propagation exposes a runtime–accuracy trade-off: the faster $k = 2$ measurement has gradient error comparable to the norm of the exact gradient, while $k = 3$ is slower than the exact reference in this setup.

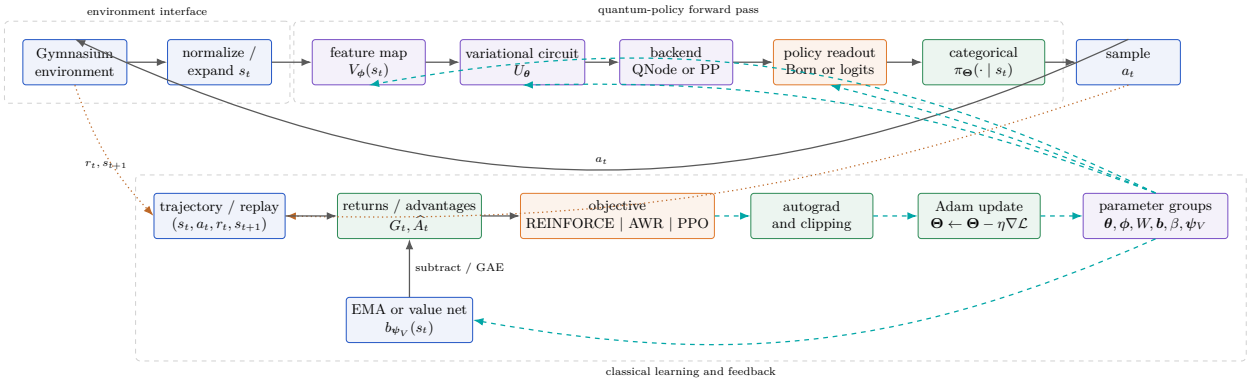


Figure 5: Detailed QASP policy-based reinforcement-learning dataflow. Solid arrows form the online interaction path, dotted orange arrows record experience, and dashed cyan arrows carry differentiable feedback. The circuit and backend are isolated inside the policy forward pass, while state processing, sampling, returns/advantages, baselines, objective selection, and optimization remain classical and independently replaceable.

4.1 Policy-based reinforcement learning

Policy-based agents parameterize the behavior distribution itself. QASP separates the quantum circuit, action readout, and classical learning rule so that observable–softmax and decoded-Born policies can share the same rollout and optimization infrastructure. This subsection first fixes the policy semantics and execution contracts, then presents the maintained trainers, benchmarks, diagnostics, and circuit-export workflow.

4.1.1 Policy and action-readout primitives

QRL connects a quantum state model to a classical Markov decision process. For state s and discrete action set $\mathcal{A} = \{0, \dots, A-1\}$, QASP separates two questions that are easily conflated: *what action does a measurement outcome mean?* and *how is the corresponding probability evaluated and differentiated?* This separation is the organizing principle of the current RL policy implementation.

Observable–softmax policies. For Pauli observables $O_1, \dots, O_q$, `PQCSoftmax` evaluates the quantum feature vector

$$\boldsymbol{z}_{\boldsymbol{\theta},\boldsymbol{\phi}}(s) = \big(\langle O_1 \rangle_s, \dots, \langle O_q \rangle_s\big). \tag{14}$$

If $q = A$, these values may be used directly as action logits. Alternatively, a trainable classical head $W \in \mathbb{R}^{A \times q}$ and $\boldsymbol{b} \in \mathbb{R}^A$ produces

$$\pi_{\boldsymbol{\Theta}}(a \mid s) = \mathrm{softmax}(\beta\,[W \boldsymbol{z}_{\boldsymbol{\theta},\boldsymbol{\phi}}(s) + \boldsymbol{b}])_a\,, \quad \boldsymbol{\Theta} = (\boldsymbol{\theta}, \boldsymbol{\phi}, W, \boldsymbol{b}, \beta), \tag{15}$$

where β is an optional fixed or trainable inverse-temperature scale. The class returns scores from `forward`; a trainer must therefore use `Categorical(logits=...)`. Signed expectation values are never silently reinterpreted as probabilities. In the current constructor only `fc_layer=None` disables the affine head; any other value, including `False`, creates it. Its bias is trainable. The older `action_probs()` shortcut bypasses that head, trainable scaling, and the Pauli-propagation route, so logits-aware training should call `forward`.

QASP feature: `PQCSoftmax`

An observable-based policy module whose quantum circuit returns Pauli expectation features. It exposes those features directly as action logits or passes them through a trainable classical head and optional inverse temperature. Its explicit logits contract makes it compatible with standard PyTorch categorical sampling and policy-gradient trainers. The legacy `action_probs()` shortcut is not equivalent when the head, scaling, or Pauli backend is active.

One decoded-Born semantics. Let $M = (w_1, \dots, w_m)$ be the ordered measured wires and $z \in \{0,1\}^m$ a computational-basis outcome in PennyLane's big-endian ordering. A Born circuit defines

$$p_{\boldsymbol{\theta},\boldsymbol{\phi}}(z \mid s) = \big|\langle z \mid \psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(s) \rangle\big|^2\,. \tag{16}$$

A deterministic decoder $d : \{0,1\}^m \to \mathcal{A}$ induces the action projectors and policy

$$\Pi_a = \sum_{z:\, d(z)=a} |z\rangle\langle z|, \tag{17}$$

$$\pi_{\boldsymbol{\theta},\boldsymbol{\phi}}(a \mid s) = \langle \psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(s) | \Pi_a | \psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(s) \rangle = \sum_{z:\, d(z)=a} p_{\boldsymbol{\theta},\boldsymbol{\phi}}(z \mid s). \tag{18}$$

Equivalently, with $D_{az} = \mathbf{1}[d(z) = a]$, one has $\boldsymbol{\pi} = D\boldsymbol{p}$ and $\sum_a D_{az} = 1$ for every basis state. The internal `_PolicyDecoder` precomputes the integer action index for all 2^m basis states. Its forward path uses a batch-preserving Torch `scatter_add_`; its sample path converts measured bits to the same big-endian index and performs a lookup. Consequently the implementation enforces the semantic invariant

$$\Pr_{Z\sim p}[\texttt{decode_samples}(Z) = a] = \sum_z D_{az} p(z) = \pi(a \mid s), \tag{19}$$

rather than maintaining separate, potentially inconsistent formulas for training and deployment.

QASP feature: `PQCBorn` and `PQCBornVariance`

High-level decoded-Born policies with one cached outcome-to-action map shared by exact probability aggregation and sampled-action decoding. `PQCBornVariance` additionally exposes the full-probability, logical-moment, and sparse-projector execution contracts behind the same normalized action distribution, so a backend can change without redefining the policy.

Decoder families. Release 0.1.0 exposes four related outcome-to-action maps. Write $J(z) = \sum_{i=1}^m 2^{m-i} z_i$ and $k = \lceil \log_2 A \rceil$.

Contiguous. Let $2^m = qA + r$, $0 \le r < A$, and let $b_a = q + \mathbf{1}[a < r]$. Consecutive basis indices are assigned to action a whenever

$$\sum_{j<a} b_j \le J(z) < \sum_{j\le a} b_j. \tag{20}$$

Thus the bins differ in size by at most one even when A is not a power of two, following the contiguous-policy construction of Jerbi et al. [16].

Recursive parity. The k logical bits are nested prefix parities

$$r_j(z) = z_1 \oplus \cdots \oplus z_{m-j}, \qquad j = 0, \ldots, k-1, \tag{21}$$

and the decoded action is

$$d_{\mathrm{RP}}(z) = \left(\sum_{j=0}^{k-1} 2^{k-1-j} r_j(z) \right) \bmod A. \tag{22}$$

This generalizes the binary global-parity readout and implements the optimized action-decoding idea of Meyer et al. [17]. The `hidden-like` variant first selects a reduced ordered wire set—every second circuit wire by default—and applies the same recursive map.

Majority–modulo. Let $B_0, \ldots, B_{k-1}$ be disjoint blocks of positions in the measured bit string. Define

$$h_j(z) = \mathbf{1}\left[\sum_{i\in B_j} z_i > \left\lfloor \frac{|B_j|}{2} \right\rfloor \right], \qquad d_{\mathrm{MM}}(z) = \left(\sum_{j=0}^{k-1} 2^{k-1-j} h_j(z) \right) \bmod A. \tag{23}$$

The strict inequality gives deterministic tie-to-zero behavior for an explicit even block. If blocks are omitted, QASP uses as many measured wires as possible in positive odd blocks whose sizes differ by at most two, leaving at most one wire unused. The current strict option additionally requires equal block sizes of at least three and no unused measured wire; users reproducing the tie-free theoretical setting should choose that common size to be odd. Blocks are specified by actual device-wire labels, while the decoder stores their positions in the ordered measurement register.

Four exact or differentiable execution contracts. The decoder fixes Π_a and therefore the mathematical policy, but it does not uniquely fix how that policy is evaluated. Figure 7 shows the four principal paths.

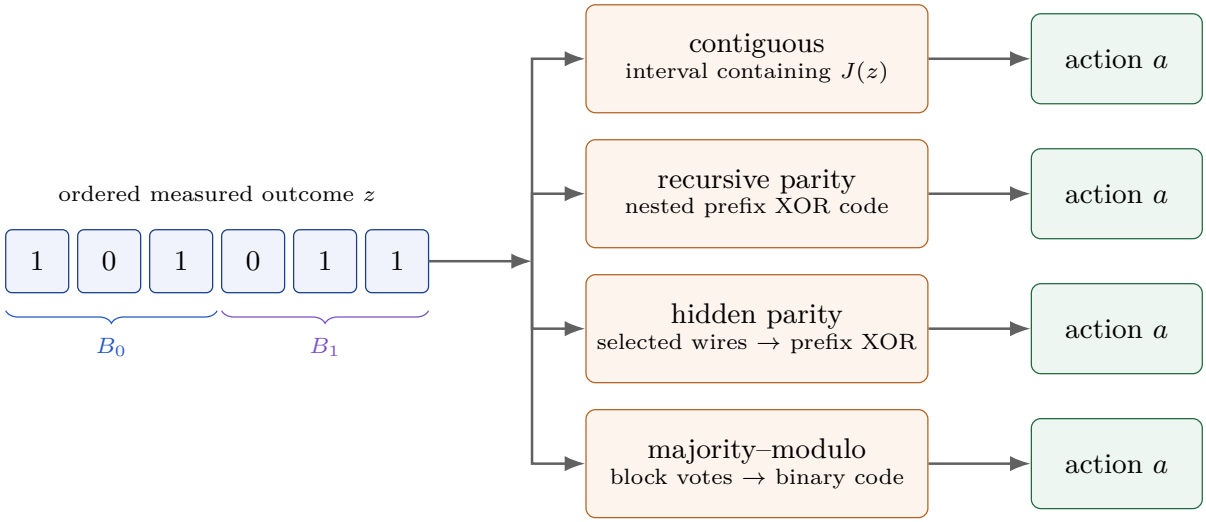


Figure 6: One measured bit string can support several explicit policy semantics. QASP precomputes the selected map once and uses it for both probability aggregation and sample decoding. The majority braces are only an illustrative block assignment; arbitrary disjoint measured-wire blocks may be configured.

Full probabilities. With `training_method="probs"`, the quantum layer returns all 2^m basis probabilities and the shared decoder computes $D\boldsymbol{p}$. This path supports every decoder and is the reference against which specialized paths are tested, but its returned vector grows exponentially in the number of measured wires.

Full Z-moment reconstruction. For compatible expectation devices, every basis probability can instead be reconstructed from $2^m - 1$ nonidentity moments $\mu_t = \langle Z^t \rangle$, $t \in \{0,1\}^m$, by

$$p(z) = 2^{-m} \sum_{t \in \{0,1\}^m} (-1)^{t \cdot z} \mu_t, \qquad \mu_0 = 1. \tag{24}$$

The PyTorch implementation stores the measurements in binary-mask order and applies a differentiable fast Walsh–Hadamard transform (FWHT), preserving arbitrary leading batch dimensions.

Logical-moment reconstruction for recursive parity. Recursive parity needs only the distribution of its k logical bits, not the full m-bit distribution. Let

$$R_j = \prod_{i=1}^{m-j} Z_{w_i}, \qquad R_S = \prod_{j \in S} R_j, \qquad \nu_S = \langle R_S \rangle. \tag{25}$$

Repeated Pauli factors cancel, so the support of R_S is the symmetric difference of the nested prefix supports. Measuring all $2^k - 1$ nonidentity R_S yields the exact logical-code distribution

$$q(y) = 2^{-k} \sum_{S \subseteq \{0,\dots,k-1\}} (-1)^{\sum_{j \in S} y_j} \nu_S, \qquad \nu_\varnothing = 1, \tag{26}$$

after which

$$\pi(a \mid s) = \sum_{y:\, J(y) \bmod A = a} q(y). \tag{27}$$

Thus the quantum output count depends on A through $2^{\lceil \log_2 A \rceil} - 1$, rather than on all 2^m measurement outcomes. The same reconstruction applies when CNOT networks materialize the logical parities on ancillas. This exact multi-action path is regression-tested for $A = 3$ and $A = 4$ against full probabilities and their parameter gradients.

Sparse majority action projectors. Majority projectors are generally dense in their Pauli–Z Fourier expansion, but diagonal in the computational basis. QASP therefore constructs each Π_a directly as a SciPy CSR diagonal matrix over all data and ancilla wires. With analytic execution,

$$\pi(a \mid s) = \langle \Pi_a \rangle, \quad a = 0, \dots, A-2, \qquad \pi(A-1 \mid s) = 1 - \sum_{a=0}^{A-2} \langle \Pi_a \rangle. \tag{28}$$

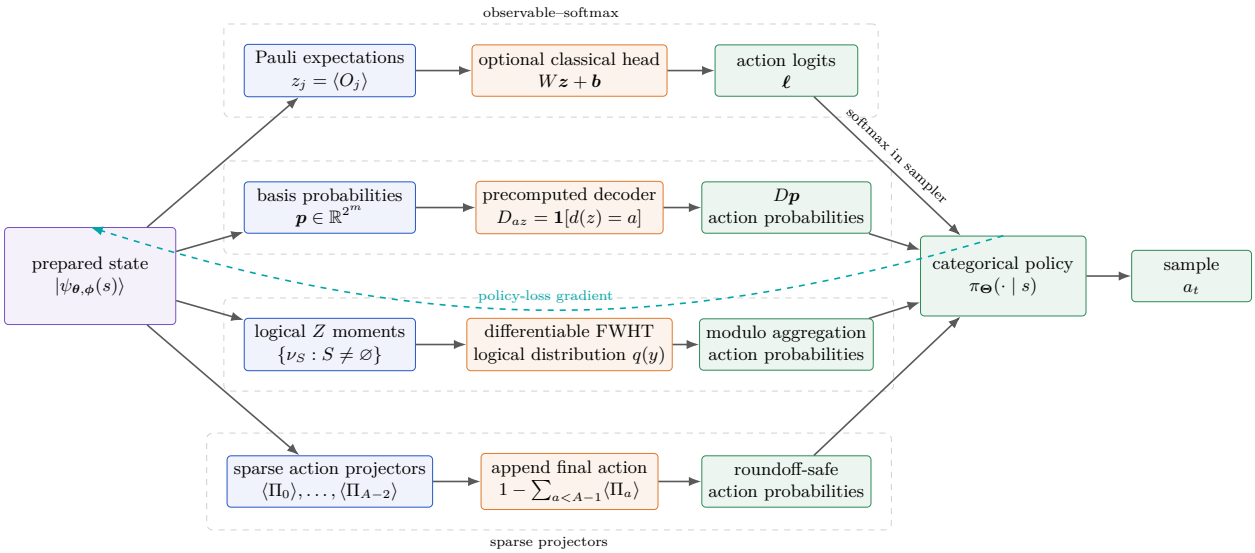


Figure 7: Policy semantics and execution are separated. The upper lane constructs a softmax policy from quantum features. The remaining lanes are mathematically equivalent decoded-Born routes when their compatibility conditions hold: full basis probabilities, logical-moment reconstruction, or direct sparse action-projector expectations.

Only $A - 1$ observables are evaluated. Machine-precision negative residuals are clamped and the vector is renormalized before construction of a categorical distribution.

This path intentionally has a narrow contract: it uses analytic `lightning.qubit` or `lightning.kokkos`, `diff_method="adjoint"`, `device_vjp=False`, and `batch_obs=False`. The device-native vector–Jacobian product is deliberately disabled because it produced incorrect gradients for three or more sparse-projector measurements in the tested PennyLane-Lightning stack; the standard adjoint path agrees with the full-probability reference. Shots, Kokkos MPI, circuit cutting, and `lightning.pp` are rejected for this mode instead of silently substituting another policy.

Adaptive measurement selection. `LearnablePQCBorn` records an experimental attempt to augment a Born policy with a classical selector over k-wire measurement subsets. Given an observation, a linear softmax model samples a wire combination and appends it to the adaptive circuit input. The current setup and post-processing path has known limitations, so the class is retained as a prototype rather than a validated adaptive-measurement trainer.

QASP feature: `LearnablePQCBorn`

A historical adaptive-measurement prototype in which a classical selector chooses a measured-wire subset before quantum execution. It requires further integration work before it should be used for learning claims.

4.1.2 Policy contract at the trainer boundary

All current RL trainers consume an ordinary PyTorch module. The only semantic distinction the trainer must know is whether `forward` returns logits or probabilities: `PQCSoftmax` is passed as logits to a `Categorical` distribution, whereas `PQCBornVariance` is passed as probabilities. The latter's `forward` and `action_probs` methods now share exactly the same decoding path. This small interface rule makes it possible to exchange full-probability, logical-moment, and sparse-projector implementations without changing the REINFORCE objective or environment interaction code.

Configuration reuse also requires care: classifier, kernel, and Q-function factories copy their dictionaries, whereas the legacy policy factory performs a shallow copy and may mutate nested component dictionaries. Reusable policy templates should therefore be deep-copied before construction. Historical policy weight setters may call a nonexistent QNode method; ordinary Torch `state_dict` loading is the supported transfer mechanism.

Table 6: Decoded-Born execution contracts in the current implementation. m is the number of measured wires and $k = \lceil \log_2 A \rceil$. "Exact" describes the mathematical readout, not the scaling of its simulator.

Training method	Quantum outputs	Compatible decoders	Post-processing	Principal boundary
`probs`	2^m probabilities	all four families	vectorized scatter-add with cached action indices	exponential output vector
`expectation`, logical	$2^k - 1$ Pauli moments	recursive parity	FWHT on logical bits, then modulo aggregation	expectation-compatible analytic or finite-shot device
`expectation`, full	$2^m - 1$ Pauli moments	contiguous, hidden parity, and ancilla parity	full FWHT, then shared decoder	no output-count advantage over full probabilities
`sparse_projectors`	$A - 1$ sparse expectations	majority–modulo	append residual action and normalize roundoff	analytic Lightning adjoint; no device VJP, MPI, cutting, or PP
direct samples	one or batched m-bit strings	all four families	big-endian lookup through the same cached decoder	gradients require a separate estimator

For one trajectory τ, the score-function identity remains

$$\nabla_{\boldsymbol{\Theta}} J = \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\Theta}}} \left[\sum_t G_t \, \nabla_{\boldsymbol{\Theta}} \log \pi_{\boldsymbol{\Theta}}(a_t \mid s_t) \right]. \tag{29}$$

The quantum-specific work is therefore concentrated in producing a normalized, differentiable $\boldsymbol{\pi}(s_t)$ and its log probability; trajectory collection, baselines, advantage normalization, entropy bonuses, and optimizer state remain classical.

4.1.3 Windy Target Navigation

The release adds `WindyTargetNavigationEnv`, a self-contained Gymnasium benchmark designed around the current three- and four-action policies. A point mass moves through a sequence of randomly placed waypoints under inertia and a persistent stochastic wind. For $A \in \{3, 4\}$, action a selects an equally spaced unit thrust direction

$$\boldsymbol{u}_a = \left(\cos \frac{2\pi a}{A}, \sin \frac{2\pi a}{A} \right). \tag{30}$$

With time step Δt, thrust c, drag δ, speed bound $v_{\max}$, wind persistence ρ, and Gaussian innovation $\boldsymbol{\epsilon}_t$, the update is

$$\begin{aligned} \widetilde{\boldsymbol{v}}_{t+1} &= \delta \boldsymbol{v}_t + \Delta t (c \boldsymbol{u}_{a_t} + \boldsymbol{w}_t), & \boldsymbol{v}_{t+1} &= \mathrm{clipnorm}_{v_{\max}}(\widetilde{\boldsymbol{v}}_{t+1}), \\ \boldsymbol{p}_{t+1} &= \boldsymbol{p}_t + \Delta t \, \boldsymbol{v}_{t+1}, & \boldsymbol{w}_{t+1} &= \mathrm{clip}_{[-w_{\max}, w_{\max}]}(\rho \boldsymbol{w}_t + \boldsymbol{\epsilon}_t). \end{aligned} \tag{31}$$

The six-dimensional observation is

$$s_t = \left[\tanh\left(\frac{\boldsymbol{g}_t - \boldsymbol{p}_t}{d_{\max}} \right), \mathrm{clip}\left(\frac{\boldsymbol{v}_t}{v_{\max}} \right), \mathrm{clip}\left(\frac{\boldsymbol{w}_t}{w_{\max}} \right) \right], \tag{32}$$

where $\boldsymbol{g}_t$ is the active waypoint. If $d_t = \|\boldsymbol{g}_t - \boldsymbol{p}_t\|_2$, the shaped reward is

$$R_t = 4(d_t - d_{t+1}) - 0.02 - 0.01 \|\boldsymbol{v}_{t+1}\|_2^2 + 3 \, \mathbf{1}_{\text{waypoint}} + 7 \, \mathbf{1}_{\text{all waypoints}}. \tag{33}$$

The task therefore rewards progress while penalizing time and excessive velocity. Episodes terminate after the configured number of waypoints or truncate at the step limit. The environment follows the Gymnasium reset/step contract, bounds every observation in $[-1, 1]^6$, and reproduces complete trajectories under identical seeds.

QASP feature: `WindyTargetNavigationEnv`

A seeded Gymnasium benchmark for three- and four-action quantum policies. It provides bounded six-dimensional observations, inertial waypoint dynamics, persistent stochastic wind, shaped rewards, and reproducible reset/step semantics without coupling the environment to a particular policy class.

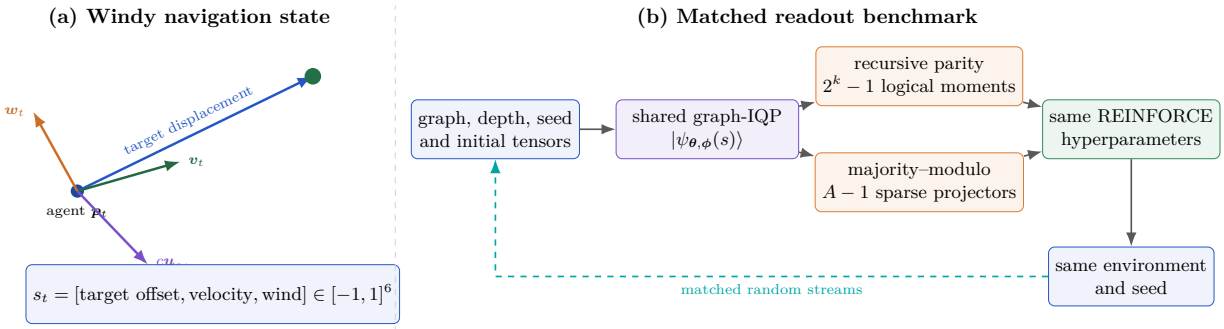


Figure 8: The new control benchmark and its intended comparison protocol. (a) The observation exposes the relative target, velocity, and wind, while the action chooses one of three or four thrust directions. (b) Parity and majority–modulo policies share the same graph-IQP circuit, initial trainable tensors, environment seeds, and trainer settings; only the readout contract changes.

4.1.4 Matched trainability-aware policy experiments

The accompanying policy-efficiency notebook implements paired experiments in which recursive-parity and majority–modulo policies receive exactly equal trainable tensors for each circuit depth and seed. The first readout uses the logical-moment path; the second uses sparse projectors. This pairing isolates the policy decoder from initialization and circuit effects.

Two bounded-degree graph families are provided for the windy task: a fast five-qubit chorded-cycle proxy and a seeded ten-qubit 3-regular graph accepted only when its adjacency spectral gap exceeds a configured threshold. The notebook spans $A \in \{3, 4\}$ and repeated graph-IQP depths $L \in \{1, 2, 3\}$. For five qubits, the majority blocks $\{0, 1, 2\}$ and $\{3\}$ avoid ties and leave wire 4 active in the circuit but unused by the classical decoder; for ten qubits, two blocks of five use every measured wire. The notebook first compares random actions with a feedback controller, then records returns, success rates, runtimes, and matched parameter checks. These are benchmark protocols supplied by the release, not performance claims in this software description; publishable numerical comparisons must be regenerated from the final tag.

4.1.5 Inspectable policy construction and circuit drawings

The high-level factory does not hide the circuit it creates. Every policy retains its resolved feature map, ansatz, builder, QNode, parameter shapes, and Torch layer. The public `draw_model` function traces that same executable circuit, including encoders, entangling topology, selected measurement wires, and parity ancillas. With `output="tikz"`, it returns Quantikz source with symbolic feature, scaling, and circuit parameters. This is useful both as a debugging check and as a reproducibility artifact: a paper figure can be generated from the experiment configuration rather than independently reconstructed in a graphics program.

Listing 3 builds a compact observable–softmax policy for a four-dimensional observation and exports its circuit. The four measured observables form the quantum feature vector; the final 4×2 classical head is outside the quantum circuit and consequently is described in the caption rather than drawn as a quantum gate.

```
from pathlib import Path
import torch
from qasp import create_model, draw_model

n_qubits = 4
softmax_config = {
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 2, "rotations": "ZY", "reuploading": False,
        "include_imprimitive": True, "imprimitive": "CZ",
        "imprimitive_pattern": "ring",
        "include_final_imprimitive": False,
        "include_feature_map_first": True,
        "include_hadamards_initial": True,
    },
    "feature_map": "NeuralEmbedding",
    "feature_map_config": {"features": 4, "rotations": "Z"},
    "n_actions": 2,
    "observables": ["Z 0", "Z 1", "ZZ 0 1", "ZZ 2 3"],
    "fc_layer": True,
```

```
    "init": "small_normal",
    "device": "default.qubit",
    "diff_method": "backprop",
}
softmax_policy = create_model(
    "PQCSoftmax", softmax_config, n_qubits=n_qubits
)
tikz = draw_model(
    model=softmax_policy, inputs=torch.zeros(4), output="tikz",
    tikz_options={
        "standalone": True, "symbolic_parameters": True,
        "compress": True,
    },
)
Path("figures").mkdir(exist_ok=True)
Path("figures/rl_softmax_observable.tex").write_text(tikz)
```

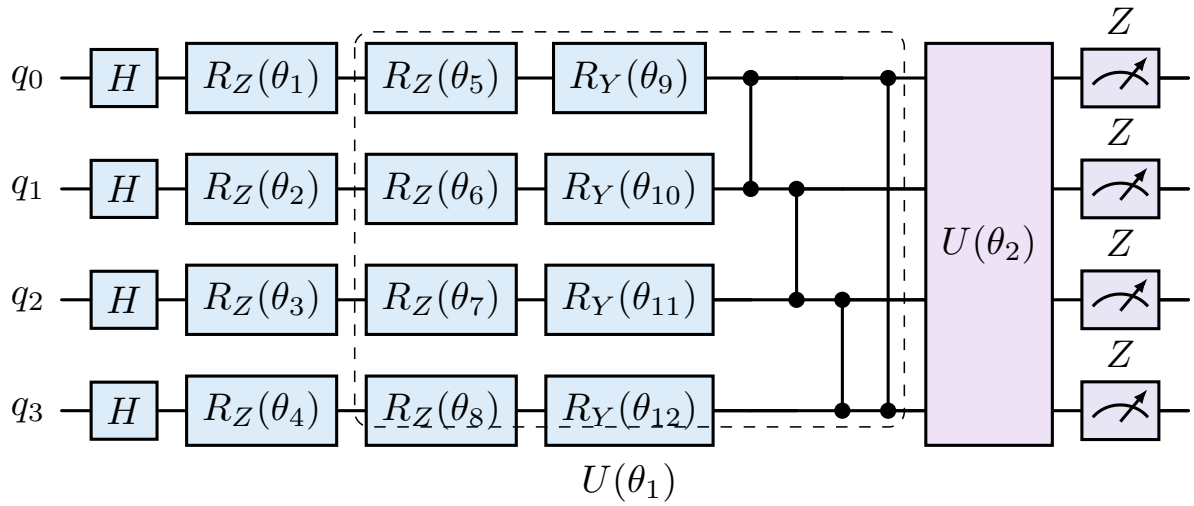
Listing 3: Constructing and drawing an observable–softmax RL policy. The saved circuit is shown in Figure 9a.

The corresponding Born construction in Listing 4 has the same circuit-level building blocks but a different measurement and action contract. It measures the full computational-basis distribution and applies recursive parity only after quantum execution. Thus the circuit drawing verifies the trainable model, while the separately reported decoder definition verifies how basis states become actions.

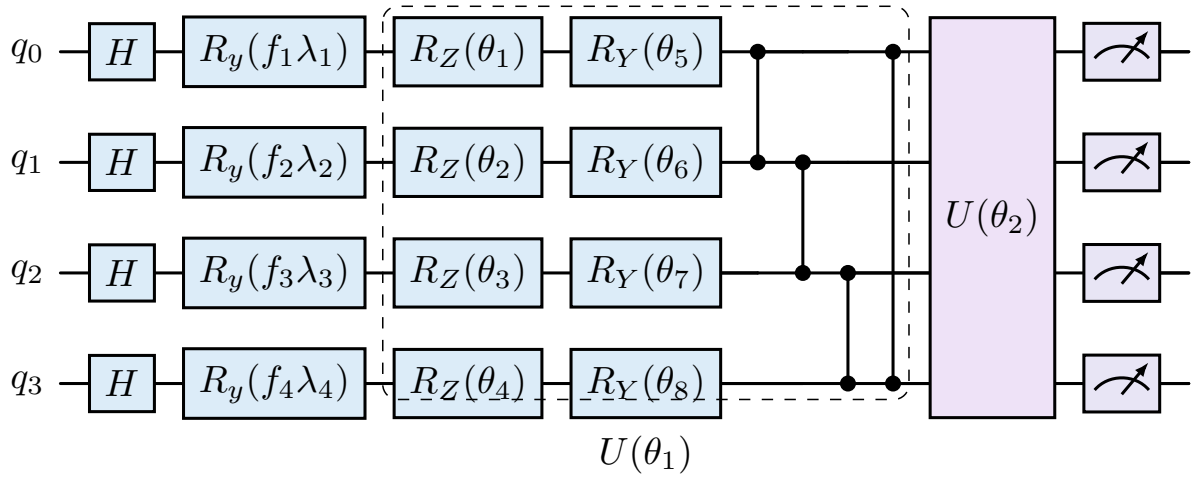
```
from pathlib import Path
from qasp import create_model, draw_model
import torch

n_qubits = 4
parity_config = {
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 2, "rotations": "ZY", "reuploading": False,
        "include_imprimitive": True, "imprimitive": "CZ",
        "imprimitive_pattern": "ring",
        "include_final_imprimitive": False,
        "include_feature_map_first": True,
        "include_hadamards_initial": True,
    },
    "feature_map": "AngleEmbedding",
    "feature_map_config": {
        "features": 4, "rotations": "Y", "input_scaling": True,
    },
    "n_actions": 2,
    "policy": "parity-like",
    "measurement_qubits": [0, 1, 2, 3],
    "training_method": "probs",
    "parity_cnot_decomposition": False,
    "init": "small_normal",
    "device": "default.qubit",
    "diff_method": "backprop",
}
parity_policy = create_model("PQCBorn", parity_config,
                             n_qubits=n_qubits)
action_probabilities = parity_policy(torch.zeros(4))
tikz = draw_model(
    model=parity_policy, inputs=torch.zeros(4), output="tikz",
    tikz_options={
        "standalone": True, "symbolic_parameters": True,
        "compress": True,
    },
)
Path("figures/rl_born_parity.tex").write_text(tikz)
```

Listing 4: Constructing and drawing a Born parity policy. The returned forward value is a normalized two-action probability vector and must be passed to `Categorical(probs=...)`.

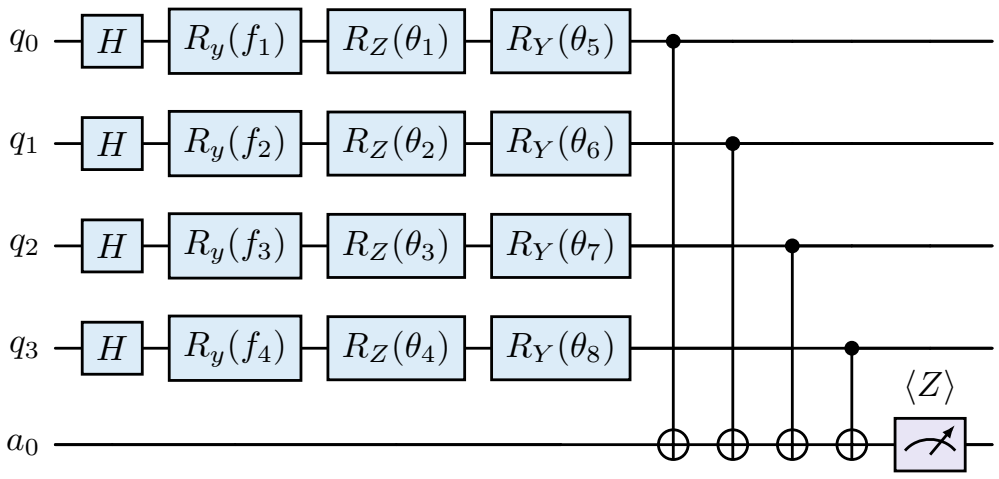


(a) Observable–softmax policy: trainable neural Z encoding, one explicit R_Z/R_Y ring-entangled layer, a compressed $U(\boldsymbol{\theta}_2)$ layer, four Pauli measurements, and a classical two-action head.

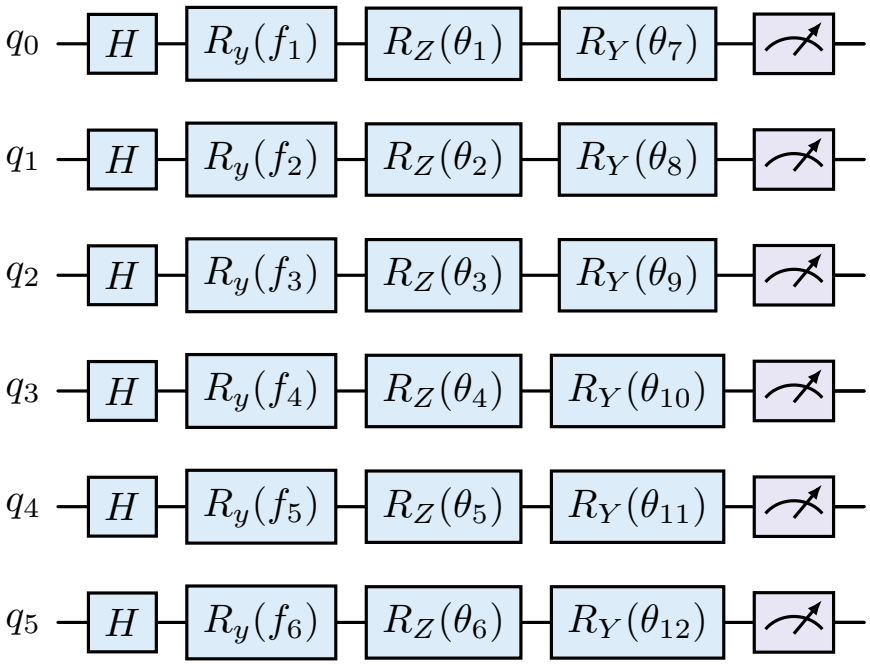


(b) Born parity policy: trainable angle scaling, one explicit variational layer, a compressed $U(\boldsymbol{\theta}_2)$ layer, full basis measurement, and an external fixed parity decoder.

Figure 9: Standard QASP reinforcement-learning readouts generated from the executable policy objects with QASP's `draw_model(output="tikz")` path. Symbolic parameters, semantic gate colors, and repeated-layer compression are supplied by the exporter. The classical softmax head and fixed parity decoder are stated in the subcaptions because they act after quantum measurement.



(a) Ancilla parity policy: four data wires fan into ancilla wire 4 with CNOTs; a single ancilla expectation represents the binary action readout.



(b) Three-action majority–modulo policy: six measured wires are decoded as odd blocks $\{0, 1, 2\}$ and $\{3, 4, 5\}$ after basis-probability evaluation.

Figure 10: Decoder-specialized QASP reinforcement-learning circuits generated through the same Quantikz path. The ancilla construction materializes parity inside the circuit; majority–modulo keeps its deterministic action decoder outside the measured circuit. The companion `render_rl_circuits.py` script retains editable standalone Quantikz and compiles the vector PDFs and PNG previews used by the paper.

4.1.6 REINFORCE

The primary `REINFORCE` trainer batches complete episodes. With discounted return $G_t = \sum_{j \geq 0} \gamma^j r_{t+j}$, baseline $b(s_t)$, and policy entropy $\mathcal{H}(\pi(\cdot \mid s_t))$, it minimizes

$$\mathcal{L}_{\mathrm{PG}} = -\frac{1}{B} \sum_{\tau \in \mathcal{B}} \sum_{t \in \tau} \widehat{A}_t \log \pi_{\boldsymbol{\theta}}(a_t \mid s_t) - \beta_{\mathcal{H}} \mathcal{H}(\pi_{\boldsymbol{\theta}}(\cdot \mid s_t)) \,, \tag{34}$$

where $\widehat{A}_t$ may be normalized across a batch. Baselines are either an exponential moving average of returns or a small classical value network. The trainer provides state scaling and nonlinear feature expansion, entropy regularization, episode batching, gradient clipping, and optional Acrobot reward shaping. These controls are particularly relevant to PQCs because the basic score-function estimator already has high variance [18].

The file also contains a configurable `Reinforce` implementation with online observation normalization, mean-return or MLP baselines, evaluation episodes, distinct learning-rate groups, and its own stopping rules. It is a different implementation, not an alias of the capitalized `REINFORCE` top-level export. The latter's CartPole-specific rule freezes optimization once the available trailing window of at most 100 training returns reaches a mean of 195; that implementation rule is not a `CartPole-v1` solved certificate and does not replace held-out evaluation.

QASP feature: `REINFORCE`

The top-level episode-batched policy-gradient trainer. It accepts any QASP policy module, selects logits or probability sampling explicitly, and combines discounted returns with optional value or moving-average baselines, entropy regularization, return normalization, gradient clipping, and environment state preprocessing.

```python
from qasp import create_model, REINFORCE

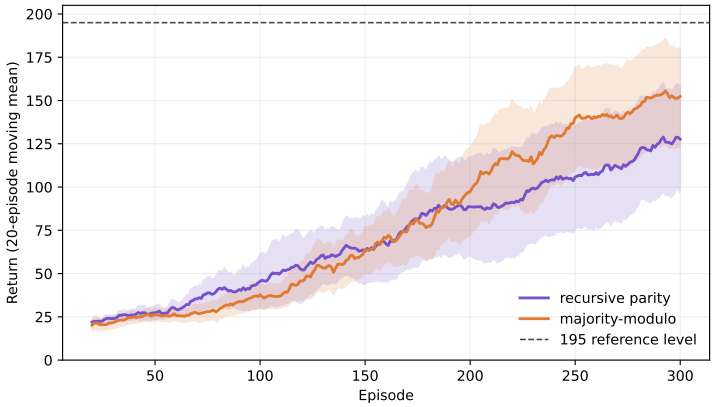
n_qubits = 6
policy = create_model("PQCSoftmax", {
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 3, "rotations": "ZY",
        "reuploading": False,
        "include_imprimitive": True,
        "imprimitive": "CZ", "imprimitive_pattern": "all_to_all",
        "include_final_imprimitive": False,
        "include_feature_map_first": False,
        "include_hadamards_initial": True,
    },
    "feature_map": "NeuralEmbedding",
    "feature_map_config": {"features": 4, "rotations": "Z"},
    "n_actions": 2,
    "observables": [f"Z {i}" for i in range(n_qubits)]
                 + [f"ZZ {i} {i+1}" for i in range(n_qubits - 1)],
    "fc_layer": True,
    "init": "small_normal",
    "device": "lightning.qubit",
    "diff_method": "adjoint",
}, n_qubits=n_qubits)

agent = REINFORCE(
    policy=policy, env_name="CartPole-v1",
    gamma=0.99, lr=1e-2, batch_episodes=16,
    logits=True, baseline_type="value",
    normalize_mode="clip", normalize_returns=True,
    entropy_coef=0.01, max_grad_norm=1.0, fc_layer=True,
)
history = agent.train(num_episodes=500)
```

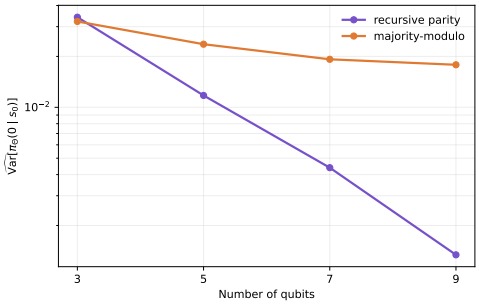
Listing 5: Configuration-driven softmax policy and episode-batched REINFORCE. `PQCSoftmax.forward` returns logits, hence `logits=True`.

4.1.7 Executable CartPole comparison

The release includes the executed notebook `examples/cartpole_policies_and_variance.ipynb`. It constructs a controlled pair of five-qubit graph-IQP Born policies for `CartPole-v1`. Both models use the same



(a) CartPole returns. Curves are 20-episode moving means and shaded regions are the standard error over five matched seeds.



(b) Initialization variance at fixed $s_0 = \mathbf{0}$ and $a = 0$, estimated from 500 reinitializations at each width.

Figure 11: Executable QASP policy examples generated by `cartpole_policies_and_variance.ipynb`. Both panels compare exact recursive-parity expectation readout with exact majority–modulo sparse projectors on one-layer ring graph-IQP circuits. These finite experiments validate the end-to-end workflow; they are not asymptotic performance claims.

neural embedding, ring graph, initial parameter tensors, environment seeds, and REINFORCE hyperparameters; only the action readout and its exact execution contract differ. The essential construction is shown in Listing 6; the notebook additionally asserts elementwise equality of all trainable tensors before optimization and uses `draw_model` on the constructed policy.

```
common = {
    "ansatz": "IQPAnsatz",
    "ansatz_config": {
        "n_layers": 1, "imprimitive_pattern": "ring",
        "reuploading": True, "include_single_qubit": True,
    },
    "feature_map": "NeuralEmbedding",
    "feature_map_config": {"features": 4, "rotations": "Z"},
    "n_actions": 2,
    "measurement_qubits": list(range(5)),
    "device": "lightning.qubit", "diff_method": "adjoint",
    "shots": None, "init": "normal_0_1",
}
parity = create_model("PQCBorn", {
    **common, "policy": "parity-like",
    "training_method": "expectation",
}, n_qubits=5)
majority = create_model("PQCBorn", {
    **common, "policy": "majority-modulo",
    "training_method": "sparse_projectors",
    "majority_blocks": [list(range(5))],
    "strict_majority_blocks": True,
}, n_qubits=5)
majority.qlayer.load_state_dict(parity.qlayer.state_dict())
```

Listing 6: Matched decoded-Born policies used by the executable CartPole example. The complete notebook includes imports, seeding, training, plotting, and data export.

The CartPole panel contains 300 episodes for each of five seeds, with an imposed 200-step episode cap. Over the final 50 episodes, the mean return was 121.09 ± 30.80 for recursive parity and 147.70 ± 28.70 for majority–modulo, where the uncertainty is the standard error across seeds. The overlapping uncertainty bands preclude a ranking claim; the purpose is to show that both readout contracts participate in the same complete learning loop. Every raw episodic return and per-run wall time is stored under `examples/results/`, so alternative smoothing or summary choices do not require rerunning the quantum simulation.

The second panel demonstrates the public `policy_variance` diagnostic. For a fixed action a and

observation s_0, randomized initializations $\mathbf{\Theta}_1, \ldots, \mathbf{\Theta}_N$ produce $p_j = \pi_{\mathbf{\Theta}_j}(a \mid s_0)$ and the estimator

$$\widehat{\mathrm{Var}}[\pi_{\mathbf{\Theta}}(a \mid s_0)] = \frac{1}{N}\sum_{j=1}^{N}(p_j - \bar{p})^2, \qquad \bar{p} = \frac{1}{N}\sum_{j=1}^{N} p_j. \tag{35}$$

The notebook evaluates the public API separately at each width so the measured wires and odd majority block remain explicit:

```
for policy_name in ("recursive parity", "majority-modulo"):
    for n_qubits in (3, 5, 7, 9):
        variance, raw_samples = policy_variance(
            policy_class="PQCBorn",
            policy_config_kwargs=variance_config(
                n_qubits, policy_name
            ),
            action=0,
            qubit_list=[n_qubits],
            n_samples=500,
            batch_size=500,
            parallel=False,
            state=torch.zeros(4),
        )
```

Listing 7: Controlled use of `policy_variance`. The notebook's `variance_config` returns the complete width-specific dictionary from Listing 6.

Fixing `action=0` is important: when `action=None`, the helper first samples an action and records the probability of that sampled action, which is a different random variable. Likewise, passing `state=torch.zeros(4)` isolates initialization variance at one input; omitting the state also resamples an observation. In the plotted one-layer ring family, the parity estimate decreases from 3.41×10^{-2} at three qubits to 1.34×10^{-3} at nine, whereas the corresponding majority estimate changes from 3.22×10^{-2} to 1.78×10^{-2}. This is an empirical diagnostic for the stated initialization ensemble, not a substitute for the architecture-specific variance bounds.

4.1.8 Contextual bandits and AWR

`ContextualBanditTrainer` implements an advantage-weighted regression (AWR) update for one-step tasks [19]. A replay tuple (s_i, a_i, r_i) receives detached weight

$$w_i = \min\left(w_{\max}, \exp\left[\frac{r_i - \bar{r}}{T}\right]\right), \qquad \mathcal{L}_{\mathrm{AWR}} = -\frac{1}{N}\sum_i w_i \log \pi(a_i \mid s_i), \tag{36}$$

where $\bar{r}$ is an exponential-moving-average baseline. The trainer evaluates samples one at a time when a quantum layer does not support the requested batch layout, preserving gradients while avoiding a false assumption of vectorized quantum execution.

The companion `ContextualBanditEnv` generates bounded continuous contexts with configurable deterministic, stochastic, raw-dense, or normalized-dense rewards. It exposes optimal actions, oracle value, reward bounds, and serialization. Its nominal oracle and bounds assume nonnegative reward scaling, and the normalized oracle omits the step formula's epsilon adjustment; reported regret should respect those conventions.

QASP feature: `ContextualBanditTrainer` and `ContextualBanditEnv`

A matched one-step research workflow: the environment supplies bounded contexts, configurable rewards, and an oracle, while the trainer applies advantage-weighted regression to any logits- or probability-valued QASP policy. Replay weighting is detached from quantum differentiation.

4.1.9 PPO and TorchRL

The module-level `PPO` utility implements clipped proximal policy optimization with a separate value network and generalized advantage estimation (GAE) [20]. With probability ratio $r_t(\boldsymbol{\theta})$ and advantage $\widehat{A}_t$, the actor term is

$$\mathcal{L}_{\mathrm{clip}} = -\mathbb{E}_t\left[\min\left(r_t\widehat{A}_t, \mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)\widehat{A}_t\right)\right]. \tag{37}$$

It is combined with value regression, entropy regularization, rollout minibatches, and gradient clipping. Each rollout resets its environment, and both true termination and time-limit truncation mask GAE continuation.

The maintained `TorchRLTrainer` wraps a PQC in a probabilistic categorical actor and can run manual REINFORCE or a supplied TorchRL loss. The separate `qasp.rl.trl` collector/PPO code is a historical prototype with known setup and post-processing limitations, not a second validated PPO route.

QASP feature: `PPO` and `TorchRLTrainer`

The maintained module-level `PPO` implements clipped updates and GAE directly; `TorchRLTrainer` adapts a quantum module to TensorDict actors and supplied TorchRL losses. Compatibility depends on the optional TorchRL version, and training-return summaries should not be relabelled as independent evaluation performance.

4.2 Value-based reinforcement learning

Value-based agents learn unconstrained action values and derive behavior from their ordering, rather than parameterizing a normalized action distribution. In QASP, the quantum model supplies differentiable Pauli features to a Q-function, while replay, Bellman targets, exploration, and target-network updates remain classical. This separation lets the same trainer compare `PQDQN` models and classical baselines under an identical interaction and evaluation contract.

4.2.1 Quantum Q-functions

Value-based reinforcement learning requires unconstrained action values rather than a categorical policy [21, 22]. The `PQDQN` module therefore reuses QASP's feature maps, ansatzes, devices, and derivative methods, but gives measurement results a separate Q-function contract. For Pauli words $P_1, \ldots, P_J$, define the raw quantum expectations

$$m_j(s; \boldsymbol{\theta}, \boldsymbol{\phi}) = \langle \psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(s) | P_j | \psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(s) \rangle. \tag{38}$$

A readout matrix $C \in \mathbb{R}^{R \times J}$ produces quantum features

$$\varphi_r(s) = \sum_{j=1}^{J} C_{rj} m_j(s), \qquad \boldsymbol{\varphi}(s) = C\boldsymbol{m}(s), \tag{39}$$

and the optional affine classical head gives

$$Q_{\boldsymbol{\Theta}}(s, a) = \sum_{r=1}^{R} W_{ar} \varphi_r(s) + b_a, \qquad \boldsymbol{\Theta} = (\boldsymbol{\theta}, \boldsymbol{\phi}, W, \boldsymbol{b}). \tag{40}$$

There is no softmax or probability clipping: `forward` returns one real action value per action. Setting `fc_layer=False` returns $\boldsymbol{\varphi}$ directly and consequently requires $R = A$; because expectation features are bounded, this direct mode is usually unsuitable for long-horizon returns. For flat shared lists and named presets, `fc_bias` defaults to `True`; for nested per-action sums it defaults to `False`. Either default can be overridden explicitly.

QASP feature: `PQDQN`, `create_qfunction`, and `draw_qfunction`

`PQDQN` is an end-to-end differentiable PyTorch action-value model. Flat Pauli lists expose shared features; nested lists define one unit-weighted sum per action; and `"parity-like"` or `"majority-modulo"` expose the distinct non-identity Pauli-Z terms in the support of the corresponding decoder-projector decomposition. Each supported Pauli expectation reaches the trainable head independently; preset projector coefficients are not applied. The model exposes its resolved Pauli strings and feature matrix, while `draw_qfunction` exports its executable circuit as Matplotlib or publication-ready Quantikz. With `device="lightning.pp"` and `diff_method="backprop"`, the same Q-function uses QASP's differentiable Pauli engine; gradients reach circuit, input-scaling, and head parameters, while batches are currently propagated sample by sample.

For a flat observable list, C is the identity. A nested list contains one nonempty group per action and sums that group's expectations, with leading minus signs allowed in Pauli specifications. Named readouts instead compute the Walsh–Pauli decomposition only to identify its support, remove the identity, and deduplicate the surviving terms. In the default `preset_mode="features"`, $C = I$ on that support: the Q-network does *not* reconstruct $\langle \Pi_a \rangle$ or an action probability. Instead, W learns an independent coefficient for every feature and b_a supplies an independent constant offset for every output action. The historical reconstruction remains available only through the explicit compatibility setting `preset_mode="projectors"` with `fc_bias=False`.

Table 7: The four `PQDQN` readouts used in the corrected CartPole study. Named presets use the decomposition support as independent features, not the decomposition coefficients. Readout-parameter counts include W and any bias but exclude circuit and input-scaling parameters.

Readout	Quantum expectations and feature map	Affine-head contract	Readout parameters
Shared	Z_0, Z_1, Z_2, Z_3 with $C = I_4$	Four shared features and bias, $\boldsymbol{Q} = W\boldsymbol{z} + \boldsymbol{b}$.	$2 \times 4 + 2 = 10$
Per-action control	$(I + Z_0 + Z_2,\ I + Z_1 + Z_3)$	Two fixed sums, followed by a bias-free 2×2 head.	$2 \times 2 = 4$
Parity $a = 2$	$Z_0Z_1Z_2Z_3$ with $C = I_1$	One full-register parity feature and two-output bias.	$2 \times 1 + 2 = 4$
Majority $a = 2$	All 15 non-identity four-wire Z products with $C = I_{15}$	Fifteen independent support features and two-output bias.	$2 \times 15 + 2 = 32$

The fast Walsh–Hadamard construction avoids dense operator matrices; the present guard permits at most 16 measured wires and at most 2^{22} projector-by-basis or feature-matrix entries.

The feature-construction size is selected by `preset_actions`, whereas `n_actions` always specifies the number of final environment Q-values. These values may differ. On four qubits, binary parity (`preset_actions` = 2) supplies only $Z_0Z_1Z_2Z_3$. Four-way parity supplies

$$\boldsymbol{z}_{\mathrm{par},4}(s) = (\langle Z_0Z_1Z_2Z_3\rangle_s, \langle Z_0Z_1Z_2\rangle_s, \langle Z_3\rangle_s)\,, \qquad \boldsymbol{Q}(s) = W\boldsymbol{z}_{\mathrm{par},4}(s) + \boldsymbol{b}, \tag{41}$$

where $W \in \mathbb{R}^{2\times 3}$ still produces two CartPole actions. This four-way preset illustrates the API distinction; it is not one of the four readouts in the final CartPole comparison.

Majority presets follow the same support-only rule. For three wires the signed binary-majority observable has the familiar expansion

$$M_3 = \frac{1}{2}\,(Z_0 + Z_1 + Z_2 - Z_0Z_1Z_2)\,, \tag{42}$$

but the default PQDQN receives the four expectations $\langle Z_0\rangle$, $\langle Z_1\rangle$, $\langle Z_2\rangle$, and $\langle Z_0Z_1Z_2\rangle$ separately; it does not apply the displayed $\pm 1/2$ coefficients. For the study's explicit even block $\{0, 1, 2, 3\}$, strict majority with ties mapped to zero has support on all 15 non-identity Z products. The supported four-way construction with blocks $\{0, 1, 2\}$ and $\{3\}$ instead yields nine terms: $Z_0Z_1Z_2Z_3$, $Z_0Z_1Z_2$, Z_0Z_3, Z_1Z_3, Z_2Z_3, and the four local Z_i. That four-way construction was deliberately excluded from the final comparison. Explicit blocks are part of the experiment definition; automatic balanced odd blocks can leave one measured wire unused.

Listing 8 shows the public construction used by the executable CartPole study. Batches are broadcast through the compatible angle embedding, and gradients flow jointly through the circuit angles, trainable input scales, and the classical head.

```python
from qasp import create_qfunction, draw_qfunction, DeepQLearning

readouts = {
    "shared": {
        "observables": ["Z 0", "Z 1", "Z 2", "Z 3"],
        "fc_bias": True,
    },
    "per_action": {
        "observables": [
            ["I 0", "Z 0", "Z 2"],
            ["I 0", "Z 1", "Z 3"],
        ],
        "fc_bias": False,
    },
    "parity_a2": {
        "observables": "parity-like",
        "preset_actions": 2, "fc_bias": True,
    },
    "majority_a2": {
        "observables": "majority-modulo",
        "preset_actions": 2,
        "majority_blocks": [[0, 1, 2, 3]],
        "fc_bias": True,
    },
}
base = {
```

```
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 4, "rotations": "YZ",
        "reuploading": True,
        "include_imprimitive": True,
        "imprimitive": "CNOT",
        "imprimitive_pattern": "ring",
        "include_final_imprimitive": True,
    },
    "feature_map": "AngleEmbedding",
    "feature_map_config": {
        "features": 4, "rotations": "Y",
        "input_scaling": True,
    },
    "fc_layer": True,
    "device": "default.qubit",
    "diff_method": "backprop",
    "batch_mode": "broadcast",
}
qfunction = create_qfunction(
    {**base, **readouts["majority_a2"]},
    n_qubits=4, n_actions=2,
)
with DeepQLearning(
    qfunction, env_name="CartPole-v1", seed=7,
    lr=1e-2, gamma=0.99, batch_size=64,
    learning_starts=256, train_frequency=4,
    target_update_interval=100,
    epsilon_decay_steps=8000,
) as agent:
    records = agent.train(num_episodes=400)
    held_out = agent.evaluate(num_episodes=100, seed=30000)

parity_a4 = create_qfunction(
    {**base, "observables": "parity-like",
     "preset_actions": 4, "fc_bias": True},
    n_qubits=4, n_actions=2,
)
assert parity_a4.observable_strings == (
    "ZZZZ 0 1 2 3", "ZZZ 0 1 2", "Z 3",
)
assert parity_a4.fc_layer.in_features == 3
assert parity_a4.fc_layer.out_features == 2

tikz = draw_qfunction(
    qfunction, output="tikz",
    tikz_options={"standalone": True, "compress": True},
)
```

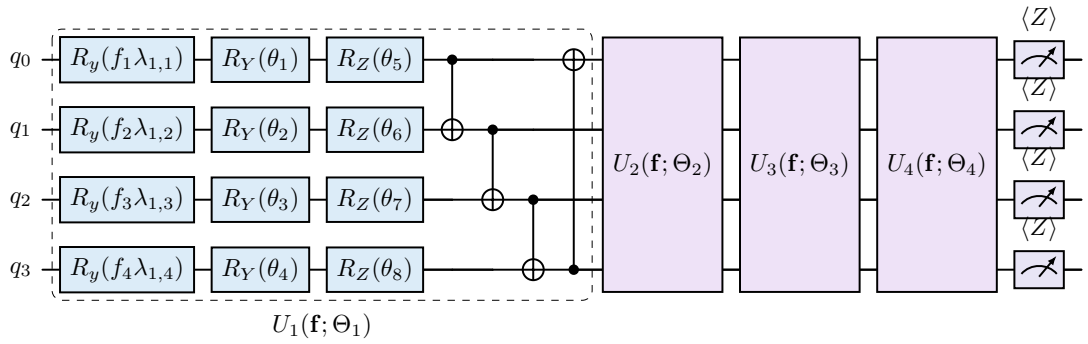
Listing 8: Constructing and training the four-layer majority-$a = 2$ PQDQN, followed by an API-only parity-$a = 4$ example with two environment actions.

4.2.2 Deep Q-learning: replay, targets, and exploration

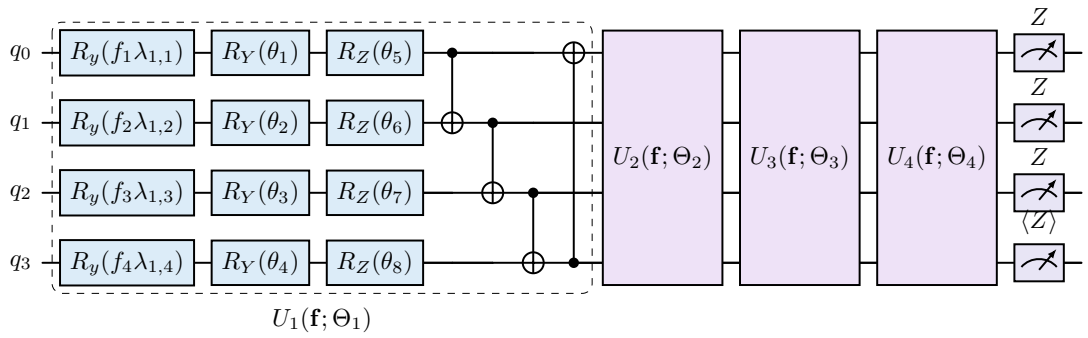
`DeepQLearning` accepts any differentiable module mapping observations to action values. A bounded uniform replay buffer stores $(s_t, a_t, r_t, s_{t+1}, d_t)$, where d_t marks true environment termination. For its default Double-DQN update, the online model selects the next action and the frozen target model evaluates it:

$$a^*(s') = \arg\max_b Q_{\boldsymbol{\Theta}}(s', b),$$
$$y = r + \gamma(1 - d) Q_{\bar{\boldsymbol{\Theta}}}(s', a^*(s')), \tag{43}$$
$$\mathcal{L}_{\mathrm{DQN}} = \frac{1}{B} \sum_{i=1}^{B} \mathrm{Huber}(Q_{\boldsymbol{\Theta}}(s_i, a_i) - y_i)\,. \tag{44}$$

Setting `double_dqn=False` replaces the online-network selection in Equation (43) by the maximum target-network value, recovering the conventional DQN target. Time-limit truncation and the trainer's `max_steps` cap end a rollout but do not set $d = 1$, so the target continues to bootstrap; only true environment termination masks the continuation value.



(a) Shared four-feature readout with affine bias. Four input-scaled R_Y encoders, trainable R_Y/R_Z rotations, and one CNOT ring are expanded; the remaining re-uploading layers are compressed as U_2, U_3, U_4.



(b) Parity $a = 4$ feature preset with two output actions. The exported metadata records three independent products: $Z_0Z_1Z_2Z_3$, $Z_0Z_1Z_2$, and Z_3. This panel illustrates `preset_actions`; the CartPole comparison uses parity $a = 2$.

Figure 12: Representative `PQDQN` circuits exported from the constructed models with `draw_qfunction(output="tikz")`. Every study circuit has four CNOT rings, hence 16 CNOT gates in total. The per-action and majority–modulo models have the same gate skeleton. Measurement meters show local bases; the Pauli products, trainable affine head, and bias act at the measurement/classical boundary and are therefore stated in Table 7, not drawn as quantum gates.

Exploration is controlled independently of learning. If F is the cumulative number of environment frames, QASP uses the clipped linear schedule

$$\epsilon(F) = \epsilon_0 + \min\left(1, \frac{F}{F_{\text{decay}}}\right)(\epsilon_{\text{final}} - \epsilon_0), \tag{45}$$

and chooses a uniformly random action with probability $\epsilon(F)$. Otherwise it evaluates the online Q-function in evaluation mode and takes its argmax. This separation makes it possible to change the exploration horizon without changing the replay warm-up or target-network cadence.

The optimization and data-collection schedules are independently configurable. The user can choose `optimizer_class` and `optimizer_kwargs`, the learning rate, discount, minibatch size, replay capacity, replay warm-up `learning_starts`, environment-step update frequency, and the number of gradient steps per update event. The frozen target is synchronized every `target_update_interval` optimizer updates, rather than environment steps. Exploration follows a linear ϵ schedule controlled by its initial value, final value, and decay horizon in environment frames. Huber loss, a configurable gradient-norm cap, and explicit nonfinite checks protect both classical and differentiable quantum Q-functions.

The trainer accepts an environment name or a caller-owned Gymnasium instance, requires zero-based discrete actions, and checks that the Q-function output width matches the action space. Observations are optionally passed through a deterministic `observation_transform`, flattened, and moved to the Q-function's dtype and device. A local seeded generator controls exploration and replay sampling; episode k is reset with `seed + k`. Repeated `train` calls deliberately continue the replay memory, frame count, update count, episode count, and exploration schedule. Each returned record contains the episode return and length, cumulative frames and updates, current ϵ, and mean minibatch loss.

Three counters make the scheduling semantics explicit. The frame counter drives Equation (45), `learning_starts`, and `train_frequency`; the optimizer-update counter drives target-network synchronization; and the episode counter determines the next reset seed. Consequently, one update event may perform several optimizer steps through `gradient_steps`, and target synchronization can occur more often than the nominal frame frequency might suggest. An update is attempted only after the warm-up and on the requested frame cadence, while `optimize` also requires at least `batch_size` stored transitions. Replay states are detached CPU clones; a sampled minibatch is moved back to the Q-function's current dtype and device immediately before evaluation. This keeps replay storage backend-independent while preserving gradients through the online model during the selected-action Huber loss.

Greedy `evaluate` runs in a distinct, deterministically seeded environment and leaves exploration, replay, counters, and the model's training/evaluation mode unchanged. When an environment instance is supplied for training, the caller retains ownership and must provide a separate evaluation instance; environments created from a name are managed by `close` or the context manager. Model weights can be checkpointed, but an exact trainer resume additionally requires optimizer, replay-buffer, and random-number-generator state, which `DeepQLearning` does not save automatically. The trainer seed controls its interaction and replay streams; it does not initialize model weights, so Torch must be seeded before model construction when matched initializations are required.

QASP feature: `DeepQLearning`

A manual PyTorch DQN trainer for any differentiable module returning one value per action. Its main customization groups are:

- *optimization:* `lr`, `optimizer_class`, `optimizer_kwargs`, `gamma`, `batch_size`, and `max_grad_norm`;
- *replay and updates:* `replay_capacity`, `learning_starts`, `train_frequency`, and `gradient_steps`;
- *targets and exploration:* `double_dqn`, `target_update_interval`, `epsilon_start`, `epsilon_end`, and `epsilon_decay_steps`;
- *interaction:* an environment name or instance, `max_steps`, `seed`, and a deterministic `observation_transform`.

Uniform replay, Huber loss, frozen-target updates, nonfinite checks, and state-preserving greedy evaluation are built in. Because the trainer is independent of `PQDQN`, the same interaction and evaluation code can be used for quantum models and classical Q-function baselines.

Table 8: `DeepQLearning` customization points. "Frame" means one environment transition; "update" means one optimizer step. The distinction is important when `train_frequency` or `gradient_steps` differs from one.

Control	Default	Effect
`qfunction`	required	Any differentiable module with output shape $(\cdots, \lvert\mathcal{A}\rvert)$; this may be a `PQDQN` or a classical baseline.
`env_name`	`"CartPole-v1"`	Gymnasium identifier or a caller-owned environment with zero-based discrete actions.
`lr`, `optimizer_class`, `optimizer_kwargs`	10^{-3}, Adam, `None`	Replaces the optimizer without changing the interaction loop; keyword arguments are forwarded to its constructor.
`gamma`, `batch_size`	0.99, 64	Discount and uniform replay minibatch size.
`replay_capacity`, `learning_starts`	10,000, 256 frames	FIFO memory budget and earliest frame at which updates may begin.
`train_frequency`, `gradient_steps`	1 frame, 1 update	How often an update event occurs and how many replay updates it performs.
`double_dqn`, `target_update_interval`	`True`, 100 updates	Double-DQN or conventional DQN target and hard-copy cadence.
`epsilon_start`, `epsilon_end`, `epsilon_decay_steps`	1.0, 0.05, 10,000 frames	Endpoints and duration of the linear ϵ-greedy schedule.
`max_steps`, `max_grad_norm`	500, 10	Episode/evaluation cap and global gradient-norm clipping threshold.
`seed`, `observation_transform`	0, `None`	Local exploration/replay/reset reproducibility and deterministic feature preprocessing before flattening.

Listing 9 illustrates how the controls compose. Here a caller-owned environment is paired with deterministic input compression, an AdamW optimizer, two replay updates every four frames, vanilla DQN targets, and a shorter exploration schedule. The same trainer can instead receive a quantum `PQDQN` with no change to the interaction code.

```python
import gymnasium as gym
import numpy as np
import torch
from qasp import DeepQLearning

def compress_cartpole(observation):
    scale = np.array([2.4, 3.0, 0.21, 3.5])
    return np.tanh(np.asarray(observation) / scale)

train_env = gym.make("CartPole-v1")
eval_env = gym.make("CartPole-v1")
agent = DeepQLearning(
    qfunction, train_env,
    optimizer_class=torch.optim.AdamW,
    optimizer_kwargs={"weight_decay": 1e-4},
    lr=3e-3, gamma=0.98,
    batch_size=32, replay_capacity=20_000,
    learning_starts=512,
    train_frequency=4, gradient_steps=2,
    double_dqn=False, target_update_interval=250,
    epsilon_start=0.9, epsilon_end=0.02,
    epsilon_decay_steps=20_000,
    max_steps=500, max_grad_norm=5.0,
    observation_transform=compress_cartpole,
    seed=19,
)
try:
    records = agent.train(num_episodes=300, log_every=25)
    returns = agent.evaluate(
        num_episodes=50, seed=40_000, env=eval_env
    )
finally:
    train_env.close()  # caller owns both environment instances
    eval_env.close()
```

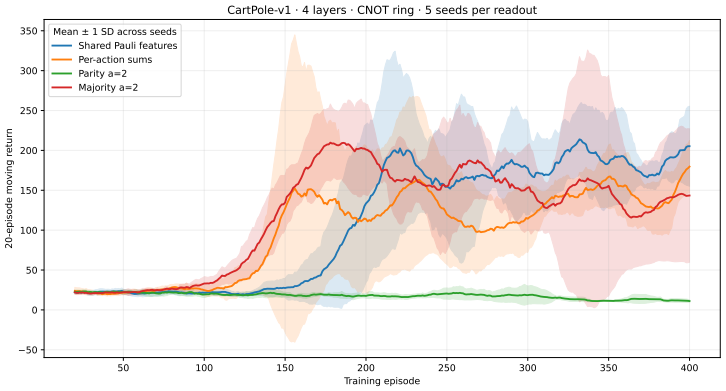
Listing 9: Customizing optimization, replay cadence, exploration, preprocessing, and environment ownership in `DeepQLearning`.

The class intentionally keeps the research contract small: replay is uniform, target transfers are hard copies, the loss is Huber, and the exploration schedule is linear. Prioritized replay, n-step returns, soft target updates, reward shaping, callbacks, and full trainer-state serialization are not constructor options in the current release. Such changes should therefore be implemented explicitly rather than inferred from similarly named DQN frameworks. Users can nevertheless call `select_action`, `optimize`, and `sync_target` directly when building a controlled outer loop around QASP's tested primitives.

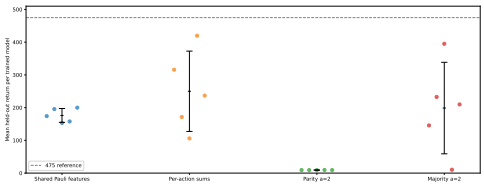
4.2.3 CartPole value-readout study

The executed notebook `examples/cartpole_pqdqn_deep_q_learning.ipynb` compares the four readouts in Table 7. Every model uses four qubits, four R_Y/R_Z ansatz layers, re-uploaded R_Y angle encoding with trainable input scales, a CNOT ring in every layer including the final layer, and a fully connected head. Shared, parity-$a = 2$, and majority-$a = 2$ use affine bias; the original identity-containing per-action sums are retained as a bias-free control. Consequently the number of trainable readout parameters differs across configurations as reported in Table 7. Training uses analytic `default.qubit` simulation with backpropagation and broadcast batches. For each readout, seeds 7–11 receive 400 training episodes with Adam learning rate 10^{-2}, $\gamma = 0.99$, batch size 64, learning beginning after 256 frames, one update every four frames, target synchronization every 100 updates, and linear ϵ decay over 8000 frames. Replay capacity is 10,000 and gradients are clipped to norm 10.

Each trained seed is evaluated greedily on 100 held-out episodes with common reset seeds 30000–30099. The five trained seeds—not the resulting 500 evaluation episodes—are the independent experimental units for each readout. Moving curves first compute a trailing 20-episode mean within each seed and then report the across-seed mean and sample standard deviation; the first 19 episodes are omitted. Shading is ± 1 sample standard deviation, not a confidence interval, and can extend below zero even though CartPole returns are



(a) Mean trailing 20-episode training return; shading is ±1 sample SD across the five trained seeds.



(b) One point per trained seed: mean return over its 100 common-seed held-out episodes. Bars show mean ±1 sample SD across trained seeds.

Figure 13: Completed PQDQN CartPole comparison. The horizontal 475 line is a reference, not a stopping rule. All 8,000 training episodes, seed-level evaluations, model checkpoints, manifests, and circuit exports are retained as machine-readable experiment artifacts.

Table 9: PQDQN CartPole outcomes. Held-out uncertainty is the sample standard deviation across five independently trained seeds; it is not the standard error and does not treat evaluation episodes as independent models.

Readout	Held-out mean for trained seeds 7–11	Across-seed held-out mean ± SD	Mean final-20 training return
Shared Pauli features	174.12, 195.58, 153.32, 157.80, 199.92	176.148 ± 21.240	205.28
Per-action sums	315.89, 171.44, 106.06, 419.78, 236.82	249.998 ± 122.710	179.64
Parity $a = 2$	9.19, 9.19, 9.19, 9.49, 9.19	9.250 ± 0.134	11.08
Majority $a = 2$	145.63, 232.69, 395.11, 10.36, 209.77	198.712 ± 139.770	143.43

positive. Equal episode budgets yielded unequal environment-step budgets, ranging from 6,347 to 63,735 frames across the 20 runs.

Under this configuration and budget, per-action sums have the highest observed mean held-out return. Shared local-Z features have a lower observed mean but substantially smaller between-seed variation. The 15-feature majority model learns useful control for several seeds but fails on one, whereas the single-feature parity model does not learn useful control. No trained model's held-out mean reaches 475. These comparisons jointly change feature content, bias, and the number of trainable readout parameters; in particular, the corrected majority result is not a bias-only ablation against the historical projector-reconstruction experiment. With only five independently trained seeds per readout and no classical function-approximation baseline, the data do not establish reliable solution of CartPole, statistically demonstrated superiority, general readout rankings, or quantum advantage.

5 Quantum kernels

The kernel layer now exposes a complete high-level path from a QASP feature map to a train, validation, or test Gram matrix. It reuses the same circuit templates, builder, device selection, differentiation methods, and PyTorch connection as the other task modules. Given an encoding circuit

$$|\psi_{\boldsymbol{\theta}}(\boldsymbol{x})\rangle = U_{\boldsymbol{\theta}}(\boldsymbol{x})|0\rangle^{\otimes n}, \tag{46}$$

a kernel is evaluated pairwise to form

$$[K_{XY}]_{ij} = k_{\boldsymbol{\theta}}(\boldsymbol{x}_i, \boldsymbol{y}_j), \qquad K_{XX} \in \mathbb{R}^{N \times N}. \tag{47}$$

This is the quantum-feature-space construction used by kernel methods [23, 24]. In QASP, the resulting array can be passed directly to a precomputed-kernel estimator; when the circuit is trainable, the same Gram

computation can instead remain inside a PyTorch computation graph.

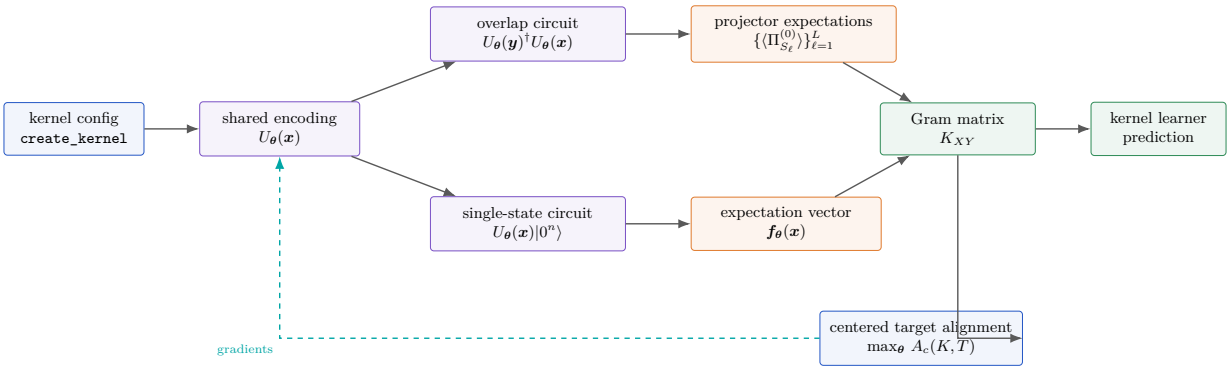


Figure 14: The QASP quantum-kernel stack. One resolved encoding circuit feeds either the projector-configurable `FidelityKernel` or single-state expectation features for `ProjectedKernel`. The pairwise result is assembled into a Gram matrix for a classical kernel learner. For trainable encodings, centered kernel–target alignment differentiates through the same Gram computation and updates the circuit parameters.

5.1 Shared construction and factory contract

The kernel factory mirrors `create_model`. A parameter-free kernel applies the selected `AngleEmbedding` or `NeuralEmbedding` feature map `reps` times. Supplying `ansatz` and `ansatz_config` instead constructs a trainable $U_{\boldsymbol{\theta}}(\boldsymbol{x})$ from `IQPAnsatz` or `AngleAnsatz`; the same $\boldsymbol{\theta}$ is shared by both sides of every pairwise circuit. Device, differentiation method, shots, and MPI selection are forwarded to `CircuitBuilder`, so kernels follow the same execution contract as QASP models. Inputs are checked to be finite arrays of shape (N, d), and a cross Gram matrix accepts independent X and Y collections. `gram(..., return_tensor=True)` preserves the graph for trainable quantum weights, whereas NumPy-returning convenience paths detach it. These input conversion paths do not promise derivatives with respect to user-supplied input arrays.

QASP feature: `create_kernel`

`create_kernel(config, n_qubits)` resolves one of two continuous-input classes: `FidelityKernel` or `ProjectedKernel`. Global and localized fidelity are configurations of the same class, selected only by the optional `projectors` list. Nontrainable feature maps use a fixed encoding; adding an ansatz creates a PyTorch-backed trainable kernel whose `parameters()` can be optimized.

QASP feature: `draw_kernel`

`draw_kernel` accepts either a resolved kernel or a configuration and traces the circuit that is actually evaluated. Fidelity kernels show the two-input overlap circuit, whereas a projected kernel shows its single-state expectation circuit. Setting `output="tikz"` returns editable Quantikz; `tikz_options` controls standalone output, symbolic parameters, wire labels, gate colors, and repeated-layer compression.

```python
import numpy as np
from qasp import create_kernel, draw_kernel

X_train = np.array([[0.10, 0.20, 0.15], [0.20, 0.10, 0.12],
                    [0.90, 1.10, 1.00], [1.00, 0.80, 0.92]])
X_test = np.array([[0.15, 0.25, 0.18],
                   [0.95, 0.85, 0.90]])

config = {
    "kernel_type": "fidelity",
    "feature_map": "AngleEmbedding",
    "feature_map_config": {
        "rotations": "Y", "imprimitive_pattern": "ring"
    },
    "reps": 2,
    "device": "default.qubit",
}
kernel = create_kernel(config, n_qubits=3)
K_train = kernel.gram(X_train)          # shape (4, 4)
K_test = kernel.gram(X_test, X_train)   # shape (2, 4)
```

```
tikz = draw_kernel(
    kernel, x=X_train[0], y=X_train[1], output="tikz",
    tikz_options={"standalone": True,
                  "symbolic_parameters": True},
)
```

Listing 10: Constructing, evaluating, and drawing a global-fidelity kernel through the public factory.

5.2 One projector-configurable fidelity kernel

Global and localized overlap similarities are now two configurations of the same `FidelityKernel`, rather than separate classes. Define the overlap state

$$|\varphi_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{y})\rangle = U_{\boldsymbol{\theta}}(\boldsymbol{y})^{\dagger}U_{\boldsymbol{\theta}}(\boldsymbol{x})|0^n\rangle. \tag{48}$$

For a nonempty wire subset $S \subseteq [n]$, QASP's label `"P i j ..."` represents the zero-state subsystem projector

$$\begin{aligned}\Pi_S^{(0)} &:= \left(\bigotimes_{i\in S}|0\rangle\langle 0|_i\right)\otimes I_{\bar{S}} \\ &= 2^{-|S|}\prod_{i\in S}(I+Z_i) = 2^{-|S|}\sum_{T\subseteq S}Z_T, \qquad Z_T := \prod_{i\in T}Z_i,\end{aligned} \tag{49}$$

with $Z_{\varnothing} = I$. If the configuration contains $\mathcal{P} = (S_1,\ldots,S_L)$, the implemented similarity is the arithmetic mean of the selected projector probabilities,

$$k_{\mathcal{P}}(\boldsymbol{x},\boldsymbol{y}) = \frac{1}{L}\sum_{\ell=1}^{L}\langle\varphi_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{y})|\Pi_{S_\ell}^{(0)}|\varphi_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{y})\rangle. \tag{50}$$

Omitting `projectors` creates the single default label containing every wire, $\mathcal{P} = ([n])$. Equation (50) then reduces to the global fidelity

$$k_{[n]}(\boldsymbol{x},\boldsymbol{y}) = \left|\langle 0^n|U_{\boldsymbol{\theta}}(\boldsymbol{y})^{\dagger}U_{\boldsymbol{\theta}}(\boldsymbol{x})|0^n\rangle\right|^2 = \left|\langle\psi_{\boldsymbol{\theta}}(\boldsymbol{y})\mid\psi_{\boldsymbol{\theta}}(\boldsymbol{x})\rangle\right|^2. \tag{51}$$

Thus the default exact train Gram matrix is positive semidefinite. Proper subsets localize the comparison, while lists may mix different locality scales. Every configuration satisfies $0 \le k_{\mathcal{P}} \le 1$ and $k_{\mathcal{P}}(\boldsymbol{x},\boldsymbol{x}) = 1$; an average involving proper subsets is not, in general, guaranteed to be positive semidefinite. QASP symmetrizes a train Gram matrix but leaves its spectrum visible to the user.

For example, on three qubits `projectors=["P 0 1", "P 0 1 2"]` gives

$$k_{\mathcal{P}}(\boldsymbol{x},\boldsymbol{y}) = \frac{1}{2}\left(\langle\Pi_{\{0,1\}}^{(0)}\rangle_{\varphi} + \langle\Pi_{\{0,1,2\}}^{(0)}\rangle_{\varphi}\right), \tag{52}$$

where the exact Pauli decompositions are

$$\begin{aligned}\Pi_{\{0,1\}}^{(0)} &= \tfrac{1}{4}(I + Z_0 + Z_1 + Z_0Z_1), \\ \Pi_{\{0,1,2\}}^{(0)} &= \tfrac{1}{8}(I + Z_0 + Z_1 + Z_2 + Z_0Z_1 + Z_0Z_2 + Z_1Z_2 + Z_0Z_1Z_2).\end{aligned} \tag{53}$$

The first term probes a two-qubit subsystem and the second is the global three-qubit fidelity. This is an exact projector average, not the earlier interface based on a manually supplied list of Z observables.

QASP feature: `FidelityKernel(projectors=...)`

The unified overlap-fidelity class. With no `projectors` argument it measures the global all-zero projector. Each label `"P i j ..."` selects the joint all-zero projector on those wires and the identity elsewhere; multiple entries are averaged. The object exposes normalized or unnormalized train/cross Gram matrices, differentiable tensor output, `projector_labels`, `projector_wires`, and Quantikz drawing. The removed `"local"` and `"local_fidelity"` factory names now fail explicitly, preventing silent use of the superseded class contract.

```python
global_kernel = create_kernel({
    **config, "kernel_type": "fidelity",
}, n_qubits=3)

localized_kernel = create_kernel({
    **config, "kernel_type": "fidelity",
    "projectors": ["P 0 1", "P 0 1 2"],
}, n_qubits=3)

K_global = global_kernel.gram(X_train)
K_localized = localized_kernel.gram(X_train)
assert localized_kernel.projector_wires == [[0, 1], [0, 1, 2]]
```

Listing 11: The same `FidelityKernel` class in global and mixed-locality projector configurations.

5.3 Projected expectation-feature kernel

The projected kernel replaces a global state overlap by a vector of measured features [25]. For observables $\mathcal{O} = \{O_1, \ldots, O_m\}$, QASP evaluates

$$f_\ell(\boldsymbol{x}) = \langle 0^n | U_{\boldsymbol{\theta}}(\boldsymbol{x})^\dagger O_\ell U_{\boldsymbol{\theta}}(\boldsymbol{x}) | 0^n \rangle, \qquad \boldsymbol{f}_{\boldsymbol{\theta}}(\boldsymbol{x}) = (f_1, \ldots, f_m), \tag{54}$$

followed by a weighted Gaussian kernel in this measured feature space,

$$k_{\mathrm{P}}(\boldsymbol{x}, \boldsymbol{y}) = \exp\left[-\gamma \sum_{\ell=1}^{m} w_\ell \big(f_\ell(\boldsymbol{x}) - f_\ell(\boldsymbol{y})\big)^2\right], \quad \gamma \geq 0, \quad w_\ell \geq 0. \tag{55}$$

The defaults are all one-qubit X_i, Y_i, and Z_i expectations with unit weights. Nonnegative weights make Equation (55) a classical Gaussian kernel on an explicitly computed quantum feature vector, hence it is positive semidefinite and has a unit diagonal. The `features(X)` method exposes this intermediate representation for analysis or reuse.

QASP feature: `ProjectedKernel`

A weighted radial-basis kernel over quantum expectation features. Users may select arbitrary valid Pauli strings, inspect the feature matrix, and control the bandwidth γ and nonnegative feature weights. The circuit is evaluated once per input feature vector rather than once for every input pair.

5.4 Trainable kernels and centered target alignment

Adding an ansatz makes each preceding kernel depend on shared variational parameters $\boldsymbol{\theta}$. When the Gram matrix is requested as a tensor, QASP preserves its autodifferentiation graph. The package trains these parameters by centered kernel–target alignment [26]. For labels $y_1, \ldots, y_N$, define

$$T_{ij} = \mathbf{1}\{y_i = y_j\}, \qquad H = I_N - \frac{1}{N}\mathbf{1}\mathbf{1}^{\mathsf{T}}, \qquad K_c = HKH, \quad T_c = HTH, \tag{56}$$

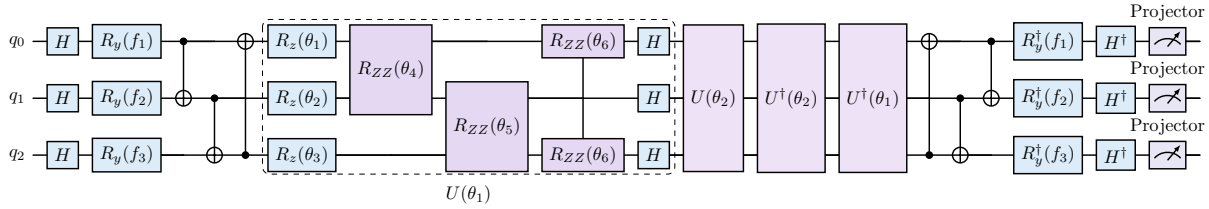
and

$$A_c(K, T) = \frac{\langle K_c, T_c \rangle_{\mathrm{F}}}{\|K_c\|_{\mathrm{F}} \|T_c\|_{\mathrm{F}}}. \tag{57}$$

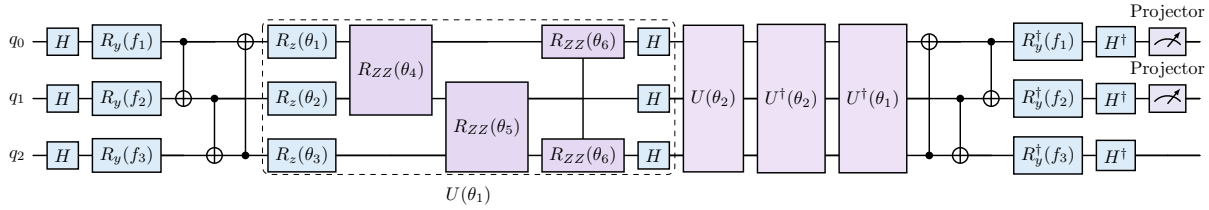
The binary or multiclass target matrix rewards large within-class similarity relative to between-class similarity. `KernelTargetAlignment` minimizes $-A_c$ with Adam by default, accepts a user optimizer, and can estimate each update from a seeded random minibatch to reduce the quadratic Gram cost.

QASP feature: `KernelTargetAlignment` and `align_kernel`

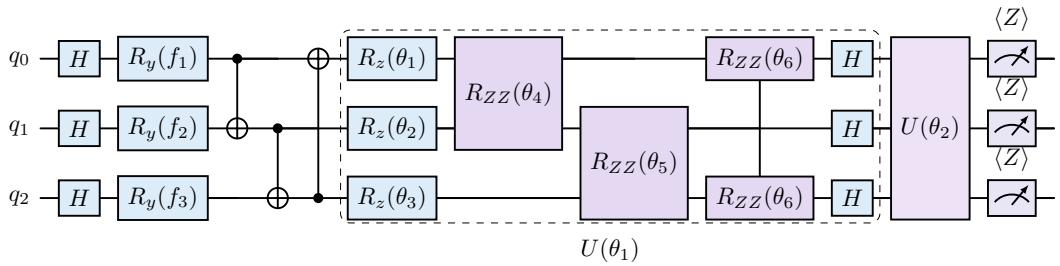
`KernelTargetAlignment.fit` differentiates centered alignment through a trainable kernel and returns loss and alignment histories. The convenience function `align_kernel` performs the same fit and returns the optimized kernel together with its history. A parameter-free kernel fails explicitly instead of silently performing a no-op optimization.



(a) Default `FidelityKernel`: the compressed IQP encoding $U_{\boldsymbol{\theta}}(\boldsymbol{x})$ is followed by $U_{\boldsymbol{\theta}}(\boldsymbol{y})^\dagger$ and the all-qubit zero projector.



(b) The same class with `projectors=["P 0 1"]`: only wires 0 and 1 enter the zero-state projector, while wire 2 remains unmeasured.



(c) `ProjectedKernel`: a single encoding circuit yields local expectation features, here $\langle Z_0 \rangle$, $\langle Z_1 \rangle$, and $\langle Z_2 \rangle$, before classical weighted-Gaussian post-processing.

Figure 15: Quantum-kernel circuits traced from executable kernel objects using QASP's `draw_kernel(output="tikz")` path. The accompanying `render_kernel_circuits.py` script retains the editable standalone Quantikz source and compiles these vector PDFs plus PNG previews. Symbolic parameters, semantic colors, inverse-layer order, and compression are supplied by QASP rather than reconstructed for the manuscript.

```python
import torch
from qasp import (KernelTargetAlignment, create_kernel,
                  target_alignment)

y_train = np.array([0, 0, 1, 1])
trainable_config = {
    "kernel_type": "fidelity",
    "feature_map": "AngleEmbedding",
    "feature_map_config": {
        "rotations": "Y", "imprimitive_pattern": "ring"
    },
    "ansatz": "IQPAnsatz",
    "ansatz_config": {
        "n_layers": 2, "imprimitive_pattern": "ring",
        "reuploading": True,
    },
    "init": "small_normal",
    "device": "default.qubit",
}
torch.manual_seed(3)
kernel = create_kernel(trainable_config, n_qubits=3)
before = target_alignment(kernel.gram(X_train), y_train)
history = KernelTargetAlignment(kernel).fit(
    X_train, y_train, steps=5, lr=0.02, seed=3, verbose=False
)
after = target_alignment(kernel.gram(X_train), y_train)
```

Listing 12: A trainable IQP fidelity kernel and centered target-alignment update.

5.5 Fisher-information kernel

`QFKernel` remains a separate, distribution-geometric construction for bit-string inputs. Given a probability-valued QNode $q_{\boldsymbol{\theta}}(z)$, it precomputes the score for each computational-basis string,

$$\boldsymbol{s}_{\boldsymbol{\theta}}(z) = \nabla_{\boldsymbol{\theta}} \log q_{\boldsymbol{\theta}}(z) = \frac{\nabla_{\boldsymbol{\theta}} q_{\boldsymbol{\theta}}(z)}{q_{\boldsymbol{\theta}}(z)}, \tag{58}$$

and constructs

$$k_{\mathrm{QF}}(z, z') = \boldsymbol{s}_{\boldsymbol{\theta}}(z)^{\mathsf{T}} \left(F + \lambda I\right)^{-1} \boldsymbol{s}_{\boldsymbol{\theta}}(z'). \tag{59}$$

The metric F can be PennyLane's quantum or classical Fisher matrix. QASP solves the regularized system rather than explicitly forming its inverse. Unlike the continuous-input feature kernels above, the current implementation enumerates every basis string and precomputes its score; its memory and circuit cost are therefore exponential in the number of qubits. Zero model probabilities can make the score division singular, and the legacy `normalize` option is currently unused. It is intended for small-system geometry experiments, including investigations motivated by vanishing-similarity concerns [27].

QASP feature: `QFKernel`

A regularized Fisher-score kernel for bit-string data. It reuses a trained probability-valued QNode, constructs quantum or classical Fisher geometry, solves the metric system, and returns a Gram matrix. Its exhaustive basis score table, zero-probability sensitivity, and unused legacy normalization flag make the present API a small-system research tool rather than a drop-in large-width kernel.

```python
from qasp import QFKernel

qf_kernel = QFKernel(qnode, params, fisher="quantum",
                     lambda_reg=1e-2)
gram_matrix = qf_kernel(data)
```

Listing 13: Constructing a Fisher-information kernel from a trained probability-valued QNode and its parameters.

5.6 Executable IQP example and validation

The release includes `examples/iqp_quantum_kernels.py` and an accompanying validation notebook. The script builds the two continuous-input kernel classes from a common three-qubit IQP configuration. It

evaluates the default global fidelity, the same `FidelityKernel` with user-defined projector subsets, and a projected kernel using all single-qubit X, Y, and Z expectations:

```
base = {
    "feature_map": "AngleEmbedding",
    "feature_map_config": {
        "rotations": "Y", "imprimitive_pattern": "ring"
    },
    "ansatz": "IQPAnsatz",
    "ansatz_config": {
        "n_layers": 2, "imprimitive_pattern": "ring",
        "reuploading": True,
    },
    "init": "zeros", "device": "default.qubit",
}

localized = create_kernel({
    **base, "kernel_type": "fidelity",
    "projectors": ["P 0 1", "P 0 2", "P 0 1 2"],
}, n_qubits=3)

projected = create_kernel({
    **base, "kernel_type": "projected",
    "observables": ["X 0", "Y 0", "Z 0",
                    "X 1", "Y 1", "Z 1",
                    "X 2", "Y 2", "Z 2"],
    "gamma": 0.5,
}, n_qubits=3)

K_localized = localized.gram(X_train)
K_projected = projected.gram(X_train)
```

Listing 14: Switching the fidelity projector list and the projected-kernel readout without changing the shared IQP circuit configuration.

For the committed example and its fixed data, the global-fidelity test-versus- train matrix has shape 2×4, and an explicit all-qubit projector `"P 0 1 2"` agrees with the default global fidelity to machine precision in the displayed run. The localized projector list is parsed as $[\{0,1\},\{0,2\},\{0,1,2\}]$. The displayed trainable run changes centered alignment from 0.9972 to 0.9973 over five deliberately short steps; it is a gradient-path smoke test, not a learning-performance claim. The focused kernel and Quantikz-export modules contain 39 passing tests covering factory construction, removal of the former local-fidelity aliases, projector parsing and averaging, train/cross Gram conventions, unit diagonals, symmetry, positive semidefiniteness where guaranteed, differentiability, parameter updates, invalid configurations, gate scheduling, measurement notation, symbolic labels, colors, standalone output, and repeated-layer compression. The notebook additionally demonstrates precomputed-kernel classifiers and circuit drawings. These examples validate software contracts on small simulators; they do not establish quantum advantage or generalization superiority.

6 Parameterized quantum classifiers

The classifier layer turns the shared QASP circuit construction into a supervised model for binary or multiclass labels, following the parameterized quantum-classifier setting exemplified by data re-uploading models [10]. It deliberately separates the quantum measurement contract from the classical meaning assigned to its outputs. A probability-readout classifier aggregates Born probabilities into genuine class probabilities, whereas an expectation-readout classifier uses observable values as signed or multiclass scores. This distinction determines which predictions, calibration map, and loss functions are mathematically appropriate.

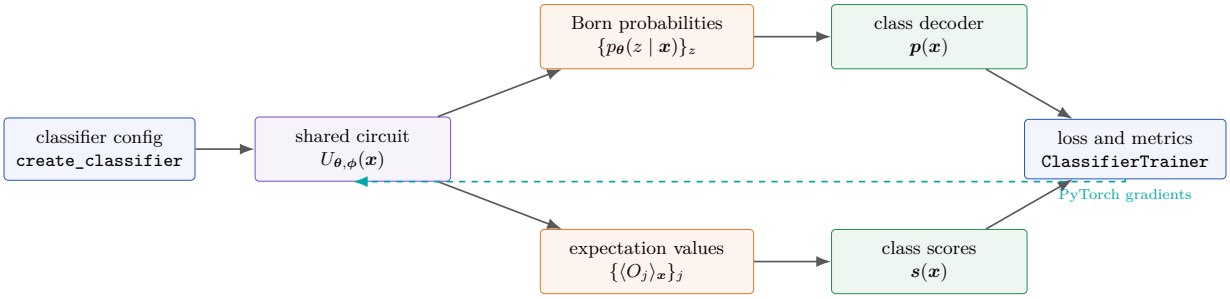


Figure 16: The QASP parameterized-classifier stack. One configuration-resolved circuit supports two explicit readout contracts. Born probabilities are partitioned into normalized class probabilities; observable expectations are interpreted as class scores and calibrated only when probabilities are requested. Both routes remain differentiable through the selected objective.

6.1 Shared construction and factory contract

For an input $\boldsymbol{x}$, QASP prepares the same parameterized feature state used throughout the package,

$$|\psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})\rangle = U_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})|0^n\rangle. \tag{60}$$

The configuration accepts `AngleEmbedding` or `NeuralEmbedding`, `AngleAnsatz` or `IQPAnsatz`, the usual device and differentiation options, and an optional preconfigured `CircuitBuilder`. The resolved `PQCClassifier` is a PyTorch module exposing `forward`, `predict`, `predict_proba`, `decision_function`, `compute_loss`, and access to the quantum-layer weights.

QASP feature: `create_classifier` and `draw_classifier`

`create_classifier(config, n_qubits, n_classes)` resolves QASP's standard feature-map and ansatz notation into a supervised PyTorch model. `draw_classifier` accepts that model or the same configuration and returns text, Matplotlib, or publication-ready Quantikz. The TikZ path traces the executable classifier circuit and supports symbolic parameters, semantic colors, wire labels, standalone output, and repeated-layer compression.

6.2 Born-probability classification

Let $M = (q_1, \ldots, q_m)$ be the ordered measured wires and $f : \{0,1\}^m \to \{0, \ldots, C-1\}$ a fixed class decoder. Probability readout implements

$$p_{\boldsymbol{\theta}}(c \mid \boldsymbol{x}) = \sum_{z : f(z) = c} \langle \psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) | \big(|z\rangle\langle z|_M \otimes I_{\bar{M}} \big) | \psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) \rangle, \qquad \sum_{c=0}^{C-1} p_{\boldsymbol{\theta}}(c \mid \boldsymbol{x}) = 1. \tag{61}$$

The vectorized implementation performs this aggregation with a cached class index and `scatter_add_`, preserving arbitrary leading batch axes and the autograd graph. The same index decodes sampled measured bit strings.

Three decoder families are available. The contiguous decoder divides the ordered basis states into C adjacent blocks whose sizes differ by at most one. For parity decoding, $r = \lceil \log_2 C \rceil$ nested parity bits are

$$b_j(z) = \bigoplus_{i=1}^{m-j+1} z_i, \qquad f_{\mathrm{par}}(z) = \left(\sum_{j=1}^{r} 2^{r-j} b_j(z) \right) \bmod C. \tag{62}$$

For disjoint majority blocks $B_1, \ldots, B_r$ in the measured string,

$$b_j(z) = \mathbf{1}\left\{ \sum_{i \in B_j} z_i > \left\lfloor \frac{|B_j|}{2} \right\rfloor \right\}, \qquad f_{\mathrm{maj}}(z) = \left(\sum_{j=1}^{r} 2^{r-j} b_j(z) \right) \bmod C. \tag{63}$$

Thus even-block ties map to zero. Automatic majority blocks are odd, nearly balanced, and maximize measured-wire coverage; strict mode requires equal, disjoint odd blocks of size at least three and no unused wire. Because modulo reduction or unequal block sizes may bias the uniform-state class prior, the resolved decoder exposes `reference_class_prior` and exact basis-state counts.

QASP feature: `ClassProbabilityDecoder`

A standalone, differentiable basis-state-to-class map shared by probability aggregation and sample decoding. It supports contiguous, recursive-parity, and majority–modulo partitions; exposes the cached `class_index`, `basis_state_counts`, and `reference_class_prior`; and validates class capacity, block disjointness, and measured-wire coverage.

6.3 Expectation-score classification

Expectation readout keeps observable scores distinct from Born probabilities. For binary classification with J observables, the signed decision score is

$$s_{\boldsymbol{\theta}}(\boldsymbol{x}) = \frac{1}{J}\sum_{j=1}^{J}\langle\psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})|O_j|\psi_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})\rangle, \qquad \widehat{y} = \mathbf{1}\{s_{\boldsymbol{\theta}}(\boldsymbol{x}) > 0\}. \tag{64}$$

The `expectation_values` method returns all J measurements before averaging. `predict_proba` applies the classical calibration $p(y = 1 \mid \boldsymbol{x}) = \sigma(s_{\boldsymbol{\theta}}(\boldsymbol{x})/\tau)$ and returns its complement for class zero. This sigmoid output is useful at the estimator boundary but must not be described as a direct Born probability.

For $C > 2$, QASP accepts one nonempty observable group $\mathcal{O}_c = \{O_{c,1}, \ldots, O_{c,J_c}\}$ per class and averages only within that class,

$$s_c(\boldsymbol{x}) = \frac{1}{J_c}\sum_{j=1}^{J_c}\langle O_{c,j}\rangle_{\boldsymbol{x}}, \qquad p(c \mid \boldsymbol{x}) = \frac{\exp(s_c(\boldsymbol{x})/\tau)}{\sum_{a=0}^{C-1}\exp(s_a(\boldsymbol{x})/\tau)}, \tag{65}$$

and predicts by `argmax`. A flat list of C observables is the singleton case $J_c = 1$ for every class; a nested list supplies the C groups explicitly. The `expectation_values` method returns the flattened, unaggregated measurements in group order. When observables are omitted, the model chooses the required number of nonidentity Z strings in increasing support order, one per class. An optional scalar or trainable per-output scale can rescale expectation scores; probability readout instead applies temperature or scale in log- probability space followed by renormalization.

QASP feature: `PQCClassifier` / `QuantumClassifier`

One differentiable classifier with explicit `readout="probabilities"` and `readout="expectation"` contracts. It supports binary and multiclass prediction, custom Pauli observables, temperature calibration, optional output scaling, batched circuit evaluation, named losses, exact device execution, and differentiable Pauli propagation through `device="lightning.pp"`. `QuantumClassifier` is an alias of `PQCClassifier`.

```python
import torch
from qasp import ClassifierTrainer, create_classifier, draw_classifier

base = {
    "feature_map": "AngleEmbedding",
    "feature_map_config": {
        "rotations": "Y", "input_scaling": False,
    },
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 2, "rotations": "ZY",
        "include_imprimitive": True,
        "imprimitive_pattern": "ring", "reuploading": True,
    },
    "init": "small_normal", "device": "default.qubit",
    "diff_method": "backprop",
}

born = create_classifier({
    **base, "readout": "probabilities", "decoder": "parity",
}, n_qubits=4, n_classes=3)
born_trainer = ClassifierTrainer(
    born, objective="categorical_cross_entropy", learning_rate=0.02,
)
born_history = born_trainer.fit(
    X_train, y_train, epochs=20, batch_size=16,
```

```
    validation_data=(X_test, y_test),
)
report = born_trainer.evaluate(X_test, y_test)

expectation = create_classifier({
    **base, "readout": "expectation",
    "observable": ["Z 0", "ZZ 0 1", "X 1"],
}, n_qubits=4, n_classes=2)
score = expectation(torch.as_tensor(X_test[0]))

multiclass_expectation = create_classifier({
    **base, "readout": "expectation",
    "observables": [
        ["Z 0", "X 0", "ZZ 0 1"],  # class 0
        ["Z 1", "X 1"],             # class 1
        ["Z 2", "X 2", "ZZ 2 3"],  # class 2
    ],
    "temperature": 0.8,
}, n_qubits=4, n_classes=3)
class_probabilities = multiclass_expectation.predict_proba(
    torch.as_tensor(X_test[0])
)

pp_classifier = create_classifier({
    **base, "readout": "expectation",
    "observables": ["Z 0", "Z 1", "ZZ 0 1"],
    "device": "lightning.pp", "diff_method": "backprop",
    "pp_truncation": 3, "pp_topM": 20_000,
    "pp_tau": 1e-10, "pp_prune_every": 5,
}, n_qubits=4, n_classes=3)
pp_trainer = ClassifierTrainer(
    pp_classifier, objective="categorical_cross_entropy",
    learning_rate=0.02,
)
pp_trainer.fit(X_train, y_train, epochs=20, batch_size=16)

paper_tikz = draw_classifier(
    expectation, inputs=X_test[0], output="tikz",
    tikz_options={"standalone": True,
                  "symbolic_parameters": True,
                  "compress": True},
)
```

Listing 15: Probability and expectation classifiers built from the same QASP circuit notation.

6.4 Objectives, training, and evaluation

The loss layer accepts class indices or, where applicable, one-hot targets. For binary expectation scores, QASP encodes $\widetilde{y} = 2y - 1 \in \{-1, +1\}$ and provides

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(s_i - \widetilde{y}_i)^2, \qquad \mathcal{L}_{\text{hinge}} = \frac{1}{N}\sum_{i=1}^{N}\max(0, 1 - \widetilde{y}_i s_i). \tag{66}$$

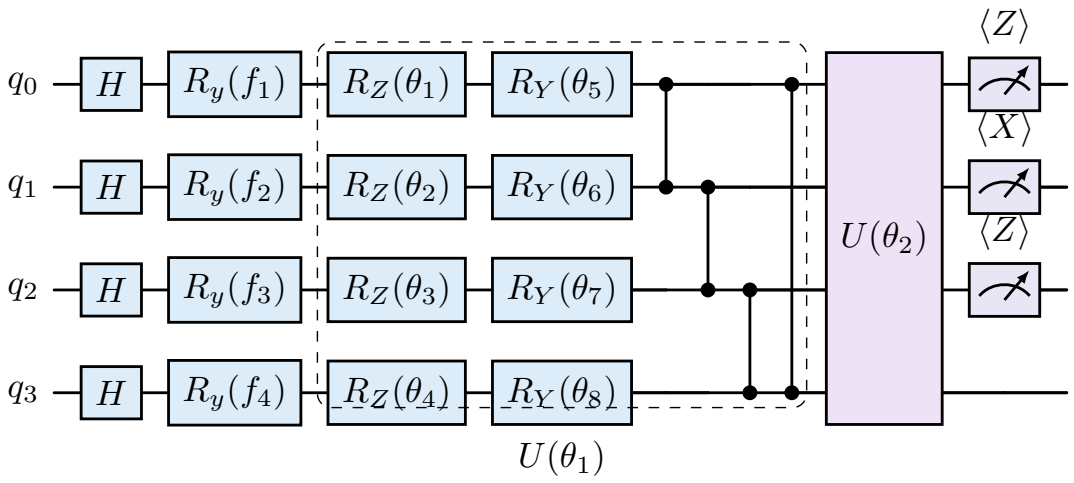
For normalized class probabilities it provides binary and categorical cross entropy,

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i}\left[y_i \log p_i + (1 - y_i)\log(1 - p_i)\right], \qquad \mathcal{L}_{\text{CCE}} = -\frac{1}{N}\sum_{i}\log p_{i,y_i}. \tag{67}$$

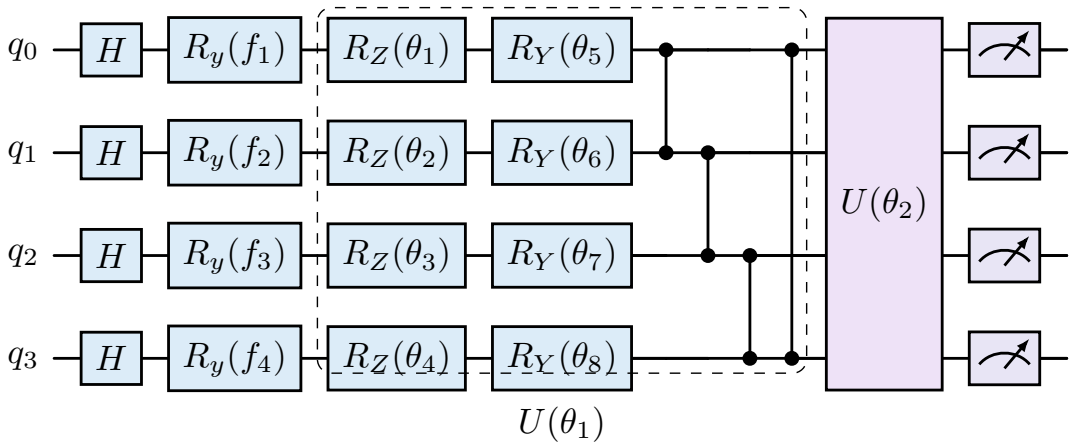
Multiclass MSE uses one-hot targets, and multiclass hinge uses the Crammer–Singer margin between the target score and the largest competing score. Every objective supports `none`, `mean`, or `sum` reduction.

QASP feature: `ClassifierLoss` and `ClassifierTrainer`

`ClassifierLoss` dispatches MSE, hinge, binary cross entropy, or categorical cross entropy without detaching the quantum computation graph. `ClassifierTrainer` supplies a compact minibatch PyTorch loop with Adam by default, user-provided optimizer support, validation histories, and held-out evaluation. The classifier chooses a compatible default objective, while an explicit objective remains part of the experiment record. Training targets are integer class labels; observable groups in expectation readout are arithmetic means, not the unit-weighted sums used by PQDQN per-action groups.



(a) Binary expectation classifier: local measurements $\langle Z_0 \rangle$, $\langle X_1 \rangle$, and $\langle Z_2 \rangle$ are averaged into the signed score in Equation (64).



(b) Three-class Born classifier: the 16 probabilities of the four-wire basis distribution are aggregated by the external recursive-parity decoder in Equation (62).

Figure 17: Classifier circuits traced from executable `PQCClassifier` objects through `draw_classifier(output="tikz")`. The companion `render_classifier_circuits.py` script retains standalone Quantikz, vector PDF, and PNG outputs. The decoder and sigmoid/softmax calibration are classical post-processing and are therefore stated in the captions rather than drawn as quantum gates.

QASP feature: Classification metrics

`classification_metrics` returns accuracy, balanced accuracy, macro/per-class precision, recall, F1, support, and a confusion matrix. When probabilities are supplied it adds binary or one-vs-rest multiclass ROC AUC and step-wise average-precision PR AUC. Standalone helpers also provide top-k accuracy, log loss, Brier score, and configurable macro, weighted, micro, or per-class reductions.

Confusion matrices place true labels in rows and predictions in columns. Macro averages retain configured classes that are absent from the sample with zero recall; multiclass Brier score sums squared class errors before averaging examples. ROC/PR AUC is undefined, and returned as NaN, when a binary sample does not contain both classes. These summaries are descriptive metrics, not independent statistical significance tests.

The executed notebook `examples/quantum_classifiers_sklearn_validation.ipynb` exercises the full public path on four familiar datasets. Its purpose is integration validation rather than a performance comparison: samples and epoch counts are deliberately small, and no accuracy threshold is asserted.

Table 10: Executable classifier integration coverage. Every case checks output shape, probability normalization, finite quantum gradients, training and evaluation, and exact agreement of QASP accuracy, macro-F1, and confusion matrices with scikit-learn.

Dataset	Classes	Readout	Decoder	Objective
Breast Cancer Wisconsin	2	expectation	–	mean squared error
Iris	3	probabilities	parity	categorical cross entropy
Wine	3	probabilities	contiguous	categorical cross entropy
Digits 0–3	4	probabilities	majority	categorical cross entropy

The notebook also checks decoder conservation, all four objective gradients, class-prior diagnostics, standalone symbolic Quantikz generation, and ROC/PR AUC against scikit-learn on deliberately imbalanced binary data with tied scores. The dedicated classifier test module contains 18 passing tests over these contracts. These checks establish internal consistency and interoperability, not quantum advantage or dataset-level superiority.

7 Generative learning

The generative module reuses QASP's circuit dictionaries, device selection, autodifferentiation, Pauli propagation, and drawing stack to learn distributions of binary strings. Its primary public entries are `PQCBM`, `create_qcbm`, `draw_qcbm`, and `PauliMMD`.

7.1 Quantum-circuit Born-machine abstraction

A quantum-circuit Born machine (QCBM) prepares $|\psi_{\boldsymbol{\theta}}\rangle = U(\boldsymbol{\theta})|0^{\otimes n}\rangle$ and represents

$$q_{\boldsymbol{\theta}}(x) = |\langle x|\psi_{\boldsymbol{\theta}}\rangle|^2, \qquad x \in \{0,1\}^n. \tag{68}$$

`PQCBM` accepts an ansatz/feature-map configuration, an existing `CircuitBuilder`, or a probability-valued PennyLane QNode. Angle and IQP ansatzes use the same component dictionaries as the RL and classifier factories. A custom QNode retains its device, shots, differentiation method, and measured-wire order; QASP does not silently reconfigure it.

The optional feature map has a deliberately narrow conditioning contract. It encodes one fixed vector shared by all target rows, rather than treating each generated bit string as an input. `AngleEmbedding`, `NeuralEmbedding`, custom feature-map classes, re-uploading, and trainable input scales are supported. Conditional data with a different input for every row requires an external training loop.

QASP feature: `PQCBM`, `create_qcbm`, and `draw_qcbm`

`PQCBM` is a configuration-driven Born-machine model and trainer with Torch or PennyLane Autograd parameters, NLL or Gaussian MMD^2 objectives, optional fixed-vector conditioning, minibatching, direct device sampling, entropy diagnostics, and small-system natural gradients. The factory copies caller configurations and builders. Drawing returns text, Matplotlib, or editable Quantikz from the constructed circuit. `BornMachine` remains a deprecated Autograd/integer-sample compatibility subclass.

Listing 16 constructs the executed example through the high-level factory. Binary training data have shape (N, n), and wire zero is the most significant output bit.

```python
import numpy as np
from qasp import create_qcbm, draw_qcbm

data = np.array([[0, 0, 0], [1, 1, 1]] * 128)
config = {
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 2, "rotations": "YZ",
        "include_imprimitive": True,
        "imprimitive": "CNOT",
        "imprimitive_pattern": "chain",
    },
    "loss": "mmd",
    "mmd_config": {
        "bandwidths": [0.5, 1.0, 2.0],
        "mode": "exact",
    },
    "device": "default.qubit",
    "seed": 7, "lr": 0.05,
}
model = create_qcbm(config, n_qubits=3, data=data)
params, history = model.train(steps=120)
bits = model.sample(2000)          # shape (2000, 3)
probs = model.get_probs()          # guarded small-system diagnostic
tex = draw_qcbm(
    model, output="tikz",
    tikz_options={"standalone": True, "compress": True},
)
```

Listing 16: Constructing, training, sampling, and drawing a three-qubit QASP Born machine.

7.2 Probability and expectation objectives

For probability-capable execution, QASP minimizes minibatch negative log likelihood

$$\mathcal{L}_{\mathrm{NLL}} = -\frac{1}{|\mathcal{B}|}\sum_{x\in\mathcal{B}} \log(q_{\boldsymbol{\theta}}(x) + \varepsilon)\,. \tag{69}$$

The accompanying KL helper expects a complete target distribution rather than binary observations. NLL, complete-probability diagnostics, and model entropy retain a 2^n output cost and are guarded by `max_probability_qubits`.

The scalable-output alternative represents Gaussian-kernel MMD^2 as a mixture of Pauli expectations. For binary strings, squared Euclidean distance is Hamming distance. QASP takes each bandwidth σ to be a standard deviation,

$$k_\sigma(x, y) = \exp\left[-\frac{\|x - y\|_2^2}{2\sigma^2}\right]. \tag{70}$$

For a mask $a \in \{0,1\}^n$, define $Z_a = \prod_{j:a_j=1} Z_j$ and $\mu_p(a) = \mathbb{E}_{x\sim p}[(-1)^{a\cdot x}]$. Drawing every mask bit independently with probability

$$r_\sigma = \frac{1 - \exp[-1/(2\sigma^2)]}{2} \tag{71}$$

gives

$$\mathrm{MMD}^2(p, q_{\boldsymbol{\theta}}) = \mathbb{E}_{a\sim \mathrm{Bernoulli}(r_\sigma)^{\otimes n}}\left[(\mu_p(a) - \langle\psi_{\boldsymbol{\theta}}|Z_a|\psi_{\boldsymbol{\theta}}\rangle)^2\right]. \tag{72}$$

This is the expectation-mixture identity of Recio-Armengol, Ahmed, and Bowles [28]. Their supplied v1 uses σ^2 in its Gaussian-kernel definition but prints σ in a later mask formula. QASP consistently follows the

standard-deviation convention in Equations (70) and (71); regression tests compare the full loss and gradient with the dense Gaussian kernel at nonunit bandwidths.

QASP feature: `PauliMMD`

`PauliMMD` supports equal-weight bandwidth mixtures, Monte Carlo masks or exact small-system enumeration, empirical target moments or a data U-statistic correction, and isolated mask seeds. The identity mask retains its original probability mass. Sampled mode avoids enumerating the Born vector; exact mode is guarded because it enumerates all 2^n masks.

For `estimator=ünbiased¨` and $N \geq 2$, the empirical data-square term is replaced by

$$\widehat{\mu_p(a)^2}_U = \frac{N\widehat{\mu}_p(a)^2 - 1}{N-1}, \tag{73}$$

which may be negative and is not clipped. With finite circuit shots, QASP evaluates the model moment twice independently and uses $(\widehat{\mu}_{q,1} - \widehat{\mu}_p)(\widehat{\mu}_{q,2} - \widehat{\mu}_p)$, avoiding the upward bias from squaring one shot estimate. Data minibatching, mask sampling, measurement shots, and Pauli-propagation truncation remain distinct approximation sources.

7.3 Training, execution, and sampling contracts

`interface=ẗorch¨` exposes live float64 tensors and Adam by default; `interface=äutograd¨` uses PennyLane NumPy and PennyLane optimizers. Initialization can be uniform on $[0, 2\pi)$, zero, or normally distributed, and explicit parameter tensors are accepted. `train` and its `fit` alias return the final ansatz parameters and pre-update loss history. Torch users can replace `model.opt`, supply an optimizer factory, or call differentiable `loss()` in an external loop. A custom `loss_fn(probabilities,batch_indices)` is a probability-readout path and is therefore unavailable to the Pauli-propagation backend. The tested small-system natural-gradient option follows Equation (10).

Table 11: Principal PQCBM execution routes. Availability of a particular plugin, gate, measurement, or derivative combination remains the installed PennyLane device's responsibility.

Backend route	Objective/readout	Contract
`default.qubit`	NLL probabilities or MMD moments	Analytic backprop, or finite-shot parameter shift where supported.
Lightning statevector/Kokkos/GPU	Probabilities with a supported method; analytic MMD moments with adjoint	MPI and plugin options pass through explicitly; no serial fallback.
`default.mixed` and other plugins	Device-supported probabilities or moments	Noise, shots, gates, and differentiation follow the plugin contract.
`lightning.pp`	Analytic Torch MMD moments	Differentiable Pauli propagation; no probabilities, finite shots, or MPI.

Pauli-propagation controls expose maximum retained Pauli weight, top-M pruning, a coefficient threshold, and pruning cadence. Unpruned, untruncated propagation provides a feasible exact reference for small circuits, but exact operator growth can still be exponential. Truncated or pruned runs propagate gradients only through retained paths and have no blanket efficiency or unbiasedness guarantee.

`sample()` executes a sampling measurement directly; it does not first construct a probability vector. The default result is a (shots, n) bit matrix, with optional big-endian indices up to 62 bits. Because the PP engine produces expectations rather than samples, a PP-trained model must name a separate sampling device:

```python
pp = create_qcbm(
    {**config, "device": "lightning.pp",
     "pp_prune": False},
    n_qubits=3, data=data,
)
pp.train(120)
bits = pp.sample(
    2000, device="default.qubit",
    device_kwargs={"seed": 19},
)
```

Listing 17: Training with Pauli propagation and deploying the same learned circuit to a sampling backend.

7.4 Executed three-qubit example

The executed notebook trains matched state-vector and unpruned PP models on 256 rows split equally between 000 and 111. Both models use two R_Y/R_Z layers with CNOT chains, seed 7, Adam learning rate 0.05, 120 updates, and exact MMD with bandwidths 0.5, 1, and 2. Full probabilities are used only for this small-system validation; 2,000 final samples are drawn directly from `default.qubit` with seed 19.

Table 12: Measured outcomes from the executed example in Figure 18. PP uses neither pruning nor weight truncation.

Quantity	Value
Initial MMD^2	0.3320805127
Final MMD^2	1.6678373×10^{-5}
Probability on $\{000, 111\}$	0.9925771886
Total variation from target	0.0074228114
Initial gradient max difference, PP vs. statevector	1.39×10^{-16}
Final probability max difference, PP vs. statevector	2.22×10^{-16}

These values establish agreement for one tiny matched circuit, not a runtime or scaling advantage. QASP implements the reference paper's expectation-mixture identity with its own execution engines; it does not implement that work's specialized IQP Monte Carlo estimator, data-dependent initialization, or thousand-qubit experiments.

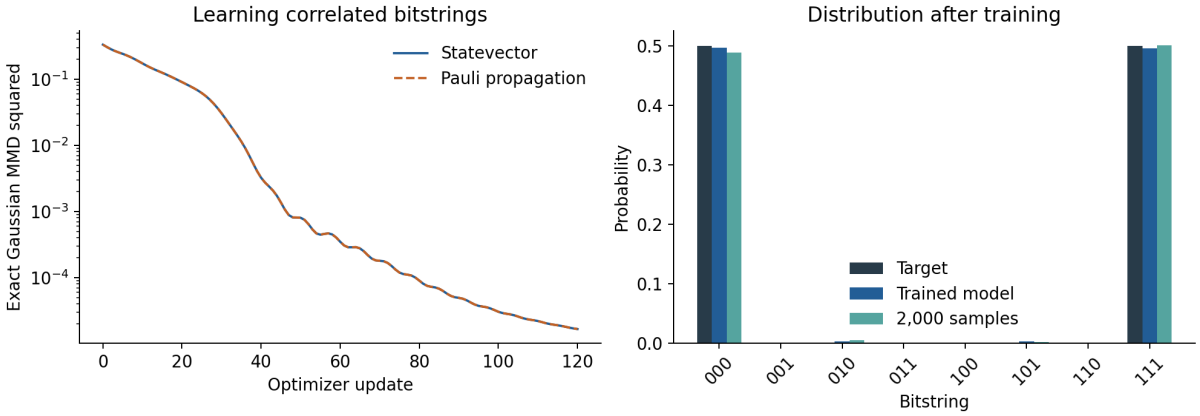
7.5 MPI and high-performance use

For MPI-enabled statevectors, every rank must use the same parameters, data, masks, optimizer state, seed, and circuit-call order. MPI distributes one simulator state rather than partitioning the data batch. The HPC inspector recognizes an MMD workload as an expectation output with a declared observable batch instead of charging it for a 2^n probability vector. Statevector memory remains exponential, and exact-mask MMD retains exponential classical storage. The Deucalion helper supplies a rank-consistent callback and collective smoke script, while the pure-statevector planner rejects PP, noisy, and tensor methods because their memory models differ.

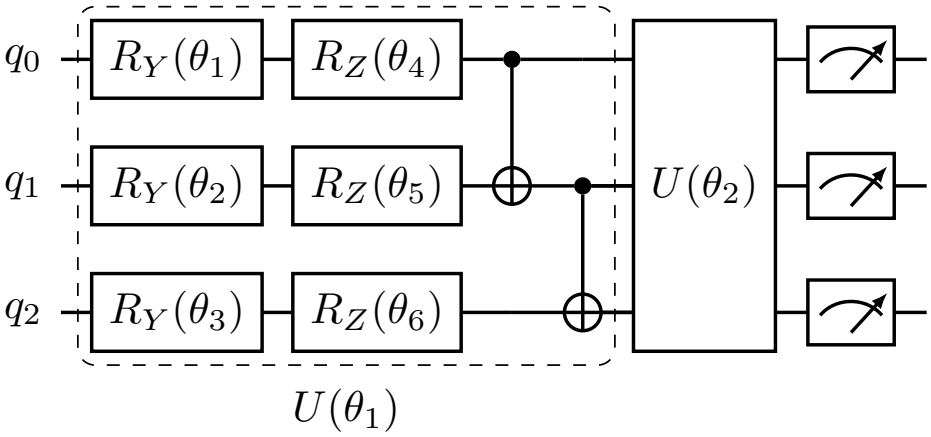
The local validation compares values, gradients, and updates across the default state-vector simulator, Lightning state-vector and Kokkos devices, the default mixed-state simulator, and unpruned QASP Pauli propagation. It covers Angle and IQP ansatzes; Torch and Autograd; finite-shot parameter shift; fixed conditioning and input scales; Fisher updates; wire ordering; drawings; a 70-wire expectation-only PP smoke case; and local HPC routing. It does not constitute an actual multi-node MPI, GPU, tensor-network, or quantum-hardware execution.

8 Combinatorial optimization

The optimization module adds an end-to-end path from a classical binary objective to a sampled assignment. It separates the problem representation, quantum algorithm, classical outer-loop optimizer, and final sampling budget, while preserving a common result and circuit-drawing contract. The supported families are standard and multi-angle QAOA, relaxation-based warm-start QAOA, recursive QAOA, fixed linear-ramp QAOA, and Pauli correlation encoding (PCE). The implementations follow their respective algorithmic constructions



(a) State-vector and exact unpruned-PP learning curves, followed by the target, trained probabilities, and empirical frequencies from 2,000 direct device samples.



(b) Paper-ready Quantikz exported from the trained `PQCBM`. MMD is classical post-processing of measured Pauli moments and is not drawn as a quantum gate.

Figure 18: Reproducible PQCBM example generated by QASP. The experiment is a three-qubit, one-seed integration validation, not a scaling benchmark.

[29–34]; QASP's contribution is to expose them through the package's existing circuit notation, device selection, testing, results, and Quantikz export.

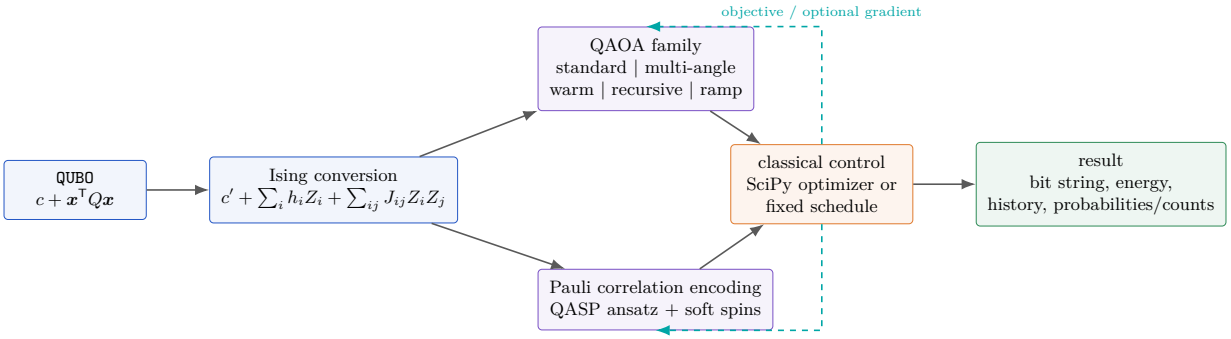


Figure 19: Optimization dataflow in QASP. A single QUBO representation supplies an exact Ising Hamiltonian to multiple solver families. Variational solvers return objective values to a reusable classical optimizer, while LR-QAOA uses a deterministic schedule. Every route decodes to the original variable ordering and records algorithm-specific metadata.

8.1 QUBO and Ising problem contract

For n binary variables, QASP defines the quadratic unconstrained binary optimization problem

$$E_Q(\boldsymbol{x}) = c + \boldsymbol{x}^\mathsf{T} Q \boldsymbol{x}, \qquad \boldsymbol{x} \in \{0,1\}^n. \tag{74}$$

Non-symmetric matrices are accepted: the two entries Q_{ij} and Q_{ji} are combined into one coefficient $b_{ij} = Q_{ij} + Q_{ji}$ for $i < j$. Writing $a_i = Q_{ii}$ and substituting $x_i = (1 - Z_i)/2$ produces

$$H_C = c' I + \sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j, \tag{75}$$

where

$$c' = c + \frac{1}{2}\sum_i a_i + \frac{1}{4}\sum_{i<j} b_{ij}, \quad h_i = -\frac{1}{2}a_i - \frac{1}{4}\sum_{j\neq i} b_{\min(i,j),\max(i,j)}, \quad J_{ij} = \frac{1}{4} b_{ij}. \tag{76}$$

Consequently $E_Q(\boldsymbol{x}) = \langle \boldsymbol{x} | H_C | \boldsymbol{x} \rangle$ under the computational-basis convention in Equation (75).

QASP feature: `QUBO` and `PauliTerm`

`QUBO` constructs objectives from a matrix, explicit linear/quadratic terms, an Ising model, or a weighted MaxCut edge list. It exposes exact assignment energies, the offset and Ising coefficients, Pauli terms, a PennyLane Hamiltonian including the constant, and a brute-force reference for small instances. `PauliTerm` is the shared weighted-Pauli representation used by cost and mixer circuits.

For weighted MaxCut, `QUBO.maxcut` returns the minimization objective

$$E_{\text{cut}}(\boldsymbol{x}) = -\sum_{(i,j)\in\mathcal{E}} w_{ij}\big(x_i + x_j - 2x_i x_j\big), \tag{77}$$

so minimizing the returned energy maximizes the cut weight. Keeping this sign convention in the problem object prevents a solver-specific reinterpretation of the reported energy.

8.2 The QAOA family

With cost Hamiltonian H_C and mixer $H_M = \sum_b m_b M_b$, depth-p standard QAOA prepares

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = \prod_{\ell=1}^{p} e^{-i\beta_\ell H_M} e^{-i\gamma_\ell H_C} |s\rangle, \qquad F(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \boldsymbol{\gamma}, \boldsymbol{\beta} | H_C | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle. \tag{78}$$

The constant term remains in H_C for objective evaluation but does not generate a circuit gate. By default, $|s\rangle = |+\rangle^{\otimes n}$ and $H_M = \sum_i X_i$. A QASP mixer dictionary can instead provide arbitrary weighted Pauli words and a plus, zero, callable, or amplitude-vector initial state.

Standard QAOA shares one cost angle and one mixer angle per layer. In the multi-angle variant, every nonconstant cost term $c_a P_a$ and mixer term $m_b M_b$ receives its own parameter,

$$U_C^{(\ell)} = \prod_a e^{-i\gamma_{\ell a} c_a P_a}, \qquad U_M^{(\ell)} = \prod_b e^{-i\beta_{\ell b} m_b M_b}, \tag{79}$$

making standard QAOA the tied-parameter special case [32].

QASP feature: `QAOA` and `WarmStartQAOA`

`QAOA` supports standard or multi-angle parameters, QASP-style custom mixers, random/zero/ramp initialization, Hamiltonian expectations, exact or sampled probabilities, gradients, solution counts, and independent training and reporting shot budgets. `WarmStartQAOA` accepts a relaxed vector or computes a bounded continuous QUBO relaxation before constructing the warm-start initial state and mixer.

For a relaxation $q_i \in [0,1]$, QASP clips the values to $[\epsilon, 1-\epsilon]$ and sets

$$\vartheta_i = 2\arcsin\sqrt{q_i}, \qquad |s_q\rangle = \bigotimes_i R_Y(\vartheta_i)|0\rangle, \qquad H_M^{(q)} = -\sum_i \left(\sin\vartheta_i X_i + \cos\vartheta_i Z_i\right). \tag{80}$$

This preserves the standard and multi-angle interfaces while changing both the prepared state and, when no custom mixer is supplied, the mixer evolution [30].

QASP feature: `LRQAOA` / `LinearRampQAOA`

Executes a deterministic opposing linear schedule without a classical parameter optimizer. With zero-based layer $\ell = 0, \ldots, p-1$, QASP uses $\gamma_\ell = (\ell+1)\Delta_\gamma/p$ and $\beta_\ell = (1-\ell/p)\Delta_\beta$, with configurable increments and the defaults $\Delta_\gamma = 0.6$, $\Delta_\beta = 0.3$.

QASP feature: `RecursiveQAOA` / `RQAOA`

At each reduction step, QASP optimizes a QAOA instance, evaluates the active $\langle Z_i\rangle$ and $\langle Z_i Z_j\rangle$ correlations from its basis probabilities, and rounds the largest-magnitude active Hamiltonian correlation. A one-body rule fixes $Z_i = \pm 1$; a two-body rule substitutes $Z_j = \pm Z_i$. Once the configured cutoff is reached, the reduced Ising problem is solved exactly and all substitutions are reconstructed in reverse. The result retains every elimination rule and intermediate QAOA result.

```python
from qasp import (
    LRQAOA, QAOA, QUBO, RecursiveQAOA, WarmStartQAOA,
    draw_optimization,
)

problem = QUBO.maxcut(3, [(0, 1), (1, 2), (0, 2)])

# One parameter per cost and mixer gate; use analytic adjoint gradients.
multi = QAOA(
    problem, n_layers=2, parametrization="multi_angle", seed=7,
)
multi_result = multi.solve(
    optimizer="BFGS", use_gradients=True,
    optimizer_options={"maxiter": 30}, result_shots=2_000,
)

# A supplied vector or True enables relaxation-based warm starting.
warm_result = WarmStartQAOA(
    problem, n_layers=2, warm_start=True, seed=7,
).solve(optimizer_options={"maxiter": 30})

# Recursive reduction and a deterministic schedule use the same QUBO.
recursive_result = RecursiveQAOA(
    problem, cutoff=2, optimizer_options={"maxiter": 20},
).solve()
ramp_result = LRQAOA(problem, n_layers=4).solve()

paper_tikz = draw_optimization(
    multi, parameters=multi_result, output="tikz",
    tikz_options={"standalone": True,
                  "symbolic_parameters": True},
```

```
)
```

Listing 18: Solving one MaxCut QUBO through the QASP QAOA family.

8.3 Pauli correlation encoding

PCE assigns each of m optimization variables to an equal-Pauli k-body correlation observable P_i on an n-qubit variational state [33]. For a chosen Pauli alphabet $\mathcal{P}$, the available capacity is

$$m_{\max} = |\mathcal{P}|\binom{n}{k}; \qquad \mathcal{P} = \{X, Y, Z\},\ k = 2 \quad \Longrightarrow \quad m_{\max} = 3\binom{n}{2}. \tag{81}$$

QASP selects the smallest n meeting this capacity unless the register size is supplied explicitly. The measured correlations become smooth spins

$$s_i(\boldsymbol{\theta}) = \tanh(\alpha \langle \psi_{\boldsymbol{\theta}} | P_i | \psi_{\boldsymbol{\theta}} \rangle)\,, \tag{82}$$

and the implemented relaxed Ising objective is

$$\mathcal{L}_{\mathrm{PCE}}(\boldsymbol{\theta}) = c' + \sum_i h_i s_i + \sum_{i<j} J_{ij} s_i s_j + \lambda \nu \left(\frac{1}{m} \sum_i s_i^2 \right)^2, \qquad \nu = \frac{|\mathcal{E}_Q|}{2} + \frac{m-1}{4}, \tag{83}$$

where $|\mathcal{E}_Q|$ is the number of nonzero quadratic Ising couplings. Decoding uses $x_i = 0$ for $s_i \geq 0$ and $x_i = 1$ otherwise, consistently with $x_i = (1 - Z_i)/2$.

QASP feature: `PauliCorrelationEncoding` / `PCE`

Builds the correlation assignment, accepts `AngleAnsatz`, `IQPAnsatz`, an ansatz class/object, or a preconfigured `CircuitBuilder`, measures all encoded correlations, optimizes the soft-spin loss, and decodes to the original QUBO variable count. The `build_pauli_correlation_encoding` helper exposes the register size and ordered observable assignment before a solver is constructed.

```
from qasp import PCE, draw_pce

pce = PCE(problem, circuit={
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 1,
        "rotations": ["Y 0", "Y 1", "ZZ 0 1"],
        "include_hadamards_initial": True,
    },
    "device": "default.qubit",
}, seed=7)

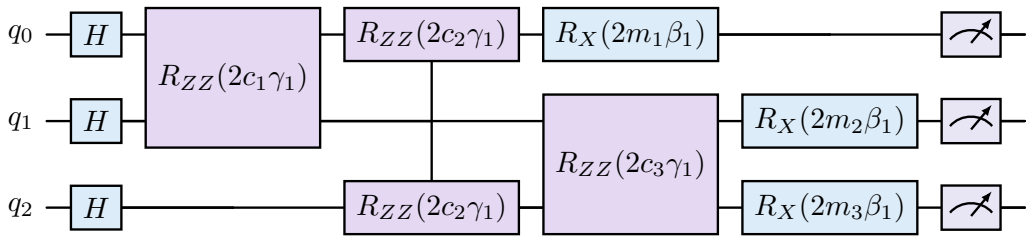
pce_result = pce.solve(optimizer_options={"maxiter": 30})
assert len(pce_result.bitstring) == problem.n_variables
pce_tikz = draw_pce(
    pce, parameters=pce_result, output="tikz",
    tikz_options={"standalone": True,
                  "symbolic_parameters": True},
)
```

Listing 19: Qubit-compressed PCE with a dictionary-defined QASP ansatz.

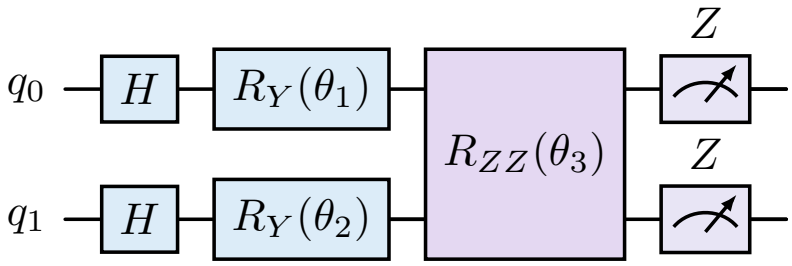
8.4 Classical control, results, and executable validation

QASP feature: Optimizers, results, and drawing

`create_optimizer` resolves COBYLA, Nelder–Mead, Powell, BFGS, L-BFGS-B, SLSQP, or a user callable behind one SciPy-compatible interface. `OptimizationResult` records parameters, decoded bit string, exact QUBO energy, optimized quantum objective, optional probabilities or counts, shots, history, status, metadata, and the raw optimizer result. `RecursiveQAOAResult` adds elimination rules and intermediate results. `draw_optimization`, `draw_qaoa`, `draw_pce`, and `draw_recursive_qaoa` emit text, Matplotlib, or symbolic Quantikz.



(a) One-layer standard QAOA for triangle MaxCut. QASP derives the three ZZ cost rotations from the QUBO, inserts the default X mixer, and labels the shared parameters γ_1 and β_1.



(b) Two-qubit PCE ansatz for the three-variable problem. The terminal Z-basis trace depicts the ZZ correlation setting; the same PCE object evaluates the corresponding XX and YY settings to encode all three variables.

Figure 20: Optimization circuits generated from executable QASP objects by `draw_optimization`. The companion renderer saves editable standalone Quantikz, vector PDF, and PNG outputs; no circuit is redrawn manually.

The two scalar fields have different semantics: `objective` is the optimized expectation or PCE surrogate, whereas `energy` evaluates the decoded binary candidate in the original QUBO. A successful classical optimizer status certifies neither that candidate's global optimality nor a better approximation ratio than a classical baseline. Probabilities may be absent for counts-only or decoded results. Dense basis probabilities grow as 2^n; requesting counts avoids that output allocation but does not remove the underlying simulator-state cost.

Analytic QAOA uses adjoint differentiation by default; finite-shot training uses parameter shift when a gradient is requested, or a derivative-free optimizer can minimize the noisy expectation directly. The independent `result_shots` argument controls only the post-training distribution and counts. This distinction is recorded in result metadata and prevents an analytic training run from being mislabeled as a shot-based solution estimate. PCE currently uses a SciPy outer loop over NumPy-valued expectation calls and should not be described as an end-to-end autodifferentiable solver.

The executable `tests/optimization_algorithms.ipynb` covers every solver family, custom mixers, both warm-start parameterizations, analytic and finite-shot gradients, sampled solutions, and standalone symbolic drawings. The focused unit module contains 14 passing tests summarized in Table 13.

Table 13: Focused validation of the optimization module. These are software contract checks on small problems, not approximation-ratio or scaling claims.

Area	Configurations	Checked invariant
QUBO representation	Matrix, terms, Ising, triangle Max-Cut	Energy equality under QUBO–Ising conversion and exact small-instance optimum
QAOA circuits	Standard, custom mixer, multi-angle	Parameter shape, normalized probabilities, one parameter per declared gate
Differentiation	Analytic adjoint, BFGS, finite-shot parameter shift	Finite gradients and agreement with central finite differences
QAOA variants	Standard/multi-angle warm starts, LR-QAOA, RQAOA	Clipping and metadata, exact ramp schedule, reduction and reconstruction
Sampling/results	Training shots and independent result shots	Normalized estimates, count totals, bit-string probability, recorded contracts
PCE and drawings	Dictionary ansatz, 100-variable capacity, all drawing outputs	Encoding size, decoded shape, finite objective, symbolic standalone Quantikz

9 Trainability and structural diagnostics

Qasp includes diagnostics intended to run directly on the same constructed models and configurations used for training.

QASP feature: `barren_plateau_diagnoser`

The lazy top-level diagnostic accepts an already constructed classifier, trainable kernel, or RL policy and reinitializes its quantum parameters from a named QASP distribution. For scalar costs C_s measured at N independent initializations, it returns the running curve and final estimator

$$\widehat{\mathrm{Var}}[C] = \frac{1}{N-1}\sum_{s=1}^{N}(C_s - \bar{C})^2. \tag{84}$$

The same initialization draws also produce a componentwise variance for the raw circuit expectations before decoder, softmax, or loss post-processing. Observable circuits expose their measured expectations directly; for Born readouts, local Z expectations are reconstructed from the measured basis distribution. QASP reports both each component variance and the scalar variance of their arithmetic mean; the latter retains cross-observable covariance rather than averaging marginal variances. Thus the diagnostic separates concentration introduced by the quantum circuit from concentration introduced by the task cost. A fixed embedded state estimates variance over parameters conditional on that input; omitting the state samples a fresh input on every initialization and therefore estimates the joint state–parameter variance. Default scalar outputs are one selected class probability, action probability, or pairwise kernel similarity. Classifier objectives and arbitrary user-defined scalar cost functions are also accepted. Original parameters and module mode are restored after the experiment. A PQDQN is handled as a generic model and therefore requires an explicit scalar `cost_function(model, state)`, for example one selected Q-value or a true TD loss. By default only quantum-layer parameters are reinitialized; `parameter_scope="all"` also includes trainable classical parameters. For a system-size study, the user supplies a qubit list and a model factory. The diagnostic constructs the same model family at every requested width, performs the full initialization ensemble independently at each width, and returns cost and scalar-expectation variance arrays aligned with the qubit list. A state factory can provide one controlled fixed input of the proper dimension at each width. Componentwise expectation variances remain keyed by width because their dimension may grow with the number of measured qubits.

The following example diagnoses the expectation-readout classifier constructed in Listing 15. A fixed input isolates parameter-initialization variance, while the two reported fields separate variation of the selected class probability from variation of the circuit expectations that feed the classical readout.

```
from qasp import barren_plateau_diagnoser

diagnosis = barren_plateau_diagnoser(
    expectation,
    n_samples=2000,
    initialization="random_-pi_pi",
    state=torch.as_tensor(X_test[0]),
    output_index=1,
    parameter_scope="quantum",
    seed=17,
    return_samples=True,
)
cost_variance = diagnosis["cost_variance"]
raw_expectation_variance = diagnosis["expectation_variance"]
running_curve = diagnosis["cost_variance_by_sample"]
```

Listing 20: Cost-level and pre-decoder concentration from the model-agnostic barren-plateau diagnoser.

The width-sweep mode makes the system-size dependence explicit. The notebook `examples/barren_plateau_diagnoser_validation.ipynb` constructs the same one-layer classifier at every width, fixes the embedded state to zero, and uses the global observable $Z^{\otimes n}$. Thus the only changing experimental variable is n. The complete executable call is reproduced in Listing 21.

```
def classifier_factory(n):
    global_z = f'{"Z" * n} ' + " ".join(map(str, range(n)))
    return create_classifier({
        "ansatz": "AngleAnsatz",
        "ansatz_config": {
            "n_layers": 1, "rotations": "Y",
            "include_imprimitive": True, "imprimitive": "CZ",
            "imprimitive_pattern": "chain", "reuploading": False,
        },
        "feature_map": "AngleEmbedding",
        "feature_map_config": {
```

```
            "features": n, "rotations": "Y",
            "imprimitive_pattern": "chain",
        },
        "readout": "expectation", "observables": [global_z],
        "init": "zeros", "device": "default.qubit",
        "diff_method": "backprop",
    }, n_qubits=n, n_classes=2)

qubit_list = list(range(2, 11))
sweep = barren_plateau_diagnoser(
    model_factory=classifier_factory,
    qubit_list=qubit_list,
    n_samples=128,
    initialization="random_-pi_pi",
    state_factory=lambda n: torch.zeros(n),
    output_index=1,
    seed=19,
)

plt.semilogy(qubit_list, sweep["cost_variance"], "o-")
plt.semilogy(
    qubit_list, sweep["expectation_value_variance"], "s--"
)
```

Listing 21: Executable cost-concentration sweep as a function of qubit count. The notebook uses 128 independent parameter initializations at each width.

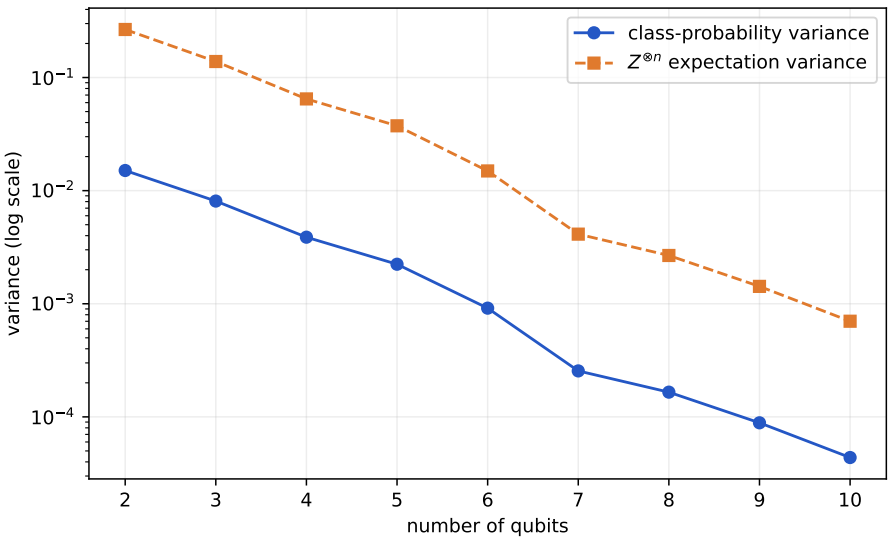


Figure 21: Finite-size concentration experiment generated by the executable barren-plateau notebook. Each point uses 128 independent Unif$[-\pi, \pi]$ parameter initializations, a fixed zero input, a one-layer chain ansatz, and global $Z^{\otimes n}$ readout. The raw expectation variance has fitted slope -1.11 per qubit on a base-two logarithmic scale over $n = 2, \ldots, 10$. This cost-concentration example validates the width-sweep workflow; it is not, by itself, a claim about gradient-variance scaling.

Cost concentration is a screening diagnostic rather than, by itself, a proof of a barren plateau. The conventional claim concerns exponential suppression of gradient variance with system size [1]; a controlled width sweep with a fixed cost, state protocol, and initialization distribution is required for that conclusion. The API records those choices explicitly so cost-concentration experiments are not accidentally mixed across conditions. The reported expectation statistic is variance across initialization draws, not the single-state quantum measurement variance $\langle O^2 \rangle - \langle O \rangle^2$.

QASP feature: `policy_variance`

Repeatedly reinitializes a policy and estimates the variance of a selected action probability across circuit sizes. The controlled call, estimator, and width sweep in Section 4.1.7, Equation (35), and Listing 7 make action and state conditioning explicit. Samples can be distributed with Ray, and qubit-dependent configuration values can be supplied as callables.

For a compact single-policy sweep, the same public routine can be called as follows; `raw_samples[n]` retains the probabilities used to form each width's variance estimate. The more tightly controlled parity-versus-majority experiment is given in Listing 7.

```python
from qasp import policy_variance

variance, raw_samples = policy_variance(
    policy_class="PQCBorn",
    policy_config_kwargs=variance_policy_config,
    action=0,
    qubit_list=[3, 5, 7, 9],
    n_samples=500,
    batch_size=500,
    parallel=False,             # set True to distribute with Ray
    state=torch.zeros(4),       # omit for joint state/parameter variance
)
```

Listing 22: A width sweep of policy-output variance at fixed state and action.

QASP feature: Legacy exploratory policy metrics

`qasp.rl.metrics` retains policy-output sampling and heuristic structural helpers used by earlier experiments. The function named `fourier_analysis` currently performs no Fourier transform and also has legacy import limitations. `compute_DLA_generators` returns closure sizes rather than generators, while `_policy_expressivity` and the Hadamard/CZ/CNOT "decomposition" helpers assemble heuristic generator lists. They are not certified circuit dynamical Lie algebras or executable unitary gate decompositions.

Listing 23 preserves the callable interface for reproducibility while labelling the result correctly as an exploratory closure size. Such a quantity can motivate a follow-up symbolic analysis, but should not be used as a proof of controllability or as a drop-in measure of policy expressivity.

```python
from qasp import compute_DLA_generators, _policy_expressivity

dla_policy_config = {
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 1, "rotations": "Y",
        "include_imprimitive": True, "imprimitive": "CZ",
        "imprimitive_pattern": "chain", "reuploading": False,
    },
    "feature_map": "AngleEmbedding",
    "feature_map_config": {
        "features": lambda n: n, "rotations": "Y",
    },
    "n_actions": 2, "policy": "contiguous-like",
    "training_method": "probs", "init": "normal_0_1",
    "device": "default.qubit", "diff_method": "backprop",
    "torchrl": False,
}

dla_dimensions = compute_DLA_generators(
    "PQCBorn", dla_policy_config, qubit_list=[2, 3, 4],
    parallel=False,
)
closure_3q = _policy_expressivity(
    "PQCBorn", dla_policy_config, n_qubits=3,
)
assert dla_dimensions[1] == len(closure_3q)
```

Listing 23: Exploratory Lie-closure sizes and elements from the legacy policy-metrics interface.

The validated diagnostics in this release are the finite-sample cost/output concentration and policy-output variance estimators above. The legacy structural routines remain useful for hypothesis generation, but

their names must not be promoted into claims of Fourier spectra, certified Lie-algebra dimensions, or task performance.

10 Validation and reproducibility

Release 0.1.0 contains focused unit tests, executable notebooks, cluster-oriented experiment drivers, and source-level API documentation for all 52 Python modules and all 662 discovered definitions. The documentation pass added 385 missing definition docstrings and verified that removing docstrings from before/after syntax trees leaves identical executable ASTs; it did not modify runtime statements. The policy/environment regression suite checks not merely normalization, but numerical equality of both forward action probabilities and backward parameter gradients across exact representations. The comparisons use absolute and relative tolerance 10^{-6}. In the documented audit environment, the existing suite produced 252 passes and one environment-only failure out of 253 tests: the `lightning.kokkos` plugin was not installed for one sparse-projector parametrization, while its `lightning.qubit` counterpart passed. That full-suite run used the temporary Python 3.13 environment `/private/tmp/qasp-slurm-venv`. After the intended Conda environment `torchlane` was identified, device creation succeeded and only the previously failing Kokkos test was rerun: it passed in 29.81 seconds without code changes under Python 3.10.17, PennyLane 0.42.2, Lightning/Lightning-Kokkos 0.42.0, Torch 2.7.0, and pytest 9.1.1. This targeted result resolves the implementation concern but is not a claim that all 253 tests were rerun together in `torchlane`. Six small package-guide examples also executed successfully. These counts establish coverage of the tested contracts, not validation of every legacy branch or optional backend.

The completed generative-learning work was validated separately in `torchlane`. A broader regression selection produced 190 passes; after the final API refinements, a focused selection produced 110 passes, including 43 generative cases plus existing MPI/HPC checks. The selections overlap and their counts are not additive. Both emitted only the two expected `BornMachine` deprecation warnings. The checks cover dense-versus-Pauli MMD values and gradients at nonunit bandwidths; Torch and Autograd; Angle and IQP circuits; finite-shot parameter shift; fixed conditioning and trainable scales; Fisher updates; wire order; direct sampling and drawings; and matched `default.qubit`, Lightning, Kokkos, mixed-state, and unpruned-PP paths. MPI routing used a serial stand-in, so this evidence does not replace an actual collective cluster run.

Table 14: Focused regression coverage for the revised RL policy layer. The full-probability backpropagation path is used as the reference where an alternative exact representation is compared.

Feature	Configurations	Checked invariant	Execution path
Majority sparse projectors	$A = 3$, six qubits	probabilities, normalization, and gradients	`lightning.qubit` adjoint in the full suite; targeted `lightning.kokkos` adjoint pass in `torchlane`
Multi-projector majority	$A = 4$, six qubits	all parameter gradients agree after disabling device VJP	`lightning.qubit` adjoint
Ancilla-inclusive projector	$A = 2$, five data qubits plus one ancilla	full-wire sparse operator and finite gradients	probability versus sparse-projector paths
Even majority block	$A = 2$, six-wire block	all wires used; ties decode to zero; gradients agree	probability versus sparse-projector paths
Recursive parity moments	$A \in \{3, 4\}$, six qubits	FWHT probabilities, normalization, and gradients	full probabilities versus logical expectations
PQDQN readouts	shared, per-action, parity, and majority	output shape, default/explicit bias, and finite gradients through circuit, input scales, and head	sequential and broadcast QNode paths
PQDQN Pauli backend	exact and truncated shared-Z models	finite circuit/input-scale/head gradients, leading batch shape, target copying, drawing, and explicit device guards	QASP `lightning.pp`/backprop versus Lightning adjoint and `default.qubit` reference
Preset feature supports	parity/majority $a \in \{2, 4\}$	identity removal, term deduplication, independent features, and `preset_actions` distinct from output actions	Walsh support extraction and explicit string checks
Historical compatibility	parity and majority projectors	explicit `preset_mode="projectors"` reconstructs decoder probabilities with legacy shapes	expectation versus Born reference
Deep Q-learning	deterministic discrete environments	replay sampling, Double-DQN target, target synchronization, evaluation isolation, and termination masking	PyTorch trainer tests
Configuration guards	invalid derivative/device combinations	incompatible contracts raise explicit errors	constructor validation
Windy navigation	$A \in \{3, 4\}$	Gymnasium API, bounded observations, truncation, and seeded trajectory equality	environment-only tests

The repository's notebooks cover feature maps, model construction, policy training, parameterized classifiers, QCBM/trainability experiments, IQP variance analyses, and the complete combinatorial-optimization family. The package guide additionally provides six executed smoke examples and a complete source inventory; the documentation audit records the 252-pass temporary- environment suite, its missing-Kokkos-plugin failure, the subsequent targeted Kokkos pass in `torchlane`, and executable-AST equivalence for the docstring pass. Neither notebook presence nor documentation coverage is counted as validation of a legacy branch. The executed CartPole notebook additionally preserves 3,000 episodic returns (2 policies, 5 seeds, and 300 episodes), ten run times, and eight fixed-state variance estimates as machine-readable CSV files. The executed PQDQN notebook adds 8,000 training episodes from 20 complete runs (four readouts and five trained seeds), 2,800 greedy evaluation episodes, per-run checkpoints and circuit metadata, and aggregate tables whose moving statistics and held-out means were independently recomputed from the raw records. A separate executed ten-qubit PQDQN notebook preserves 420 raw timing rows, 30 independently recomputed accuracy rows, matched states and inputs for five parameter seeds, compact and expanded circuit exports, hardware and dependency metadata, and source fingerprints. Its fixed forward/gradient workload adds no CartPole episodes and does not replace the four-qubit learning study. The companion observable-validation notebook checks finite gradients, the $a = 2$ and $a = 4$ support lists, default and overridden bias semantics, explicit historical

projector compatibility, and a complete DQN smoke path. The policy-efficiency notebook adds matched CartPole and Windy Target Navigation protocols, graph-IQP depth sweeps, automatic block diagnostics, parameter-copy assertions, and explicit correction notes for superseded experimental runs. The executed PQCBM notebook adds the complete three-qubit experiment from Section 7.4: 120 statevector and 120 matched unpruned-PP updates, 2,000 direct samples, saved loss/probability arrays, parameters, validation metadata, and editable Quantikz output. The `tests/tests_deucalion` drivers provide contextual-bandit and CartPole examples, parity actions, and scheduled cluster runs. Because notebooks can preserve outputs from different environments and development points, publication-quality numerical claims must be regenerated from a tagged release in a fresh environment rather than copied from exploratory cell output.

10.1 Recommended experiment record

For every reported result, we recommend recording:

1. the QASP version, Git commit, Python version, and complete dependency lock file;
2. random seeds for Python, NumPy, PyTorch, environment, and device where applicable;
3. feature map, ansatz, topology, layer count, parameter initialization, ordered measured wires, decoder family, majority blocks and unused decoder wires, observables, action or class count, classifier readout, reference class prior, and calibration temperature;
4. training method (probabilities, moments, or sparse projectors), device, shots, differentiation method, device-VJP and observable-batching settings, and hardware details;
5. optimizer, objective, parameter groups, learning rates, batch/rollout size, baseline, entropy coefficient, gradient clipping, data split, preprocessing, and reported evaluation metrics;
6. for value-based RL, the Pauli strings, feature matrix, preset mode, `preset_actions`, environment `n_actions`, explicit majority blocks, affine-bias choice, and resulting readout-parameter count; plus replay capacity, learning-start threshold, update and target-sync frequencies, Double-DQN choice, exploration schedule in frames, termination/truncation treatment, actual frame count, held-out reset seeds, and trained-seed unit of analysis;
7. for combinatorial optimization, the exact QUBO matrix and offset, Ising convention, algorithm variant, depth, mixer and initial state, warm-start source and clipping, recursive cutoff and elimination rules, PCE correlation order/alphabet/register size, training and result shots, decoded energy, solution probability, and a feasible exact or classical reference;
8. for Pauli propagation, k, τ, top-M, pruning interval, peak retained terms, and error against a feasible exact reference;
9. for Born-machine experiments, the ansatz, interface, initialization, target bit-matrix and any fixed conditioning vector; NLL versus MMD, bandwidth convention, mask mode/count/seed and estimator; minibatching, shots, independent finite-shot moment evaluation, PP truncation/pruning, natural-gradient Fisher and regularization choices, sampling backend, and probability-enumeration guards;
10. for HPC execution, the hardware-profile date, inferred dimensions and explicit memory hints, analytical plan, Slurm script, actual allocation, MPI world layout, and measured peak memory/runtime as separate fields;
11. wall-clock time separated into circuit construction, forward execution, gradient evaluation, and optimizer overhead.

10.2 Benchmark protocol for the release

A defensible performance figure should compare equivalent observables and gradients, not merely backend API calls. We propose the following release benchmark:

1. Generate seeded ring and all-to-all `AngleAnsatz` circuits across (n, L), with fixed local Z and pairwise ZZ observables.
2. Measure forward and forward-plus-backward latency after one untimed warm-up for `default.qubit`, `lightning.qubit`, `lightning.kokkos`, `default.tensor`, and multiple Pauli truncation budgets.
3. For approximate runs, report maximum and mean absolute expectation error and relative gradient error against the largest feasible exact state-vector instance.

4. Publish medians and interquartile ranges over at least 20 parameter draws, together with peak resident memory and retained Pauli-term counts.
5. Separate single-thread CPU, multi-thread CPU, GPU, and MPI results; never combine them in one scaling curve without explicit labels.

The ten-qubit microbenchmark in Section 3.3.3 supplies one machine-described point under this discipline. It is not the proposed scaling suite: broader circuit families, memory and retained-term measurements, more parameter draws, and independently repeated machine sessions remain future work. Keeping that distinction explicit prevents a local microbenchmark from becoming a release-wide performance guarantee.

The RL release benchmark should additionally preserve the following order of evidence:

1. verify probability and gradient equivalence of every specialized readout against the full-probability reference on small circuits;
2. construct parity and majority policy pairs from the same graph-IQP configuration, copy the complete initial state dictionary, and assert elementwise equality before training;
3. run the five-qubit proxy and ten-qubit cubic graph for $A \in \{3, 4\}$, depths $L \in \{1, 2, 3\}$, and a published seed list;
4. use identical environment and action-sampling seeds within every pair, and report random-policy and feedback-controller baselines;
5. publish per-seed returns, waypoint success rates, time to a declared criterion, wall-clock time, and aggregate uncertainty rather than only a smoothed mean curve.

This sequence distinguishes an implementation-equivalence claim from a policy optimization result and, in turn, from a simulator-performance claim.

11 Limitations and responsible interpretation

Qasp is research software at an alpha-stage release. The following boundaries are material.

- The package does not establish a quantum speedup, sample-complexity advantage, superior classification accuracy, or superior RL return. Those are properties of a fully specified algorithm, data-access model, baseline comparison, and execution regime.
- Generic state-vector simulation is exponential in qubit count. Kokkos, batching, and MPI improve execution capacity but do not remove worst-case exponential memory.
- HPC placement and memory numbers are analytical estimates from a dated machine profile, not measured peak-memory or runtime guarantees. Rendering a Slurm script neither submits it nor verifies that the requested PennyLane plugin and MPI stack are installed.
- Pauli propagation is efficient only when the active operator expansion remains controlled. Truncation introduces circuit- and parameter-dependent error; detached pruning masks yield piecewise gradients through retained terms.
- The current Pauli engines support CNOT, CZ, Hadamard, R_X, R_Y, R_Z, and one- or two-qubit `PauliRot` with XX, YY, or ZZ generators, and require Pauli expectation measurements.
- Full-probability Born readout scales as 2^m in measured qubits. Full moment reconstruction requires $2^m - 1$ nontrivial Z moments, although recursive parity reduces this to $2^{\lceil \log_2 A \rceil} - 1$ logical moments.
- Measuring only $A - 1$ sparse majority projectors reduces the number of returned observables, not the Hilbert-space dimension: the current CSR construction still has dimension $2^{n+n_{\mathrm{anc}}}$ and is an exact small- and medium-system route rather than a polynomial-time simulator guarantee.
- Sparse-projector training depends on the standard Lightning adjoint path with device-native VJPs disabled. This is a tested workaround for the current multi-projector stack and should be revalidated when the PennyLane-Lightning version changes.
- Automatic majority blocks are odd and nearly balanced, while explicit even blocks use tie-to-zero. The decoder choice can therefore bias action or class masses before learning; block membership, unused wires, and the reference prior must be reported.

- Named `PQDQN` presets use an exponentially sized Walsh transform to discover Pauli support and are currently limited to 16 measured wires and 2^{22} projector-by-basis or feature-matrix entries. The default exposes non-identity support terms independently; it does not reconstruct projector probabilities. Pauli-expectation readouts can use QASP's `lightning.pp` selector with Torch backpropagation, but batches currently execute sample by sample and truncated gradients can differ substantially from an exact state-vector reference. The ten-qubit microbenchmark shows that a faster small-k workload need not preserve the quantum-gradient direction. Without its classical head, bounded expectation features are generally a poor scale match for long-horizon action values.
- The completed CartPole Q-function comparison has five independently trained seeds per readout and no classical function-approximation baseline. Feature counts, bias, and head-parameter counts differ; the majority model succeeds on several seeds and fails on one. These variations and sub-475 held-out returns do not establish reliable solution, statistically demonstrated superiority, a general readout ranking, or quantum advantage. A saved model state is also not an exact trainer resume unless optimizer, replay, and RNG states are preserved.
- A probability-readout classifier requires the complete 2^m Born vector on its measured wires in the current implementation. An expectation classifier avoids that output dimension, but its sigmoid/softmax `predict_proba` values are classical calibration of observable scores rather than directly measured Born probabilities.
- The bundled classifier notebook is an integration and gradient test on deliberately small preprocessed subsets. It is not a competitive dataset benchmark, and the package does not presently implement cross-validation, early stopping, uncertainty calibration, or shot-aware confidence intervals in `ClassifierTrainer`.
- QAOA, warm-start QAOA, RQAOA, LR-QAOA, and PCE are heuristic solvers; the module does not guarantee an optimum or a better approximation ratio than a classical method. `QUBO.brute_force` is an exponential small-instance reference, not a scalable baseline.
- Exact QAOA probabilities require a 2^n output vector, while finite-shot objectives and correlations are noisy. RQAOA inherits errors from every intermediate QAOA solve and finishes with an exact search exponential in the chosen cutoff.
- PCE compresses the number of qubits by assigning variables to Pauli correlations, but it does not by itself reduce the number of encoded observables or prove a runtime advantage. The current PCE outer loop is NumPy/SciPy based rather than end-to-end autodifferentiable, and its regularized soft-spin relaxation is a heuristic surrogate.
- The automatic warm start is a locally solved box-constrained continuous QUBO relaxation, and the default LR-QAOA increments are a configurable schedule rather than a universal optimum.
- PQCBM NLL, complete-probability diagnostics, model entropy, exact-mask MMD, and dense Fisher matrices have exponential output, enumeration, or parameter-space costs and are small-system routes. Sampled-mask MMD avoids enumerating outputs but does not make arbitrary circuits efficiently simulable. Truncated/pruned PP has no general polynomial-time or unbiasedness guarantee, and PP-trained models need a separately selected device for sampling. The supplied example has three qubits and one seed; it is an integration validation rather than a scaling result. The Fisher kernel likewise enumerates basis scores, can be singular at zero probabilities, and currently ignores its legacy normalization option.
- `UQC`, `LearnablePQCBorn`, `qasp.rl.trl`, and legacy policy weight setters are retained historical paths with documented setup or post-processing restrictions. Their presence does not imply validation in the current suite.
- `fourier_analysis` performs no Fourier transform, and the legacy generator/decomposition helpers are exploratory lists or closure sizes, not certified dynamical Lie algebras or executable decompositions.
- Hardware support is inherited from installed PennyLane plugins, but device noise, queueing, calibration drift, and shot allocation are not abstracted away by QASP.
- Some module-level RL utilities remain experimental and are not all re-exported from the top-level namespace. Users should pin a release and import experimental classes from their defining module.
- `WindyTargetNavigationEnv` is a controlled software benchmark, not a substitute for a diverse RL task suite or evidence of practical control advantage.

These limitations motivate, rather than weaken, the package's explicit configuration model: a study can state exactly which approximation, decoder, and derivative estimator produced a result.

12 Availability, extensibility, and research impact

The source is available at https://github.com/andre-sequeira10/QASP under the MIT license. Installation from a release checkout uses `python -m pip install -e .`; development dependencies are available through the `dev` extra. The declared runtime stack includes PennyLane, PyTorch, NumPy, SciPy, Gymnasium, TorchRL, TensorDict, Ray/RLlib, Matplotlib, and pandas.

The repository's package guide maps all 52 Python modules to their source contracts and includes verified smoke examples for circuits and gradients, Quantikz export, classifier training, fidelity kernels, QUBO evaluation, and PQDQN output shapes. The generative guide additionally records the complete PQCBM construction, objective, backend, sampling, drawing, and cluster contracts, together with the executed small-system validation. Its accompanying machine-readable documentation audit records coverage, source hashes, the executable-AST equivalence check, and the single optional-backend test limitation. Documentation completeness should not be read as evidence that every legacy branch or external plugin has been validated.

New circuit components implement a small protocol: a feature map is callable and declares its qubit/feature metadata; an ansatz is callable, references its optional feature map, and reports parameter shapes. New decoded-Born policies extend the basis-state-to-action map in `_PolicyDecoder`; classifier partitions extend `ClassProbabilityDecoder`. In both cases the existing scatter-add and sample-lookup paths inherit the same declared semantics. New Q-function readouts can provide explicit Pauli strings or derive a non-identity Pauli support from a decoder preset. The feature matrix C is the identity for shared and default preset features, while nested lists encode fixed per-action sums. `preset_actions` controls only the preset construction; the affine head still emits `n_actions` environment values. `PQDQN` preserves the common circuit and autodifferentiation boundary while `DeepQLearning` remains unaware of the chosen feature construction or whether Pauli expectations came from a state vector or QASP's differentiable Pauli engine. New optimization algorithms can consume `QUBO.to_ising` and return the shared `OptimizationResult`; new QAOA mixers reuse the package's weighted Pauli-word notation, while new compressed encodings can reuse `CircuitBuilder` and the optimization drawing dispatch. Specialized exact policy execution may add a corresponding moment or projector construction, but it must be validated against `training_method="probs"`. New execution strategies can consume the builder's callable circuit or generated tape. This separation makes it possible to add a backend without rewriting task training, or add a decoder without rewriting circuit construction.

The immediate research impact is methodological: QASP consolidates configurations and implementations that would otherwise be copied across notebooks for QRL, quantum kernels, supervised classification, generative learning, combinatorial optimization, and trainability studies. Its reusable decoders and exact-vs-approximate execution paths support controlled ablations of where expressivity, gradient concentration, and simulation cost enter a quantum learning model. Concrete impact claims—external adopters, released benchmark datasets, archived experiment artifacts, and papers enabled by the package—should be added as they become citable: **[TODO: list publications, external users, or archived datasets enabled by QASP, if available]**.

13 Conclusion

QASP organizes the full path from classical data to differentiable quantum model and learning objective. Its central abstraction is not a new circuit language, but a reproducible connection between feature maps, ansatz templates, label or action semantics, measurement representations, execution backends, and gradient estimators. The revised RL layer makes that separation concrete: one vectorized decoder governs probability aggregation and sampled actions; recursive parity has an exact logical-moment route; majority–modulo has an exact sparse-projector route; and incompatible device contracts fail explicitly. The value-based path adds differentiable Pauli-readout Q-functions, decoder-support feature presets, configurable affine bias, and a replay-based Double-DQN trainer. Its corrected five-seed CartPole study demonstrates why the exact feature list, bias, head size, and trained-seed variability must be reported: per-action sums have the highest observed mean, shared features are more stable across these seeds, majority learns on several seeds but fails on one, and parity does not learn useful control under the stated budget. These observations do not support a general ranking. The kernel layer adds unified projector-configurable fidelity, projected expectation features, centered alignment, and Fisher geometry. The classifier layer then reuses the same construction for Born-probability or expectation-score prediction, explicit class decoders, supervised losses, training, and evaluation. The generative layer adds a configuration-driven PQCBM with probability NLL, exact or sampled Pauli-expectation MMD, direct device sampling, optional conditioning, and matched statevector/Pauli-propagation execution. Its three-qubit example demonstrates numerical equivalence at the reported settings without implying a scaling advantage. The optimization layer adds one QUBO/Ising contract for standard, multi-angle, warm-start, recursive, linear-ramp, and correlation-encoded solvers. Together with accelerated

conventional simulation and differentiable Pauli propagation, these modules support controlled comparisons across task families. The ten-qubit PQDQN microbenchmark illustrates the intended reporting standard: the observed $k = 2$ runtime reduction is presented together with its near-unit relative quantum-gradient error, while the slower $k = 3$ result rules out a blanket backend ranking. The public drawing path exports both interactive Matplotlib views and publication-ready Quantikz source from the circuit objects that are actually executed. By making decoder semantics, projector locality, readout contracts, and approximation budgets first-class configuration, QASP supports matched studies in which only the scientifically intended factor changes. The next priorities are a tagged benchmark suite with accuracy–runtime and learning curves, broader unit coverage, stabilized public exports, and archived release artifacts.

Author contributions

André Sequeira: conceptualization, software, methodology, investigation, and manuscript review. **[TODO: revise the CRediT statement and add all software/paper contributors before submission]**.

Acknowledgements

[TODO: add funding, institutional computing resources, and contributor acknowledgements; write "None" if not applicable].

AI-use disclosure

OpenAI Codex was used to assist with source-code inspection, literature discovery, diagram design, and preparation of an initial manuscript draft. It was not treated as a source of experimental evidence. All scientific claims, examples, references, and generated text must be checked by the named authors against the tagged software release before submission; responsibility for the final manuscript remains with the authors.

Data and software availability

QASP source code, examples, the complete package guide, and the machine-readable documentation audit are available at https://github.com/andre-sequeira10/QASP. The archival DOI for the exact release described by this paper is **[TODO: create a Zenodo or equivalent archive for the release and insert its DOI]**. No new external dataset is required for the software description. The corrected feature/bias PQDQN CartPole directory preserves its manifest, all raw training and evaluation records, seed-level summaries, checkpoints, and editable circuit figures; historical projector-reconstruction and aborted partial runs are kept separately and are not combined with it. The separate ten-qubit simulator directory preserves raw timings, per-seed timing medians, accuracy rows, independently recomputed summaries, matched parameter/input checkpoints, hardware and verification metadata, source fingerprints, and editable circuit and plot exports. The PQCBM example directory preserves its configuration, summary and machine-readable validation, loss histories, target/model/sample distributions, trained parameters, 2,000 sampled bit strings, and editable Quantikz source. These and the remaining reproducible benchmark outputs should be attached to the archived release.

References

[1] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9(1):4812, 2018. doi: 10.1038/s41467-018-07090-4. URL https://doi.org/10.1038/s41467-018-07090-4.

[2] Martin Larocca, Piotr Czarnik, Kunal Sharma, Gopikrishnan Muraleedharan, Patrick J. Coles, and M. Cerezo. Diagnosing barren plateaus with tools from quantum optimal control. *Quantum*, 6:824, 2022. doi: 10.22331/q-2022-09-29-824. URL https://quantum-journal.org/papers/q-2022-09-29-824/.

[3] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shahnawaz Ahmed, Vishnu Ajith, M. Sohaib Alam, Guillermo Alonso-Linaje, B. AkashNarayanan, Ali Asadi, et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*, 2018. doi: 10.48550/arXiv.1811.04968. URL https://arxiv.org/abs/1811.04968.

[4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[5] Damian S. Steiger, Thomas Häner, and Matthias Troyer. Projectq: An open source software framework for quantum computing. *Quantum*, 2:49, 2018. doi: 10.22331/q-2018-01-31-49. URL https://quantum-journal.org/papers/q-2018-01-31-49/.

[6] Ryan LaRose, Andrea Mari, Sarah Kaiser, Peter J. Karalekas, Andre A. Alves, Piotr Czarnik, Mohamed El Mandouh, Max H. Gordon, Yousef Hindy, Aaron Robertson, et al. Mitiq: A software package for error mitigation on noisy quantum computers. *Quantum*, 6:774, 2022. doi: 10.22331/q-2022-08-11-774. URL https://quantum-journal.org/papers/q-2022-08-11-774/.

[7] Shi-Xin Zhang, Jonathan Allcock, Zhou-Quan Wan, Shuo Liu, Jiace Sun, Hao Yu, Xing-Han Yang, Jiezhong Qiu, Zhaofeng Ye, Yu-Qin Chen, et al. Tensorcircuit: A quantum software framework for the NISQ era. *Quantum*, 7:912, 2023. doi: 10.22331/q-2023-02-02-912. URL https://quantum-journal.org/papers/q-2023-02-02-912/.

[8] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. doi: 10.48550/arXiv.2407.17032. URL https://arxiv.org/abs/2407.17032.

[9] Albert Bou, Matteo Bettini, Sebastian Dittert, Vikash Kumar, Shagun Sodhani, Xiaomeng Yang, Gianni De Fabritiis, and Vincent Moens. TorchRL: A data-driven decision-making library for PyTorch. *arXiv preprint arXiv:2306.00577*, 2023. doi: 10.48550/arXiv.2306.00577. URL https://arxiv.org/abs/2306.00577.

[10] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. Data re-uploading for a universal quantum classifier. *Quantum*, 4:226, 2020. doi: 10.22331/q-2020-02-06-226. URL https://quantum-journal.org/papers/q-2020-02-06-226/.

[11] Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99(3):032331, 2019. doi: 10.1103/PhysRevA.99.032331. URL https://doi.org/10.1103/PhysRevA.99.032331.

[12] James C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, 37(3):332–341, 1992. doi: 10.1109/9.119632. URL https://doi.org/10.1109/9.119632.

[13] James Stokes, Josh Izaac, Nathan Killoran, and Giuseppe Carleo. Quantum natural gradient. *Quantum*, 4:269, 2020. doi: 10.22331/q-2020-05-25-269. URL https://quantum-journal.org/papers/q-2020-05-25-269/.

[14] Enrico Fontana, Manuel S. Rudolph, Ross Duncan, Ivan Rungger, and Cristina Cîrstoiu. Classical simulations of noisy variational quantum circuits. *npj Quantum Information*, 11:84, 2025. doi: 10.1038/s41534-024-00955-1. URL https://doi.org/10.1038/s41534-024-00955-1.

[15] Manuel S. Rudolph, Tyson Jones, Yanting Teng, Armando Angrisani, and Zoë Holmes. Pauli propagation: A computational framework for simulating quantum systems. *arXiv preprint arXiv:2505.21606*, 2025. doi: 10.48550/arXiv.2505.21606. URL https://arxiv.org/abs/2505.21606.

[16] Sofiene Jerbi, Casper Gyurik, Simon C. Marshall, Hans J. Briegel, and Vedran Dunjko. Parametrized quantum policies for reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, pages 28362–28375, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/eec96a7f788e88184c0e713456026f3f-Abstract.html.

[17] Nico Meyer, Daniel D. Scherer, Axel Plinge, Christopher Mutschler, and Michael J. Hartmann. Quantum policy gradient algorithm with optimized action decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 24592–24613. PMLR, 2023.

[18] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992. doi: 10.1007/BF00992696. URL https://doi.org/10.1007/BF00992696.

[19] Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019. doi: 10.48550/arXiv.1910.00177. URL https://arxiv.org/abs/1910.00177.

[20] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347. URL https://arxiv.org/abs/1707.06347.

[21] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. doi: 10.1038/nature14236.

[22] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.

[23] Maria Schuld and Nathan Killoran. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters*, 122(4):040504, 2019. doi: 10.1103/PhysRevLett.122.040504. URL https://doi.org/10.1103/PhysRevLett.122.040504.

[24] Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567:209–212, 2019. doi: 10.1038/s41586-019-0980-2. URL https://doi.org/10.1038/s41586-019-0980-2.

[25] Hsin-Yuan Huang, Michael Broughton, Masoud Mohseni, Ryan Babbush, Sergio Boixo, Hartmut Neven, and Jarrod R. McClean. Power of data in quantum machine learning. *Nature Communications*, 12:2631, 2021. doi: 10.1038/s41467-021-22539-9. URL https://doi.org/10.1038/s41467-021-22539-9.

[26] Corinna Cortes, Mehryar Mohri, and Afshin Rostamizadeh. Algorithms for learning kernels based on centered alignment. *Journal of Machine Learning Research*, 13(28):795–828, 2012. URL https://www.jmlr.org/papers/v13/cortes12a.html.

[27] Yudai Suzuki, Hideaki Kawaguchi, and Naoki Yamamoto. Quantum fisher kernel for mitigating the vanishing similarity issue. *Quantum Science and Technology*, 9(3):035050, 2024. doi: 10.1088/2058-9565/ad4b97. URL https://doi.org/10.1088/2058-9565/ad4b97.

[28] Erik Recio-Armengol, Shahnawaz Ahmed, and Joseph Bowles. Train on classical, deploy on quantum: scaling generative quantum machine learning to a thousand qubits, 2025. URL https://arxiv.org/abs/2503.02934.

[29] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014. doi: 10.48550/arXiv.1411.4028. URL https://arxiv.org/abs/1411.4028.

[30] Daniel J. Egger, Jakub Marecek, and Stefan Woerner. Warm-starting quantum optimization. *Quantum*, 5:479, 2021. doi: 10.22331/q-2021-06-17-479. URL https://quantum-journal.org/papers/q-2021-06-17-479/.

[31] Sergey Bravyi, Alexander Kliesch, Robert Koenig, and Eugene Tang. Obstacles to state preparation and variational optimization from symmetry protection. *Physical Review Letters*, 125(26):260505, 2020. doi: 10.1103/PhysRevLett.125.260505. URL https://doi.org/10.1103/PhysRevLett.125.260505.

[32] Rebekah Herrman, Phillip C. Lotshaw, James Ostrowski, Travis S. Humble, and George Siopsis. Multi-angle quantum approximate optimization algorithm. *Scientific Reports*, 12:6781, 2022. doi: 10.1038/s41598-022-10555-8. URL https://doi.org/10.1038/s41598-022-10555-8.

[33] Marco Sciorilli, Lucas Borges, Taylor L. Patti, Diego García-Martín, Giancarlo Camilo, Anima Anandkumar, and Leandro Aolita. Towards large-scale quantum optimization solvers with few qubits. *Nature Communications*, 16:476, 2025. doi: 10.1038/s41467-024-55346-z. URL https://doi.org/10.1038/s41467-024-55346-z.

[34] J. A. Montañez-Barrera and Kristel Michielsen. Toward a linear-ramp QAOA protocol: evidence of a scaling advantage in solving some combinatorial optimization problems. *npj Quantum Information*, 11:131, 2025. doi: 10.1038/s41534-025-01082-1. URL https://doi.org/10.1038/s41534-025-01082-1.

A Configuration and API reference

Table 15: Principal public entry points and their role. Experimental module-level classes are listed separately in the text.

Entry point	Purpose
`CircuitBuilder`	Build a plain quantum function, bind its backend/QNode with `compile`, and allocate trainable weights through a Torch layer.
`AngleEmbedding`, `NeuralEmbedding`	Direct angle encoding and bias-free trainable linear-projection encoding; the neural reduction is currently single-sample.
`AngleAnsatz`, `IQPAnsatz`	Maintained general Pauli-rotation and commuting IQP-style circuit families; `UQC` is retained only as a historical function.
`create_model`, `draw_model`	Resolve model configurations and draw the executed circuit as Matplotlib output or editable Quantikz.
`circuit_to_tikz`, `save_tikz`	Export a builder, tape, or callable circuit as styled Quantikz source, including symbolic parameters and repeated-layer compression.
`PQCBorn`, `PQCBornVariance`	Decoded-Born policies; the factory currently dispatches `"PQCBorn"` to the enhanced vectorized implementation with probability, moment, and sparse-projector paths.
`PQCSoftmax`	Pauli-expectation logits with an optional biased affine head; use `forward`, because the legacy `action_probs` bypasses head, scaling, and PP execution.
`LearnablePQCBorn`	Historical adaptive-measurement prototype with known setup/post-processing limitations.
`PQDQN`, `create_qfunction`	Differentiable action-value model and configuration factory with shared, per-action, parity, or majority Pauli features, configurable affine bias, preset size independent of output actions, and optional QASP `lightning.pp` execution.
`draw_qfunction`	Draw the executable Q-function circuit as Matplotlib output or editable, compressed Quantikz.
`DeepQLearning`	Replay-based DQN/Double-DQN training with injectable optimizers, configurable replay/update/target/exploration schedules, observation transforms, gradient safeguards, and isolated greedy evaluation.
`REINFORCE`, `Reinforce`	Distinct episode-based policy-gradient implementations with different normalization, baselines, parameter groups, and stopping contracts.
`PPO`, `TorchRLTrainer`	Maintained clipped-PPO/GAE utility and an optional-version-dependent TensorDict/TorchRL adapter; `qasp.rl.trl` is legacy.
`BanditAWRConfig`, `ContextualBanditTrainer`	Advantage-weighted one-step training.
`WindyTargetNavigationEnv`	Seeded six-feature waypoint navigation with three or four discrete thrust actions.
`FastTruncatedPauliProp`	Non-differentiable packed-key expectation evaluator.
`FidelityKernel`	Unified global/localized overlap kernel configured by zero-state projector labels.
`ProjectedKernel`	Weighted Gaussian kernel over selected quantum expectation features.
`create_kernel`, `draw_kernel`	Resolve continuous-input kernel configurations and draw their evaluated circuits as Matplotlib or Quantikz.
`KernelTargetAlignment`, `align_kernel`	Differentiate centered kernel–target alignment through trainable circuit parameters.
`QFKernel`	Small-system regularized Fisher-score Gram matrix with exhaustive basis enumeration and a currently unused normalization flag.
`PQCClassifier`, `QuantumClassifier`	Binary or multiclass parameterized classifiers with Born-probability or expectation-score readout.
`create_classifier`, `draw_classifier`	Resolve supervised model configurations and draw classifier circuits as text, Matplotlib, or Quantikz.
`ClassProbabilityDecoder`	Aggregate basis probabilities or decode samples with contiguous, parity, or majority class partitions.
`ClassifierLoss`, `ClassifierTrainer`	Differentiable supervised objectives and minibatch PyTorch training/evaluation.
`classification_metrics`	Accuracy, confusion-matrix, precision/recall/F1, ROC-AUC, PR-AUC, log-loss, and Brier-score utilities.
`PQCBM`, `create_qcbm`	Primary Born-machine model and configuration factory with Torch/Autograd, NLL or Pauli-expectation MMD, optional fixed conditioning, direct device sampling, and tested small-system natural gradients.
`PauliMMD`	Exact or sampled Gaussian-kernel mixtures over Pauli-Z masks, bandwidth mixtures, empirical or data-U-statistic targets, and independent finite-shot model factors.
`draw_qcbm`	Draw a configured or constructed Born machine as text, Matplotlib, or editable Quantikz; `BornMachine` is retained as a deprecated compatibility subclass.
`QUBO`, `PauliTerm`	Build binary objectives, convert exactly to Ising/Pauli form, evaluate assignments, and obtain small-instance references.
`QAOA`, `WarmStartQAOA`	Standard or multi-angle QAOA with custom mixers, gradients, analytic/finite-shot execution, and relaxation-based warm starts.
`RecursiveQAOA`, `RQAOA`	Correlation-driven variable elimination, exact cutoff solve, and reverse reconstruction.
`LRQAOA`, `LinearRampQAOA`	Fixed opposing linear schedules without classical parameter optimization.
`PauliCorrelationEncoding`, `PCE`	Correlation-compressed variables, QASP ansatz dictionaries, soft-spin optimization, and decoding.
`create_optimizer`, `OptimizationResult`	Reusable SciPy optimizers and common solver outputs.
`draw_optimization`, `draw_qaoa`, `draw_pce`	Draw optimization circuits as text, Matplotlib, or publication-ready Quantikz.
`policy_variance`	Sampling-based variance of a selected policy output under controlled parameter/state initialization.
`fourier_analysis`	Legacy helper that currently performs no Fourier transform; not a spectral-analysis claim.
`barren_plateau_diagnoser`	Cross-module cost-concentration variance with fixed or resampled embedded states.
`compute_DLA_generators`	Legacy exploratory Lie-closure sizes from heuristic generator lists, not returned generators or a certified circuit algebra.
`inspect_workload`, `MemoryHints`, `plan_deucalion`	Side-effect-free workload inspection and analytical memory/placement planning.
`create_deucalion_job`, `render_slurm`, `ExecutionContext`	Generate (but do not submit) Slurm scripts and run MPI-aware callbacks with rank-consistent experiment seeds.

B Ancilla parity policy example

For binary actions, recursive parity may be materialized as a circuit-level readout instead of computed from the full basis distribution. Setting `parity_cnot_decomposition=True` makes the factory allocate one ancilla, append the required data-to-ancilla CNOT network, and measure the logical parity on that ancilla. This changes the execution contract from a 2^m probability vector to an expectation-based readout while preserving the intended action semantics.

```python
from pathlib import Path
from qasp import create_model, draw_model
import torch

n_qubits = 4
ancilla_config = {
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 1, "rotations": "ZY",
        "reuploading": False,
        "include_imprimitive": True, "imprimitive": "CZ",
        "imprimitive_pattern": "ring",
        "include_final_imprimitive": False,
        "include_feature_map_first": True,
        "include_hadamards_initial": True,
    },
    "feature_map": "AngleEmbedding",
    "feature_map_config": {
        "features": 4, "rotations": "Y", "input_scaling": False,
    },
    "n_actions": 2,
    "policy": "parity-like",
    "measurement_qubits": [0, 1, 2, 3],
    "training_method": "expectation",
    "parity_cnot_decomposition": True,
    "init": "small_normal",
    "device": "default.qubit",
    "diff_method": "backprop",
}
ancilla_policy = create_model("PQCBorn", ancilla_config,
                              n_qubits=n_qubits)
tikz = draw_model(
    model=ancilla_policy, inputs=torch.zeros(4), num_inputs=4,
    output="tikz",
    tikz_options={"standalone": True,
                  "symbolic_parameters": True,
                  "wire_labels": {4: "a_0"}},
)
Path("figures/rl_ancilla_parity.tex").write_text(tikz)
```

Listing 24: Building and drawing a binary ancilla-parity policy. The generated circuit is Figure 10a.

C Exact majority–modulo policy example

```python
from copy import deepcopy
from pathlib import Path
from qasp import create_model, draw_model
import torch

majority_config = {
    "ansatz": "AngleAnsatz",
    "ansatz_config": {
        "n_layers": 1, "rotations": "ZY", "reuploading": False,
        "include_imprimitive": True,
        "imprimitive": "CZ", "imprimitive_pattern": "ring",
        "include_final_imprimitive": False,
        "include_feature_map_first": True,
        "include_hadamards_initial": True,
    },
    "feature_map": "AngleEmbedding",
    "feature_map_config": {
        "features": 6, "rotations": "Y", "input_scaling": False,
    },
```

```
    "n_actions": 3,
    "policy": "majority-modulo",
    "measurement_qubits": [0, 1, 2, 3, 4, 5],
    "majority_blocks": [[0, 1, 2], [3, 4, 5]],
    "strict_majority_blocks": True,
    "training_method": "probs",
    "device": "default.qubit",
    "diff_method": "backprop",
    "shots": None,
    "init": "small_normal",
}
policy = create_model("PQCBorn", majority_config, n_qubits=6)

tikz = draw_model(
    model=policy, inputs=torch.zeros(6), num_inputs=6,
    output="tikz",
    tikz_options={"standalone": True,
                  "symbolic_parameters": True,
                  "compress": True},
)
Path("figures/rl_majority_modulo.tex").write_text(tikz)

# Same action semantics, but evaluate A - 1 exact projectors.
projector_config = deepcopy(majority_config)
projector_config.update({
    "training_method": "sparse_projectors",
    "device": "lightning.qubit",
    "diff_method": "adjoint",
})
training_policy = create_model("PQCBorn", projector_config, n_qubits=6)
```

Listing 25: Drawing the probability-mode majority–modulo circuit and constructing its exact sparse-projector training variant.

The two majority bits encode four logical values; the final modulo operation maps them into three actions. The sparse-projector path evaluates only $A - 1 = 2$ projectors and reconstructs the last action probability as one minus their sum. For this mode, QASP includes identities on unmeasured wires and ancillas so each sparse operator spans the full device wire set.

D Exact multi-action recursive parity

The expectation path below constructs the three nonidentity products of the $k = 2$ logical parity observables needed for four actions. The circuit returns those moments, the differentiable FWHT reconstructs the four logical-code probabilities, and modulo aggregation is the identity because $A = 2^k$.

```
import torch
from qasp import create_model

parity4 = create_model("PQCBorn", {
    "ansatz": "IQPAnsatz",
    "ansatz_config": {
        "n_layers": 2,
        "imprimitive_pattern": "ring",
        "reuploading": True,
        "include_single_qubit": True,
    },
    "feature_map": "NeuralEmbedding",
    "feature_map_config": {"features": 6, "rotations": "Z"},
    "n_actions": 4,
    "policy": "parity-like",
    "measurement_qubits": list(range(6)),
    "training_method": "expectation",
    "parity_cnot_decomposition": False,
    "device": "lightning.qubit",
    "diff_method": "adjoint",
    "shots": None,
    "init": "normal_0_1",
}, n_qubits=6)

observation = torch.zeros(6)
probabilities = parity4(observation)
loss = -torch.log(probabilities[2])
```

```
loss.backward()
assert probabilities.shape == (4,)
assert any(
    parameter.grad is not None
    and torch.isfinite(parameter.grad).all()
    for parameter in parity4.parameters()
)
```

Listing 26: Four-action recursive parity using exact logical moments rather than a 2^6 probability vector.

For $A = 3$, the construction is identical except that the logical code 11_2 is reduced modulo three and contributes to action zero. This convention is also used by direct sample decoding, so training and deployment remain consistent for non-power-of-two action sets.

E Windy navigation with an exact majority policy

Listing 27 uses six observation features, two majority bits, and three actions. The full Hilbert-space probabilities are never returned by the policy: two sparse action projectors are evaluated and the third probability is the residual. Passing the environment object directly to `REINFORCE` uses the same Gymnasium interface as a registered environment name.

```
from qasp import create_model, REINFORCE, WindyTargetNavigationEnv

n_qubits = 6
policy = create_model("PQCBorn", {
    "ansatz": "IQPAnsatz",
    "ansatz_config": {
        "n_layers": 2,
        "imprimitive_pattern": "ring",
        "reuploading": True,
        "include_single_qubit": True,
    },
    "feature_map": "NeuralEmbedding",
    "feature_map_config": {"features": 6, "rotations": "Z"},
    "n_actions": 3,
    "policy": "majority-modulo",
    "measurement_qubits": list(range(6)),
    "majority_blocks": [[0, 1, 2], [3, 4, 5]],
    "strict_majority_blocks": True,
    "training_method": "sparse_projectors",
    "device": "lightning.qubit",
    "diff_method": "adjoint",
    "shots": None,
    "init": "normal_0_1",
}, n_qubits=n_qubits)

environment = WindyTargetNavigationEnv(
    n_actions=3, max_episode_steps=180, n_waypoints=3
)
agent = REINFORCE(
    policy=policy,
    env_name=environment,
    logits=False,
    gamma=0.99,
    lr=2e-2,
    batch_episodes=5,
    normalize_mode="none",
    normalize_returns=True,
    entropy_coef=3e-2,
    baseline_type="ema",
    max_grad_norm=0.5,
    max_steps=180,
)
returns = agent.train(num_episodes=600)
```

Listing 27: Three-action majority–modulo training on the bundled windy-navigation environment.

F Release checklist

The following items should be completed before the manuscript and repository are made public:

1. replace author, affiliation, funding, impact, and archive placeholders;
2. tag the exact commit as v0.1.0 and update `pyproject.toml` URLs to the public repository;
3. stabilize public exports for module-level PPO and contextual-bandit APIs, and make the distinct `REINFORCE`/`Reinforce` names unambiguous;
4. run the complete test suite in clean Python 3.10, 3.11, and 3.12 environments, install and exercise every advertised optional backend, and retain the current environment-specific failure record rather than hiding it;
5. rerun the PQCBM backend matrix and executed notebook against the final release tag, retain the exact MMD/dense-kernel and PP/statevector equivalence records, and execute the supplied collective smoke test on the target MPI installation before making cluster claims;
6. consolidate the TorchRL implementations, retire or repair `qasp.rl.trl`, and validate the maintained `TorchRLTrainer` against pinned TorchRL/TensorDict versions;
7. repair or formally deprecate `UQC`, `LearnablePQCBorn`, legacy policy weight setters, and the misleading Fourier/DLA helper names;
8. add exact-vs-approximate forward and gradient tests for both Pauli engines across all supported gates;
9. publish the benchmark protocol from Section 10.2 with machine-readable results;
10. freeze an environment/lock file and archive the release with a DOI;
11. render every example in the paper against the tag and record the command in continuous integration;
12. run the optimization notebook and focused tests against the release tag, preserve small-instance exact energies, and benchmark every heuristic against declared classical references before making performance claims;
13. select a venue and adapt the front matter and length: the present manuscript is a full preprint, while a Journal of Open Source Software submission would require a substantially shorter paper and venue-specific metadata.